



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

MACROAREA DI INGEGNERIA

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA
INFORMATICA

A.A 2025/2026

Tesi di laurea magistrale

Scheduling adattivo carbon-aware in ambienti serverless:
gestione del trade-off energia-accuratezza tramite varianti di
funzione

RELATORI:

Prof.ssa Valeria Cardellini
Prof. Gabriele Russo Russo

CANDIDATO:

Lorenzo Grande
0350212

Indice

Abstract	5
Ringraziamenti	6
1 Introduzione	7
1.1 Contesto e motivazioni	7
1.1.1 L’impatto ambientale dell’elaborazione e la carbon intensity . .	8
1.1.2 Approximate Computing come leva per il trade-off energia-accuratezza	9
1.1.3 Il problema della granularità nella misurazione energetica	11
1.1.4 Verso uno scheduling carbon-aware	11
1.2 Contributi e obiettivi della tesi	13
1.3 Struttura della tesi	15
2 Stato dell’arte	17
2.1 Il monitoraggio dell’energia software	17
2.1.1 Misuratori hardware del consumo energetico	19
2.1.2 Misuratori software e modelli di stima	23
2.2 Elaborazione serverless sostenibile	27
2.2.1 Scheduling energeticamente consapevole nelle piattaforme FaaS	28

2.2.1.1	Scheduling per consolidamento energetico con vincoli di deadline	30
2.2.1.2	Scheduling energy-aware in ambienti FaaS federati ed eterogenei	32
2.2.2	Approximate Computing come leva applicativa per il trade-off energia-accuratezza	35
2.2.2.1	Generazione automatica di varianti energeticamente efficienti	37
2.2.2.2	Controllo runtime del trade-off energia-accuratezza . .	39
2.2.3	Carbon-aware computing	42
2.2.3.1	Temporal shifting su scala industriale: il Carbon-Intelligent Computing System	44
2.2.3.2	Spatial shifting carbon-aware per funzioni serverless . .	47
2.3	Il paradigma FaaS in ambienti cloud-edge	49
2.3.1	La piattaforma Serverledge	50
2.4	Posizionamento del contributo	51
3	Architettura del Sistema	53
3.1	Integrazione di Kepler per il monitoraggio energetico	53
3.1.1	Funzionamento di Kepler	54
3.1.2	Infrastruttura di supporto	55
3.2	Il modello di funzione logica con varianti	57
3.2.1	Struttura della funzione e profilo energetico	57
3.2.1.1	Determinazione dei profili energetici iniziali	58
3.2.2	Il modello di output e la classificazione dell'accuratezza	61

3.2.3	Registrazione e indicizzazione delle varianti in etcd	63
3.2.3.1	Convenzione di denominazione	64
3.2.3.2	Struttura delle chiavi in etcd	64
3.3	Misurazione e aggiornamento adattivo dei profili energetici	65
3.3.1	Tracciamento dei container e conteggio delle invocazioni	66
3.3.2	Il worker periodico di raccolta	67
3.3.3	Attribuzione dell'energia per invocazione e gestione del lag . . .	68
3.3.4	Aggiornamento del profilo tramite media mobile esponenziale . .	69
3.3.5	Archiviazione su InfluxDB	70
3.4	Esplorazione LCB per il raffinamento delle stime energetiche	70
3.4.1	Il principio di ottimismo sotto incertezza	71
3.4.2	Proprietà di convergenza	72
4	Lo scheduler carbon-aware	73
4.1	La carbon intensity come segnale decisionale	73
4.1.1	La funzione sigmoide calibrata per zona	74
4.2	La frontiera di Pareto nello spazio energia-errore	76
4.3	Scalarizzazione e selezione della variante	78
4.3.1	Normalizzazione e funzione obiettivo	78
4.3.2	Analisi delle soglie di transizione	79
4.4	Formalizzazione della logica decisionale	80
5	Risultati Sperimentali	83
5.1	Setup sperimentale	83
5.1.1	Funzioni utilizzate	84
5.1.2	Strategie di confronto	87

5.2	Protocollo di valutazione	87
5.3	Esperimento 1: calibrazione europea della carbon intensity	90
5.3.1	Obiettivo	90
5.3.2	Configurazione	90
5.3.3	Risultati	91
5.3.4	Interpretazione	95
5.4	Esperimento 2: calibrazione locale della carbon intensity	97
5.4.1	Obiettivo	97
5.4.2	Configurazione	97
5.4.3	Risultati	98
5.4.4	Interpretazione	99
5.5	Esperimento 3: comportamento con flusso di arrivo variabile	100
5.5.1	Obiettivo	100
5.5.2	Configurazione	101
5.5.3	Risultati	101
5.5.4	Interpretazione	102
5.6	Discussione generale	103
6	Conclusioni	105
	Elenco delle figure	106
	Elenco dei codici	110
	Elenco degli algoritmi	111

Abstract

Ringraziamenti

Capitolo 1

Introduzione

1.1 Contesto e motivazioni

Negli ultimi anni si è assistito a una progressiva trasformazione dell'architettura dei sistemi distribuiti: servizi e applicazioni, tradizionalmente eseguiti in data center centralizzati, vengono sempre più frequentemente spostati verso il bordo della rete. Questa tendenza, nota come *edge computing*, nasce dalla necessità di ridurre la latenza percepita dagli utenti, supportare applicazioni real-time e limitare il traffico verso il cloud [1]. Il paradigma emergente non è più quello di un cloud monolitico, ma di un *continuum Edge-Cloud*, in cui risorse computazionali eterogenee e geograficamente distribuite collaborano per soddisfare i requisiti applicativi. In tale contesto, l'elaborazione può essere collocata dinamicamente lungo la catena edge-cloud in funzione delle condizioni di rete, della disponibilità di risorse e dei vincoli applicativi.

Serverless Computing e FaaS nel continuum Edge-Cloud. Parallelamente all'evoluzione verso l'edge, il paradigma Serverless Computing ha acquisito crescente rilevanza come modello di programmazione cloud-native. Nel modello serverless, lo sviluppatore distribuisce codice senza occuparsi della gestione dell'infrastruttura sottostante, che viene completamente delegata al provider [2]. Il modello di fatturazione

pay-as-you-go e la scalabilità automatica hanno contribuito alla sua ampia adozione nell'industria.

All'interno dell'ecosistema serverless, il modello Function-as-a-Service (FaaS) rappresenta la forma più diffusa di esecuzione event-driven. In una piattaforma FaaS, l'unità fondamentale di esecuzione è una funzione stateless, invocata in risposta a eventi o richieste e tipicamente eseguita all'interno di container o ambienti isolati equivalenti. L'estensione del paradigma FaaS al continuum Edge-Cloud risulta particolarmente promettente: eseguire funzioni direttamente su nodi edge consente di ridurre la latenza di risposta e migliorare la qualità del servizio percepita. Recenti lavori hanno mostrato come architetture FaaS decentralizzate possano abilitare l'esecuzione distribuita di funzioni tra edge e cloud, introducendo meccanismi di offloading verticale e orizzontale [3].

1.1.1 L'impatto ambientale dell'elaborazione e la carbon intensity

La crescente diffusione di infrastrutture computazionali distribuite pone con urgenza il problema del loro impatto ambientale. L'elaborazione informatica è oggi responsabile di una quota significativa delle emissioni globali di CO₂, e la crescita prevista dei carichi di lavoro, in particolare nel continuum edge-cloud, rende la sostenibilità energetica una dimensione imprescindibile nella progettazione delle piattaforme di esecuzione [4].

Tuttavia, ridurre il consumo energetico di una singola esecuzione rappresenta una condizione necessaria ma non sufficiente per contenere l'impatto ambientale complessivo. A parità di energia consumata, la quantità effettiva di CO₂ emessa dipende da *come* quell'energia è stata prodotta, ovvero dalla composizione del mix energetico della rete elettrica a cui il nodo è connesso nel momento dell'esecuzione. Questa dimensione

contestuale è catturata dalla *carbon intensity* (CI), definita come la quantità di CO₂ emessa per ogni kilowattora prodotto dalla rete elettrica locale.

Per dare la misura del fenomeno, secondo l'International Energy Agency la carbon intensity media globale della generazione elettrica si attestava nel 2024 a 445 g CO₂/kWh [5], un valore che riflette la persistente dipendenza da fonti fossili nel mix energetico mondiale. Tale media nasconde tuttavia una forte eterogeneità geografica: nello stesso anno, la CI media dell'Unione Europea era di circa 175 g CO₂/kWh, mentre quella della Cina superava i 565 g CO₂/kWh [5].

La carbon intensity è inoltre una grandezza intrinsecamente variabile nel tempo: muta in funzione della disponibilità di fonti rinnovabili, della domanda e degli scambi tra zone di rete [6]. Ne consegue che, in una piattaforma FaaS distribuita su nodi geograficamente eterogenei, una stessa quantità di energia consumata può tradursi in emissioni molto diverse a seconda della zona e dell'istante in cui l'esecuzione ha luogo.

Questa osservazione suggerisce che le decisioni di scheduling di una piattaforma FaaS possano, e debbano, tenere conto del segnale ambientale fornito dalla CI, modulando il comportamento del sistema in funzione del contesto geografico e temporale. Si apre così la questione di quali leve siano disponibili per ridurre l'impatto ambientale dell'esecuzione senza compromettere il servizio offerto all'utente.

1.1.2 Approximate Computing come leva per il trade-off energia-accuratezza

Una direzione promettente per modulare il consumo energetico in funzione del contesto ambientale è rappresentata dal paradigma dell'*Approximate Computing*, che consente di ridurre il costo computazionale accettando, in modo controllato, una diminuzione dell'accuratezza del risultato prodotto. In letteratura, lavori come JouleGuard [7]

hanno mostrato come sia possibile progettare meccanismi di controllo runtime capaci di regolare dinamicamente il livello di approssimazione dell'esecuzione in funzione di un budget energetico, gestendo esplicitamente il compromesso tra efficienza energetica e qualità del risultato.

Applicata al paradigma FaaS, questa idea si traduce nella possibilità di associare a una stessa funzione logica più *varianti implementative*, caratterizzate da diversi profili energetici e da diversi livelli di accuratezza del risultato. Le varianti a consumo ridotto possono derivare da semplificazioni algoritmiche, da modelli più leggeri o da tecniche di approssimazione numerica, e costituiscono scelte progettuali controllate il cui impatto sull'accuratezza è noto e quantificabile.

La disponibilità di varianti con profili diversi apre la possibilità di trasformare la selezione della variante da eseguire in una decisione dello scheduler, guidata dal contesto ambientale: in condizioni di alta carbon intensity, privilegiare una variante più efficiente — accettando una riduzione controllata dell'accuratezza — produrrebbe un beneficio ambientale concreto; in condizioni di bassa carbon intensity, sarebbe invece possibile privilegiare la variante più accurata senza conseguenze ambientali significative.

Affinché tale selezione sia operativamente realizzabile, sono necessari due prerequisiti. Il primo è la disponibilità di misurazioni energetiche affidabili e aggiornate, a granularità sufficientemente fine da distinguere il consumo associato alle singole invocazioni. Il secondo è un criterio decisionale che integri il segnale ambientale con il trade-off energia-accuratezza, determinando quale variante selezionare in funzione della carbon intensity corrente.

1.1.3 Il problema della granularità nella misurazione energetica

Un prerequisito fondamentale per qualsiasi meccanismo di scheduling carbon-aware è la disponibilità di misure energetiche affidabili e aggiornate a granularità sufficientemente fine. Storicamente, la stima del consumo energetico nei sistemi virtualizzati è stata affrontata a livello di macchina fisica o di macchina virtuale, rendendo difficile attribuire il costo energetico a singoli componenti applicativi.

Nel contesto FaaS, questa difficoltà è amplificata dalla natura fine-grained delle invocazioni e dalla variabilità delle condizioni runtime, quali il carico concorrente, le interferenze tra container e la distinzione tra esecuzioni a freddo (*cold start*) e a caldo (*warm start*). Negli ultimi anni, strumenti di osservabilità energetica per ambienti containerizzati hanno reso disponibile una misurazione a granularità significativamente più fine.

In particolare, Kepler (*Kubernetes-based Efficient Power Level Exporter*) stima il consumo energetico a livello di singolo processo e container in ambienti Kubernetes, esportando metriche accessibili in tempo reale tramite Prometheus [8]. La disponibilità di misurazioni energetiche a livello di container costituisce un abilitatore fondamentale: consente di caratterizzare il profilo energetico delle singole varianti di funzione e di aggiornarlo a runtime, rendendo possibile una selezione informata e adattiva da parte dello scheduler.

1.1.4 Verso uno scheduling carbon-aware

La combinazione tra (i) la crescente attenzione alla sostenibilità ambientale dell'elaborazione distribuita, (ii) la possibilità di associare a una funzione logica più varianti implementative con diversi profili energetici e di accuratezza, (iii) la variabilità geografica e temporale della carbon intensity e (iv) la disponibilità di misurazioni energetiche

a granularità container-level rende oggi possibile e necessario progettare meccanismi di scheduling che integrino il segnale ambientale come variabile decisionale nativa.

Un tale scheduler deve essere in grado di modulare dinamicamente il peso attribuito all'efficienza energetica rispetto all'accuratezza del risultato, in funzione della carbon intensity della zona geografica del nodo al momento dell'invocazione. Quando la CI è elevata, come in zone alimentate prevalentemente da fonti fossili, privilegiare una variante più efficiente energeticamente, anche a scapito di una riduzione controllata dell'accuratezza, produce un beneficio ambientale concreto. Quando la CI è bassa — come in zone prevalentemente rinnovabili — l'impatto ambientale di un'esecuzione ad alta intensità computazionale è marginale, e la piattaforma può privilegiare la variante più accurata senza conseguenze significative.

Questa visione pone diverse sfide aperte. In primo luogo, occorre un modello capace di tradurre la carbon intensity corrente in un segnale decisionale normalizzato, che tenga conto delle specificità del regime energetico di ciascuna zona geografica: la stessa variazione assoluta di CI ha significati differenti in zone con distribuzioni storiche diverse. In secondo luogo, è necessario un criterio di selezione tra le varianti disponibili che gestisca formalmente il trade-off energia-accuratezza, evitando soluzioni dominate e individuando il miglior compromesso in funzione del contesto ambientale. Infine, le stime energetiche su cui si fonda la selezione devono essere aggiornate a runtime per riflettere il comportamento effettivo delle varianti sul nodo corrente, anziché dipendere esclusivamente da profilazioni statiche soggette a deriva.

Un ulteriore vincolo è di natura contrattuale: la riduzione dell'accuratezza introdotta dalle varianti approssimate non può essere imposta unilateralmente dalla piattaforma, ma richiede il consenso esplicito dell'utente che formula la richiesta di invocazione. Solo in presenza di tale consenso lo scheduler può attivare la selezione carbon-aware;

in assenza, la piattaforma deve garantire il comportamento originale, preservando la piena compatibilità retroattiva.

1.2 Contributi e obiettivi della tesi

L'obiettivo principale di questo lavoro è la progettazione e realizzazione di un'estensione carbon-aware della piattaforma Serverledge [3], in grado di adattare dinamicamente le decisioni di selezione delle varianti di funzione in funzione della carbon intensity della zona geografica del nodo, minimizzando l'impatto ambientale dell'esecuzione senza rinunciare interamente all'accuratezza del risultato.

Il sistema è progettato attorno al principio che il trade-off tra consumo energetico e qualità del risultato non debba essere risolto staticamente: è la piattaforma stessa a determinare, a runtime, il bilanciamento ottimale tra le due dimensioni, sulla base del contesto ambientale del nodo.

In particolare, il lavoro si propone di:

- **Introdurre il modello di funzione logica con varianti energetiche.** Ciascuna funzione logica è associata a un insieme di varianti implementative alternative, ognuna descritta da un profilo energetico che ne quantifica il costo stimato per invocazione, il costo di cold start e, ove applicabile, il livello di accuratezza del risultato prodotto.
- **Integrare Kepler per il monitoraggio energetico e l'aggiornamento adattivo dei profili.** Il sistema si avvale di Kepler per acquisire stime del consumo energetico a granularità di container nei nodi Kubernetes, esportate tramite Prometheus. Le misurazioni sono raccolte a runtime durante l'esecuzione delle funzioni e alimentano un meccanismo periodico che aggiorna i profili energetici

delle varianti tramite media mobile esponenziale, in modo che le stime riflettano il comportamento effettivo sul nodo corrente anziché dipendere da profilazioni statiche.

- **Gestire l'incertezza sulle stime iniziali tramite un meccanismo di esplorazione.** Per compensare l'inaccuratezza delle stime energetiche nelle fasi iniziali, il sistema integra un meccanismo ispirato al principio Lower Confidence Bound (LCB), che favorisce l'osservazione delle varianti meno misurate e converge progressivamente verso stime affidabili al crescere dei dati disponibili.
- **Progettare e implementare uno scheduler carbon-aware basato su frontiera di Pareto.** A seguito del consenso esplicito dell'utente, lo scheduler recupera la carbon intensity corrente della zona geografica del nodo e seleziona, tra le varianti Pareto-ottimali, quella che offre il miglior compromesso tra consumo energetico e accuratezza in funzione del contesto ambientale corrente. La frontiera di Pareto è costruita a runtime sulla base dei profili energetici aggiornati, garantendo che la selezione rifletta il comportamento effettivo delle varianti sul nodo corrente.
- **Valutare sperimentalmente il sistema su un insieme di funzioni rappresentative.** Quantificare il compromesso tra consumo energetico e accuratezza del risultato su un insieme di funzioni rappresentative, dimostrando la fattibilità e l'efficacia di un modello di esecuzione adattivo per il paradigma Function-as-a-Service nel continuum Edge-Cloud

Il contributo complessivo della tesi consiste nell'estensione di una piattaforma FaaS open-source con un meccanismo di scheduling carbon-aware che, a fronte del consenso dell'utente, seleziona autonomamente la variante più appropriata in funzione del

contesto ambientale del nodo, senza richiedere all'utente alcuna conoscenza del regime energetico locale. Il sistema introduce un trade-off controllato tra consumo energetico e accuratezza del risultato, modulato da un segnale ambientale geolocalizzato e adattivo, e ne valuta quantitativamente l'efficacia su funzioni di natura eterogenea e su zone geografiche con carbon intensity strutturalmente diverse.

1.3 Struttura della tesi

Il capitolo seguente, **Stato dell'arte**, analizzerà le principali soluzioni proposte in letteratura per la sostenibilità energetica nel Serverless Computing, a partire dagli strumenti di monitoraggio del consumo energetico del software fino alle tecniche di scheduling energy-aware, all'Approximate Computing e alle proposte di carbon-aware computing. Verrà inoltre introdotta la piattaforma Serverledge, adottata come base sperimentale per lo sviluppo del sistema, e sarà discusso il posizionamento del contributo rispetto allo stato dell'arte.

Nel capitolo **Architettura del Sistema** verrà descritto il sistema realizzato: il modello di funzione logica con varianti energetiche, l'integrazione di Kepler per il monitoraggio dei consumi a livello di container, il meccanismo di aggiornamento adattivo dei profili energetici tramite media mobile esponenziale e il meccanismo di esplorazione LCB per il raffinamento delle stime nelle fasi iniziali.

Nel capitolo **Scheduler Carbon-Aware** sarà presentata la logica decisionale del sistema: la calibrazione della sigmoide per zona geografica, la costruzione della frontiera di Pareto nello spazio energia-accuratezza e il criterio di selezione della variante in funzione della carbon intensity corrente.

Nel capitolo **Risultati Sperimentali** saranno riportati e discussi i risultati della valutazione sperimentale, quantificando il trade-off tra consumo energetico e accuratez-

za del risultato al variare della carbon intensity, della zona geografica e delle condizioni iniziali di stima.

Nell'ultimo capitolo, **Conclusioni e Sviluppi Futuri**, verranno discussi i risultati ottenuti in relazione agli obiettivi definiti e identificate le principali direzioni di ricerca futura nell'ambito della sostenibilità nel Serverless Computing.

Capitolo 2

Stato dell'arte

In questo capitolo si analizzeranno le principali soluzioni proposte in letteratura per il monitoraggio del consumo energetico e la gestione dell'efficienza energetica nel Serverless Computing. Verranno esaminate le tecniche di monitoraggio energetico a livello software, le soluzioni di scheduling energy-aware e le tecniche applicative basate sul paradigma dell'Approximate Computing. Verrà inoltre introdotto il concetto di carbon intensity come segnale di contesto per le decisioni di esecuzione, esaminando le proposte di carbon-aware computing presenti in letteratura.

2.1 Il monitoraggio dell'energia software

Il monitoraggio del consumo energetico a livello software rappresenta un tema di ricerca centrale nell'ambito dell'efficienza energetica dei sistemi informatici moderni. La crescente complessità delle architetture hardware e la diffusione di modelli di calcolo dinamici e distribuiti hanno reso sempre più difficile ottenere misure energetiche accurate e direttamente attribuibili alle singole componenti software. Questo problema risulta particolarmente rilevante nel contesto del Serverless Computing, dove le unità di esecuzione sono caratterizzate da invocazioni brevi, stateless e altamente dinamiche.

Le soluzioni tradizionali di monitoraggio dell'energia si basano prevalentemente su

misurazioni a livello hardware, che forniscono una stima del consumo complessivo di una macchina fisica o di specifici componenti, come CPU e memoria. Sebbene tali approcci offrano un'elevata precisione, la loro granularità risulta spesso insufficiente per analizzare il comportamento energetico delle applicazioni software, soprattutto in ambienti multi-tenant e containerizzati [9]. In questi scenari, il consumo energetico osservato è il risultato dell'interazione di più entità software concorrenti, rendendo complessa l'attribuzione dei costi energetici alle singole esecuzioni.

Per superare queste limitazioni, negli ultimi anni sono state proposte numerose soluzioni di monitoraggio dell'energia software, che mirano a stimare il consumo energetico a livelli di granularità più fini, quali processo, container o applicazione. Tali approcci si basano generalmente su modelli di stima che correlano metriche osservabili del sistema con misure energetiche ottenute a livello hardware, come nel caso di PowerAPI e SmartWatts [10].

Nel contesto del Serverless Computing, il problema del monitoraggio energetico è ulteriormente aggravato dalla crescente diffusione di architetture cloud-edge. Per ridurre la latenza e migliorare la qualità del servizio, sempre più spesso la computazione viene spostata verso nodi edge prossimi alla sorgente dei dati, anziché su data center remoti [1].

Come già accennato, tali nodi sono frequentemente caratterizzati da risorse computazionali ed energetiche significativamente più limitate. In questi scenari, la conoscenza del profilo energetico delle esecuzioni non è soltanto un'informazione utile ai fini dell'ottimizzazione, ma diventa un prerequisito per qualsiasi decisione consapevole sull'esecuzione. Le condizioni operative dei nodi edge possono variare nel tempo in funzione del carico di lavoro, delle caratteristiche dell'hardware sottostante e, in alcuni casi, della disponibilità energetica stessa del nodo. Di conseguenza, metriche

energetiche statiche o ottenute tramite profilazione offline risultano spesso inadeguate a rappresentare il comportamento reale delle esecuzioni. Diventa quindi fondamentale disporre di metriche energetiche aggiornate nel tempo, in grado di supportare decisioni di esecuzione che tengano conto sia del consumo energetico sia della qualità del risultato prodotto, ponendo le basi per politiche di scheduling capaci di gestire in modo controllato il compromesso tra queste due dimensioni.

2.1.1 Misuratori hardware del consumo energetico

I misuratori hardware del consumo energetico rappresentano l'approccio più diretto per la misurazione dell'energia assorbita da un sistema di calcolo. Tali soluzioni si basano su sensori fisici, esterni o integrati nell'hardware, e forniscono misure caratterizzate da un'elevata accuratezza, in quanto l'energia viene misurata direttamente a livello fisico. Il monitoraggio può avvenire a livello di intera macchina o di specifici componenti hardware, come CPU e memoria.

Una prima categoria è rappresentata dai misuratori hardware esterni, ovvero dispositivi interposti fisicamente tra la sorgente di alimentazione e il sistema sotto analisi. Questi strumenti consentono di misurare l'energia realmente dissipata dall'intero ecosistema hardware, includendo componenti spesso non osservabili tramite sensori software, come gli alimentatori (PSU), i sistemi di raffreddamento e le perdite resistive dei circuiti. Per tale motivo, essi forniscono una visione completa del consumo energetico del sistema.

Tuttavia, come ampiamente documentato in letteratura [11], l'adozione di misuratori hardware esterni introduce una serie di criticità che ne limitano l'applicabilità in contesti operativi su larga scala. In primo luogo, la misurazione avviene tipicamente a livello di intero sistema (*full-system power*), rendendo complesso isolare il contributo

Tabella 2.1: Confronto tra principali tipologie di misuratori hardware esterni per il monitoraggio del consumo energetico.

Categoria	Esempio di Dispositivo	Costo Stimato	Ambito di utilizzo
Entry-level	TP-Link Kasa, Kill-A-Watt	€30 – €60	Analisi preliminari e misure indicative su singole workstation.
Enterprise	APC Metered Rack PDU, Raritan PX	€800 – €2.500	Monitoraggio energetico di rack o gruppi di server in data center.
Laboratorio / R&D	Yokogawa WT310E, ZES ZIMMER LMG	€2.500 – €6.000+	Misurazioni ad alta precisione per validazione scientifica e ricerca accademica.

energetico di singoli componenti hardware o di specifiche entità software senza ricorrere a strumentazione aggiuntiva e altamente invasiva. In secondo luogo, l'integrazione di tali dispositivi in infrastrutture distribuite o in ambienti di data center comporta costi economici e complessità operative significative, legate sia all'acquisizione dell'hardware sia alla gestione del cablaggio e della raccolta dei dati.

Per fornire un ordine di grandezza dell'investimento richiesto, nella Tabella 2.1 sono riportate alcune delle principali categorie di strumenti utilizzati per il monitoraggio energetico hardware, insieme ai relativi ambiti di applicazione.

I dispositivi di fascia entry-level offrono generalmente frequenze di campionamento limitate, dell'ordine di pochi Hertz, risultando adeguati per analisi preliminari su singole macchine. Al contrario, gli strumenti utilizzati in ambito di ricerca e validazione scientifica consentono misurazioni ad alta frequenza e un'elevata precisione, a fronte di costi significativamente più elevati.

Oltre ai misuratori hardware esterni, i moderni processori integrano sensori energetici direttamente a livello di chip, rendendo possibile l'accesso a stime del consumo

energetico tramite interfacce software. Un esempio significativo è rappresentato dalla tecnologia *Running Average Power Limit* (RAPL), introdotta da Intel, che consente di ottenere informazioni sul consumo energetico di diversi domini di potenza interni al processore [12, 13].

RAPL suddivide il consumo energetico del sistema in un insieme di domini logici, ciascuno dei quali rappresenta una porzione specifica dell'hardware. In particolare, il dominio *Package* (PKG) rappresenta il consumo energetico complessivo del processore, includendo i core, le cache e le interconnessioni interne. Il dominio *Power Plane 0* (PP0) è invece associato principalmente ai core di calcolo della CPU, mentre altri domini, come *DRAM*, forniscono una stima del consumo energetico della memoria principale, quando supportati dall'architettura.

La Figura 2.1 illustra un esempio di suddivisione del consumo energetico secondo il modello RAPL, evidenziando come i diversi domini di potenza corrispondano a specifiche componenti hardware del sistema. Questa rappresentazione consente di comprendere come le misure fornite da RAPL siano intrinsecamente legate all'organizzazione fisica del processore e non alle singole entità software in esecuzione.

Il funzionamento di RAPL si basa su modelli energetici interni al processore, calibrati dal produttore, che stimano l'energia consumata a partire da eventi hardware e contatori interni. Sebbene tali misure non rappresentino una misurazione fisica diretta dell'energia, diversi studi hanno dimostrato come esse offrano un buon compromesso tra accuratezza e semplicità di utilizzo, risultando particolarmente adatte al monitoraggio energetico a runtime[14]. Le informazioni rese disponibili da RAPL possono essere lette tramite interfacce standard del sistema operativo, consentendo la loro integrazione in strumenti di monitoraggio software.

Nonostante questi vantaggi, i sensori hardware integrati presentano limiti strut-

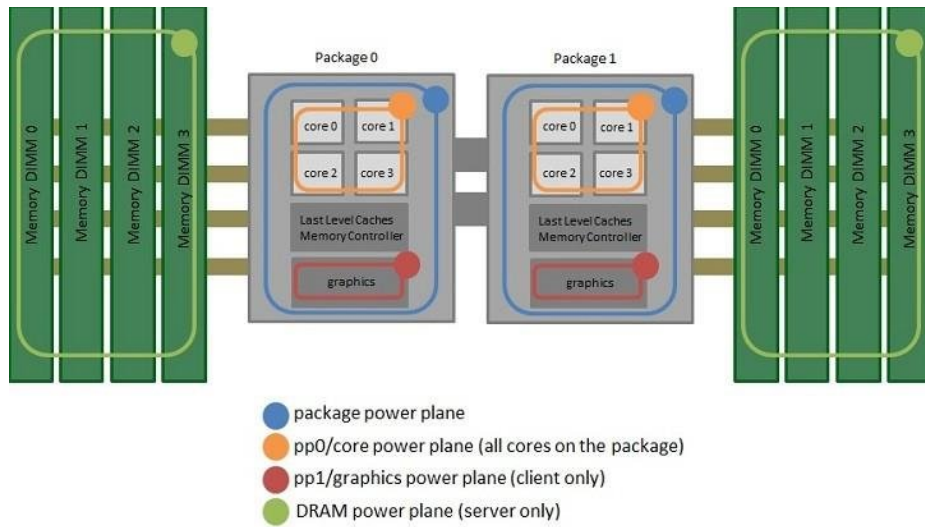


Figura 2.1: Esempio di domini di potenza definiti dal modello RAPL. Il dominio *Package* rappresenta il consumo energetico complessivo della CPU, mentre domini più specifici, come *PP0* e *DRAM*, forniscono stime riferite rispettivamente ai core di calcolo e alla memoria principale.

turali quando applicati a contesti software moderni. La granularità delle misure è tipicamente limitata al livello di macchina o di componente hardware e non consente di attribuire in modo diretto il consumo energetico alle singole applicazioni, ai container o alle singole invocazioni. In ambienti multi-tenant e containerizzati, come quelli tipici del Serverless Computing, il consumo energetico osservato è quindi il risultato dell'esecuzione concorrente di più entità software, rendendo complessa l'analisi del comportamento energetico delle singole funzioni.

Inoltre, le misure fornite da RAPL sono di natura cumulativa e non distinguono in modo esplicito tra le diverse fasi di esecuzione, quali inizializzazione, esecuzione vera e propria e periodi di inattività. Questo aspetto risulta particolarmente critico nel Serverless Computing, dove fenomeni come il cold start e il riutilizzo dei container hanno un impatto significativo sul consumo energetico complessivo. Di conseguenza, pur costituendo una base affidabile per la stima dell'energia a livello di sistema, i

sensori hardware integrati non risultano sufficienti per supportare analisi a grana fine e decisioni di gestione energetica a livello software.

Alla luce di queste considerazioni, sebbene i misuratori hardware costituiscano una base affidabile per la misurazione dell'energia, essi risultano insufficienti per supportare il monitoraggio continuo e a grana fine richiesto da ambienti dinamici e distribuiti. In tali contesti emerge la necessità di soluzioni in grado di offrire una maggiore flessibilità e di integrarsi in modo più stretto con i meccanismi di esecuzione del software.

2.1.2 Misuratori software e modelli di stima

Le limitazioni intrinseche dei misuratori hardware, quali la scarsa granularità temporale, i costi elevati di implementazione su larga scala e l'impossibilità di attribuire direttamente il consumo energetico a singole entità logiche in sistemi multi-tenant, hanno favorito l'emergere dei misuratori software. Questi strumenti non si basano su sensori fisici dedicati per ogni applicazione, ma utilizzano modelli di stima che correlano l'attività del software con il consumo energetico complessivo del sistema. Il principio fondamentale consiste nell'impiegare metriche di utilizzo delle risorse hardware, come cicli di clock della CPU, istruzioni eseguite, accessi alla cache e operazioni di input/output, come base per stimare l'energia assorbita dalle singole entità di esecuzione. Questo approccio consente di ottenere una visibilità energetica più granulare rispetto alle sole misure hardware, risultando particolarmente adatto a contesti moderni quali il cloud computing, le architetture a microservizi e i sistemi containerizzati.

Uno dei framework più rappresentativi in questo ambito è PowerAPI, una libreria modulare progettata per il monitoraggio del consumo energetico a livello di processo attraverso componenti indipendenti e meccanismi di raccolta dati asincroni e scalabili [10]. L'architettura di PowerAPI è progettata per separare nettamente la fase di raccol-

ta delle metriche da quella di stima energetica, favorendo l'estendibilità e la portabilità del sistema. Il pilastro tecnologico di PowerAPI è rappresentato dall'**HWPC Sensor** (*Hardware Performance Counter Sensor*), un componente specializzato nel campionamento ad alta frequenza dei contatori di prestazione hardware [15]. Il sensore si interfaccia direttamente con il sottosistema `perf_event` del kernel Linux, permettendo di raccogliere metriche dettagliate sull'attività della CPU associate a specifici *Control Groups* (cgroups) ¹ In parallelo, l'HWPC Sensor interroga i *Model Specific Registers* (MSR) tramite l'interfaccia *Running Average Power Limit* (RAPL), ottenendo misure cumulative del consumo energetico dei principali domini della CPU [14].

La Figura 2.2 mostra una rappresentazione concettuale dell'architettura di PowerAPI. Le metriche di utilizzo delle risorse e le informazioni energetiche fornite dai sensori hardware vengono elaborate dal modello di stima **SmartWatts**, adottato da PowerAPI per attribuire il consumo energetico ai singoli processi.

SmartWatts utilizza tecniche di regressione lineare per scomporre il consumo energetico globale in una componente statica e una dinamica, consentendo un'attribuzione più accurata rispetto a modelli puramente statici [16]. Tale approccio richiede una fase di auto-calibrazione *online* e l'accesso diretto all'hardware sottostante.

Tali requisiti rendono PowerAPI particolarmente adatto a scenari *bare-metal*, ma ne limitano l'applicabilità in ambienti virtualizzati e cloud pubblici, dove l'accesso ai registri hardware della CPU e ai performance counter è spesso inibito dagli hypervisor per motivi di sicurezza e isolamento. In questi contesti emerge la necessità di strumenti di monitoraggio energetico progettati nativamente per ambienti orchestrati.

¹Il sottosistema `perf_event` del kernel Linux fornisce un'interfaccia unificata per il monitoraggio di eventi hardware e software a basso livello, come cicli di clock della CPU, istruzioni eseguite e cache miss. Attraverso la chiamata di sistema `perf_event_open()`, consente di associare le misurazioni a processi, thread o *cgroups*, esponendo i dati tramite *file descriptor* e buffer ad anello con overhead contenuto. `perf_event` costituisce inoltre la base del tool `perf`, ampiamente utilizzato per la profilazione delle prestazioni nei sistemi Linux.

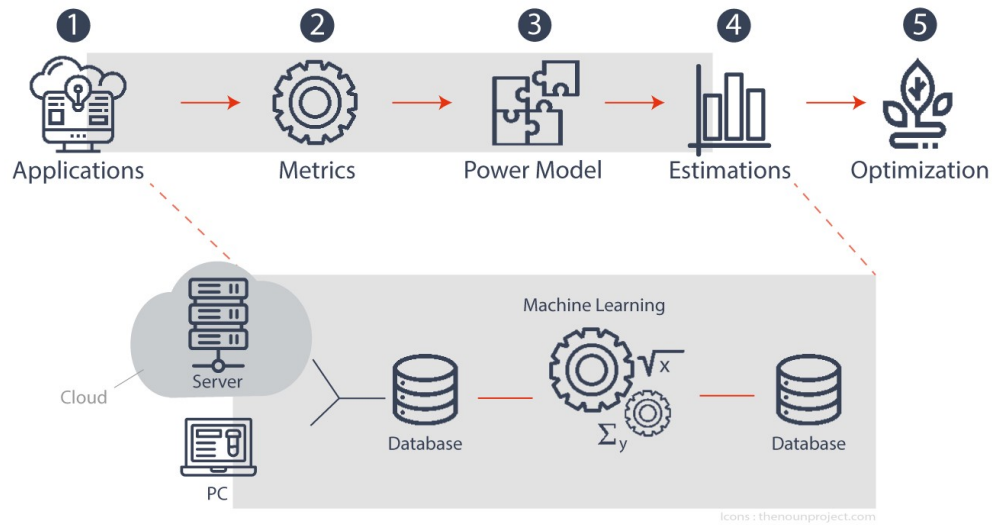


Figura 2.2: Architettura concettuale di PowerAPI. I contatori di prestazione hardware e le misure energetiche aggregate (RAPL) vengono combinati dal modello SmartWatts per stimare il consumo energetico dei singoli processi.

Un approccio rappresentativo in questa direzione è **Kepler** (*Kubernetes-based Efficient Power Level Exporter*), un progetto *cloud-native* orientato al monitoraggio energetico di container e Pod in cluster Kubernetes [17, 18]. A differenza di PowerAPI, Kepler non richiede accesso diretto ai registri hardware della CPU, ma sfrutta la tecnologia *eBPF* (*Extended Berkeley Packet Filter*) per raccogliere metriche a livello kernel in modo non intrusivo e con overhead computazionale ridotto.

La Figura 2.3 mostra l'architettura complessiva di Kepler. Il sistema utilizza un *eBPF Program Generator* per generare dinamicamente programmi eBPF che vengono agganciati ai *tracepoint* del kernel e ai performance counter della CPU. Questi programmi intercettano eventi di schedulazione e statistiche di esecuzione, consentendo di associare l'attività computazionale ai singoli container in modo fine-grained.

Le informazioni raccolte tramite eBPF vengono arricchite dal *Pod Lister*, che interroga le API di Kubernetes per mappare gli identificativi dei container ai rispettivi

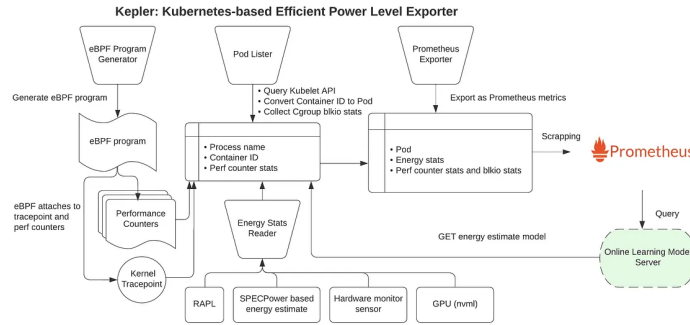


Figura 2.3: Architettura di Kepler.

Pod e raccoglie statistiche aggiuntive dai *control groups*. Parallelamente, il componente *Energy Stats Reader* acquisisce informazioni energetiche aggregate dal nodo, utilizzando diverse fonti a seconda della disponibilità dell'hardware sottostante, tra cui RAPL, sensori hardware esterni, interfacce GPU (NVML) e modelli basati su benchmark standardizzati.

Quando le misure hardware dirette non risultano accessibili, scenario tipico delle macchine virtuali nei cloud pubblici, Kepler ricorre a modelli di stima basati su *Machine Learning*, denominati *Estimators*. Tali modelli, addestrati offline su differenti architetture hardware e carichi di lavoro, ricevono in input le metriche raccolte a livello kernel e producono una stima della potenza assorbita dai container, che viene successivamente integrata nel tempo per ottenere una stima energetica complessiva.

Le metriche prodotte da Kepler vengono infine esposte tramite un *Prometheus Exporter*, rendendo il consumo energetico direttamente interrogabile attraverso i meccanismi di *scraping* standard e facilmente integrabile con strumenti di visualizzazione. Questa integrazione nativa con l'ecosistema di osservabilità cloud consente di affiancare le metriche energetiche a quelle di performance, abilitando analisi congiunte e politiche di gestione consapevoli dal punto di vista energetico.

2.2 Elaborazione serverless sostenibile

Nel paradigma serverless, e in particolare nelle piattaforme Function-as-a-Service (FaaS), il consumo energetico è strettamente legato alla natura del modello di esecuzione: le funzioni sono entità effimere, stateless e isolate in ambienti containerizzati, invocate su richiesta con cicli di vita che si misurano spesso in millisecondi. Tale granularità, combinata con l'elevata variabilità del carico di lavoro, rende la gestione dell'energia una dimensione strutturalmente più complessa rispetto ai contesti HPC o ai data center tradizionali, nei quali le tecniche consolidate operano tipicamente a livello di nodo fisico e su scale temporali più ampie.

Allo stato dell'arte, la sostenibilità energetica in ambienti serverless è affrontata attraverso soluzioni che si collocano a livelli diversi dello stack di esecuzione. Un primo insieme di approcci opera a livello infrastrutturale, integrando il consumo energetico tra i criteri che guidano le decisioni di scheduling e di allocazione delle risorse: in questa prospettiva, l'obiettivo è determinare *dove* eseguire una funzione, su quale nodo o in quale data center, in modo da ridurre il consumo complessivo dell'infrastruttura.

Una direzione complementare consiste nell'agire sulla funzione stessa piuttosto che sull'infrastruttura. Attraverso il paradigma dell'*Approximate Computing*, è possibile associare a una medesima funzione logica più varianti implementative, caratterizzate da profili energetici e livelli di accuratezza differenti. La decisione non riguarda più soltanto dove eseguire la funzione, ma *quale* variante della funzione eseguire, introducendo un trade-off esplicito tra consumo energetico e qualità del risultato.

Più recentemente, un ulteriore filone di ricerca ha introdotto la *carbon intensity* della rete elettrica come segnale di contesto per guidare le decisioni di esecuzione. In questi approcci, denominati *carbon-aware*, il criterio di ottimizzazione si estende

dal solo consumo energetico all'impatto ambientale complessivo, tenendo conto della composizione del mix energetico disponibile al momento dell'invocazione.

Nei paragrafi successivi vengono analizzati i principali contributi proposti in ciascuna di queste direzioni, delineando il contesto nel quale si colloca il contributo della presente tesi.

2.2.1 Scheduling energeticamente consapevole nelle piattaforme FaaS

Nel paradigma Function-as-a-Service (FaaS), lo scheduling costituisce il punto di decisione centrale attraverso il quale la piattaforma determina le modalità di esecuzione di ciascuna invocazione. In una piattaforma FaaS tradizionale, le decisioni di scheduling sono guidate prevalentemente da criteri prestazionali: disponibilità delle risorse, bilanciamento del carico, minimizzazione della latenza e rispetto degli accordi di servizio. In questa prospettiva, il consumo energetico associato all'esecuzione è una conseguenza delle scelte effettuate, non un parametro che le orienta.

L'integrazione della dimensione energetica nel processo decisionale trasforma lo scheduling da meccanismo di allocazione delle risorse a strumento di controllo del profilo energetico dell'infrastruttura. In uno scheduler *energy-aware*, il consumo energetico diventa un criterio esplicito nella valutazione delle alternative di esecuzione: non si tratta soltanto di determinare se un nodo dispone di risorse sufficienti per eseguire una funzione, ma di valutare quale tra le opzioni disponibili produca il risultato desiderato con il minor impatto energetico complessivo.

Nel contesto serverless, questa integrazione è resa più complessa dalla natura stessa del modello di esecuzione. Le funzioni FaaS sono entità effimere, con cicli di vita che si misurano spesso in millisecondi o pochi secondi, il che implica una frequenza di

decisioni di scheduling significativamente superiore rispetto a quella tipica dei sistemi HPC o dei workload tradizionali su macchine virtuali. Ogni decisione ha un orizzonte temporale ridotto, ma la frequenza con cui tali decisioni si ripetono fa sì che anche differenze marginali nel consumo per singola invocazione si traducano, su base oraria o giornaliera, in variazioni apprezzabili del profilo energetico complessivo del nodo. L'inizializzazione di nuovi ambienti di esecuzione (*cold start*) rappresenta un'ulteriore fonte di variabilità: il costo energetico della prima invocazione su un container appena creato può essere sensibilmente superiore a quello delle invocazioni successive sullo stesso ambiente (*warm start*), rendendo la gestione del riutilizzo dei container una dimensione rilevante anche dal punto di vista energetico.

A questa granularità temporale si aggiunge, nelle architetture cloud-edge, una significativa eterogeneità spaziale. I nodi che compongono l'infrastruttura possono differire per architettura hardware, capacità computazionale, profilo di consumo, disponibilità energetica e connettività di rete. In ambienti edge, alcuni nodi possono essere alimentati da fonti rinnovabili intermittenti o da batterie con capacità limitata, introducendo una variabilità temporale nella stessa disponibilità delle risorse di esecuzione. In ambienti federati o multi-sito, la decisione di *placement* deve inoltre considerare il costo — anche energetico — della movimentazione dei dati tra siti, che in alcuni scenari può dominare il bilancio complessivo dell'invocazione. Questa molteplicità di fattori rende la progettazione di scheduler energeticamente consapevoli per piattaforme FaaS un problema intrinsecamente multi-dimensionale.

È importante osservare che, in tutti gli approcci proposti in letteratura per lo scheduling energy-aware nel contesto FaaS, la consapevolezza energetica riguarda esclusivamente *dove* eseguire la funzione, trattando quest'ultima come un'entità con profilo energetico invariante. Questa delimitazione caratterizza lo scheduling energy-aware

come una famiglia di soluzioni che opera esclusivamente a livello infrastrutturale.

Nei paragrafi seguenti vengono analizzati due contributi rappresentativi di questa famiglia, ciascuno dei quali affronta il problema con formulazione e obiettivi distinti. E²FIS [19] adotta una strategia di consolidamento del carico, formulando il problema di allocazione come un'istanza di ottimizzazione vincolata con l'obiettivo di minimizzare il numero di nodi attivi e, di conseguenza, il consumo energetico dell'infrastruttura. GreenFaaS [20] estende invece la prospettiva agli ambienti federati multi-sito, integrando nella funzione di scheduling una stima del costo energetico associato ai trasferimenti dati tra domini eterogenei.

2.2.1.1 Scheduling per consolidamento energetico con vincoli di deadline

Un primo approccio alla riduzione del consumo energetico in piattaforme FaaS consiste nel *workload consolidation*: anziché distribuire uniformemente il carico tra tutti i nodi disponibili, lo scheduler concentra le invocazioni su un sottoinsieme selezionato, consentendo ai nodi rimanenti di essere portati in stato inattivo. L'intuizione alla base di questa strategia è che il consumo di un nodo in stato di *idle* — pur non eseguendo alcuna funzione — rappresenta una quota non trascurabile del budget energetico totale, e la sua eliminazione può produrre risparmi significativi a livello di infrastruttura.

E²FIS (*Energy-Efficient Function Invocation Scheduling*) [19], proposto per piattaforme FaaS nell'edge-cloud continuum, adotta questo paradigma formulando il problema di allocazione come un'istanza di Programmazione Lineare Intera Mista (MILP). Il sistema assume un insieme di nodi eterogenei, ciascuno caratterizzato da capacità computazionale, memoria disponibile, throughput effettivo e profilo di consumo energetico. Le funzioni sono descritte tramite requisiti di memoria, carico computazionale e *deadline* di completamento; la stima del tempo di esecuzione tiene conto delle diffe-

renze architetturali tra nodi, consentendo allo scheduler di verificare la compatibilità di ciascuna assegnazione con i vincoli temporali imposti.

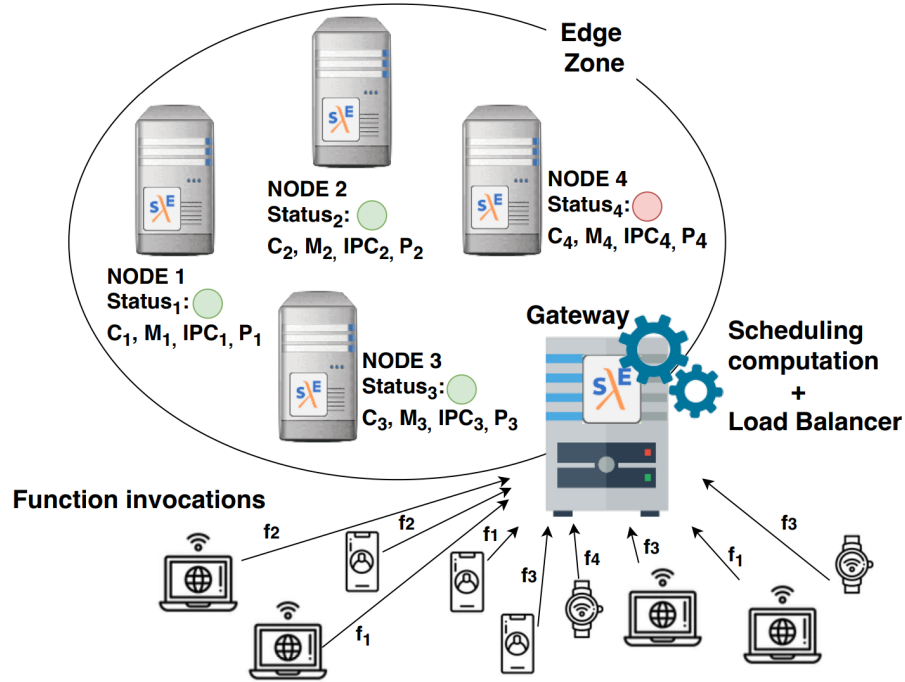


Figura 2.4: Architettura di E²FIS. I nodi edge sono caratterizzati da differenti capacità computazionali e profili di consumo. Il modulo di *Scheduling computation* determina periodicamente l’allocazione delle funzioni e il sottoinsieme di nodi da mantenere attivi.

Come illustrato in Figura 2.4, l’approccio adotta una pianificazione periodica basata su epoche temporali discrete. Prima dell’inizio di ciascuna epoca, la funzione obiettivo minimizza il consumo energetico associato ai nodi attivi nell’intervallo considerato, selezionando il sottoinsieme minimo necessario a soddisfare tutte le richieste entro le rispettive deadline. Ne deriva una logica di consolidamento: quando il carico complessivo lo consente, le invocazioni vengono concentrate sui nodi con il miglior rapporto tra capacità computazionale e consumo energetico, mentre i restanti possono essere portati in stato inattivo.

Per garantire robustezza operativa in presenza di istanze computazionalmente one-

rose o di vincoli particolarmente stringenti, il sistema prevede un limite temporale per la risoluzione del problema di ottimizzazione: qualora la soluzione ottima non sia ottenibile entro la soglia stabilita, viene adottata la migliore soluzione ammissibile disponibile. In condizioni limite, il framework può inoltre ricorrere a una politica di fallback ispirata a EDF (*Earliest Deadline First*), privilegiando il rispetto delle deadline rispetto all'ottimizzazione energetica. Tale meccanismo evidenzia un compromesso ricorrente nella progettazione di scheduler energy-aware: in presenza di vincoli temporali stringenti, il rispetto delle deadline può richiedere l'adozione di assegnazioni energeticamente subottimali pur di garantire il completamento delle invocazioni entro i termini stabiliti. E²FIS gestisce ciò attraverso una gerarchia esplicita tra gli obiettivi, nella quale le deadline assumono priorità rispetto all'efficienza energetica.

2.2.1.2 Scheduling energy-aware in ambienti FaaS federati ed eterogenei

In ambienti FaaS federati o multi-sito, la decisione di *placement* si arricchisce di una dimensione ulteriore rispetto agli scenari considerati da E²FIS. Quando risorse eterogenee e geograficamente distribuite — tra cui cluster HPC, workstation e infrastrutture cloud — partecipano alla stessa piattaforma di esecuzione, le funzioni possono operare su dataset residenti in domini amministrativi distinti. In tali scenari, il trasferimento dei dati di input verso il nodo selezionato per l'esecuzione introduce un costo energetico aggiuntivo che può risultare, in alcune configurazioni, paragonabile o superiore a quello dell'esecuzione stessa. Ne consegue che l'allocazione ottimale deve considerare congiuntamente il costo di *computation* e di *communication*, evitando scelte che riducono il consumo di esecuzione introducendo al contempo overhead energetici significativi dovuti alla movimentazione dei dati.

GreenFaaS [20] affronta questo problema progettando un framework di orchestra-

zione per invocazioni FaaS in ambienti distribuiti, nel quale la decisione di scheduling integra una stima esplicita del costo energetico dei trasferimenti. Come mostrato in Figura 2.5, l'architettura si articola in tre componenti principali: un modulo di *Resource Monitoring*, un *Transfers Manager* e uno *Scheduler*.

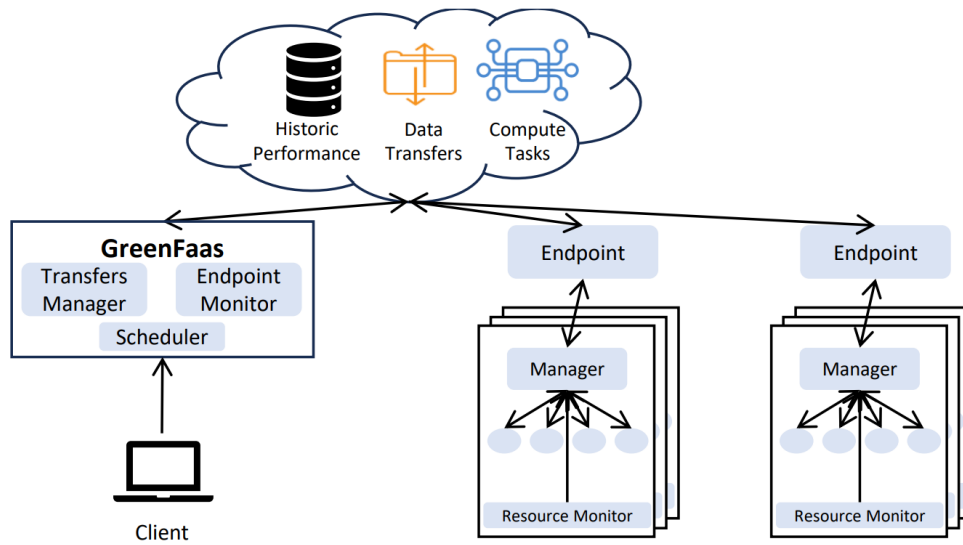


Figura 2.5: Architettura di alto livello di GreenFaaS: monitoraggio degli endpoint, stima del costo energetico dei trasferimenti e scheduling energy-aware per l'allocazione dei task in ambienti federati eterogenei.

Il modulo di monitoraggio raccoglie a runtime metriche energetiche e prestazionali dei nodi, sfruttando interfacce hardware — tra cui RAPL per CPU e NVML per GPU — per derivare stime di potenza e caratterizzare il comportamento energetico delle risorse disponibili. Tali misure consentono di stimare il costo di esecuzione su piattaforme eterogenee, superando la limitazione di approcci che utilizzano il solo tempo di esecuzione come proxy indiretto del consumo.

Il *Transfers Manager* modella il contributo energetico associato alla movimentazione dei dati tra siti. Quando l'input risiede in un dominio diverso da quello candidato

all'esecuzione, il trasferimento introduce un costo legato alla trasmissione in rete e alle operazioni di I/O agli estremi della comunicazione. GreenFaaS stima tale costo in funzione della dimensione dei dati e delle caratteristiche del collegamento, consentendo di quantificare l'impatto energetico della scelta del sito anche nei casi in cui il nodo remoto risulti computazionalmente più efficiente ma geograficamente distante.

Sulla base di queste informazioni, lo *Scheduler* effettua una selezione multi-criterio che integra il consumo energetico previsto per l'esecuzione sul nodo candidato e il costo energetico dei trasferimenti eventualmente necessari, affiancando a tali grandezze una stima del tempo di completamento. Il peso relativo delle componenti è configurabile, consentendo di modulare il compromesso tra efficienza energetica e prestazioni in funzione degli obiettivi applicativi. Per mantenere la reattività in presenza di carichi dinamici e infrastrutture di grandi dimensioni, il framework adotta strategie euristiche che evitano la necessità di risolvere un problema di ottimizzazione globale a ogni invocazione.

GreenFaaS amplia dunque la nozione di consapevolezza energetica nello scheduling FaaS oltre la sola efficienza del nodo di calcolo, estendendola al costo di comunicazione tra siti. Questo contributo è particolarmente rilevante in architetture con forte dispersione geografica, dove i trasferimenti dati possono incidere in modo significativo sul bilancio energetico complessivo di un'invocazione.

Nel complesso, gli approcci analizzati in questa sottosezione condividono un elemento strutturale comune: la consapevolezza energetica si traduce in una decisione che riguarda esclusivamente il contesto di esecuzione — la scelta del nodo, la topologia dei nodi attivi, il percorso dei dati — mentre il contenuto dell'esecuzione rimane invariante. E²FIS concentra le invocazioni su un sottoinsieme di nodi selezionato per minimizzare il consumo complessivo dell'infrastruttura; GreenFaaS estende la valuta-

zione incorporando il costo energetico della comunicazione tra siti. In entrambi i casi, la funzione invocata è trattata come un'entità con un profilo computazionale e un consumo energetico per invocazione dati e non modificabili: lo scheduler determina *dove* eseguirla, ma non può influenzare *come* viene eseguita né il consumo che ne deriva.

Questa delimitazione evidenzia come l'intero spazio di ottimizzazione esplorato da questi approcci sia circoscritto al livello infrastrutturale. La possibilità di agire sul consumo energetico modulando la logica applicativa della funzione — ad esempio selezionando, tra più implementazioni della stessa funzione, quella che offre il miglior compromesso tra consumo e qualità del risultato in funzione delle condizioni correnti — rappresenta una direzione complementare, esplorata nella letteratura attraverso il paradigma dell'*Approximate Computing*, analizzato nella sottosezione successiva.

2.2.2 Approximate Computing come leva applicativa per il trade-off energia-accuratezza

La sottosezione precedente ha evidenziato come le strategie di scheduling energeticamente consapevole nelle piattaforme FaaS si concentrino esclusivamente sulla selezione del contesto di esecuzione, trattando la funzione invocata come un'entità atomica, il cui consumo energetico per invocazione è assunto come dato e immutabile. Tale impostazione, tuttavia, delimita lo spazio di ottimizzazione, lasciando inesplorata una prospettiva concettualmente distinta: anziché intervenire su *dove* eseguire la funzione, è possibile agire su *cosa* viene effettivamente eseguito al fine di ottenere un profilo energetico più favorevole.

Il paradigma dell'*Approximate Computing* fornisce il fondamento teorico per questa direzione. L'idea centrale è che molte computazioni ammettono risultati approssimati, e che accettare una riduzione controllata della precisione può tradursi in un consu-

mo di risorse computazionali — e quindi energetiche — significativamente inferiore. Questo principio è stato ampiamente studiato in contesti HPC e di calcolo scientifico, dove tecniche quali la *loop perforation* (eliminazione selettiva di iterazioni in cicli computazionali), la riduzione della precisione numerica e la semplificazione algoritmica sono state impiegate per ottenere compromessi controllati tra accuratezza e costo computazionale [21].

Applicato al contesto delle piattaforme FaaS, il paradigma dell'approximate computing si traduce nella possibilità di associare a una stessa funzione logica più *varianti implementative*, ciascuna caratterizzata da un diverso profilo energetico e da un diverso livello di accuratezza del risultato. Le varianti possono differire per molteplici aspetti: la complessità dell'algoritmo impiegato (ad esempio, un modello di machine learning con un numero ridotto di parametri rispetto alla versione completa), il numero di iterazioni o campioni utilizzati nel calcolo, il linguaggio di programmazione — e di conseguenza il runtime di esecuzione — o la granularità dei dati elaborati. L'elemento comune è che tutte le varianti risolvono lo stesso problema funzionale: producono un risultato semanticamente equivalente per lo stesso input ma con costi computazionali ed energetici differenti, e con livelli di accuratezza potenzialmente diversi.

L'esistenza di varianti con profili energia-accuratezza differenziati introduce un nuovo spazio decisionale per lo scheduler: la selezione della variante da eseguire diventa una decisione che può essere guidata da informazioni energetiche, da vincoli definiti dall'utente o da segnali di contesto esterni.

È importante osservare che la generazione delle varianti e la loro selezione a runtime sono due problemi distinti, affrontati in letteratura con approcci diversi. La generazione riguarda il modo in cui le varianti vengono create: manualmente, tramite tecniche di trasformazione del codice, o attraverso la sostituzione di componenti algoritmici. La se-

lezione riguarda invece il criterio con cui, al momento dell'invocazione, il sistema decide quale variante eseguire. Nei paragrafi seguenti vengono analizzati due contributi che affrontano questi problemi da prospettive complementari: *Code Once, Run Green* [22] si concentra sulla generazione automatica di varianti attraverso la traduzione del codice verso linguaggi più efficienti dal punto di vista energetico; JouleGuard [7] affronta il problema della selezione dinamica, introducendo un meccanismo di controllo runtime che regola il livello di approssimazione in funzione di un vincolo energetico esplicito.

2.2.2.1 Generazione automatica di varianti energeticamente efficienti

Un primo aspetto del problema riguarda la possibilità di ottenere varianti con profili energetici differenti senza richiedere una riprogettazione manuale della funzione. In ambienti serverless, la piattaforma dispone tipicamente dell'accesso al codice sorgente delle funzioni e del controllo completo sull'ambiente di esecuzione: questa combinazione apre la possibilità di intervenire sull'implementazione delle funzioni in modo trasparente per lo sviluppatore.

Code Once, Run Green [22] esplora questa possibilità proponendo ReFaaS, un servizio integrato nella piattaforma serverless Fission che traduce automaticamente funzioni da un linguaggio di programmazione a un altro utilizzando modelli linguistici di grandi dimensioni (LLM). L'obiettivo è generare una versione funzionalmente equivalente della funzione originale, ma eseguita su un runtime con un profilo energetico più favorevole. La motivazione alla base dell'approccio risiede nella significativa variabilità del consumo energetico tra linguaggi di programmazione differenti, documentata in letteratura con differenze che possono raggiungere un ordine di grandezza a parità di funzionalità implementata [22].

Come illustrato in Figura 2.6, la traduzione avviene in fase di build attraverso una

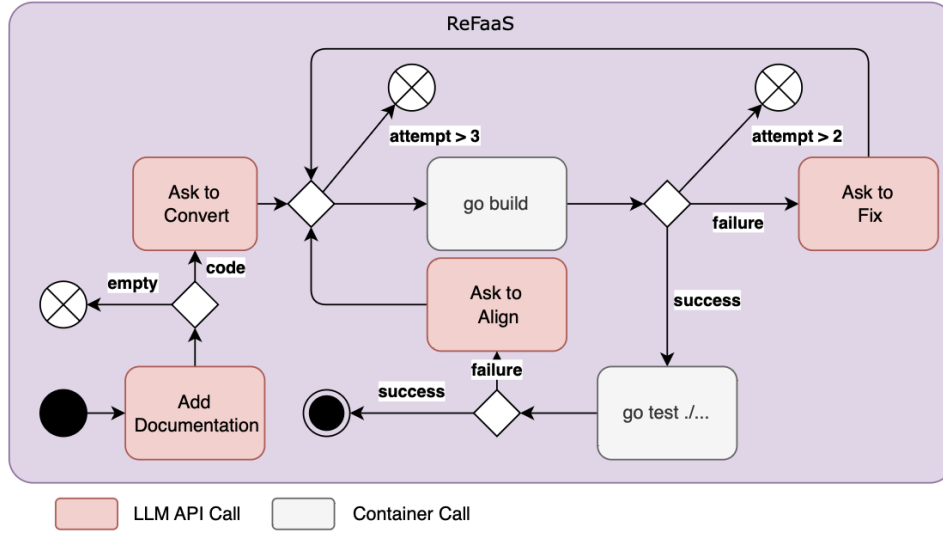


Figura 2.6: Pipeline di traduzione automatica proposta in *Code Once, Run Green*. La trasformazione avviene in fase di build tramite un processo iterativo di conversione e validazione del codice, con passaggi di recupero in caso di errori di compilazione o di test.

pipeline iterativa strutturata in tre fasi: traduzione del codice tramite LLM, compilazione e verifica della correttezza mediante test black-box sugli stessi input della funzione originale. In caso di errori di compilazione o di discrepanze nei risultati, la pipeline prevede passaggi di recupero che reinviano il codice al modello linguistico con il messaggio di errore, consentendo tentativi iterativi di correzione fino a un numero massimo predefinito. Se la traduzione ha successo e la funzione tradotta supera tutti i test, il nuovo artefatto sostituisce quello originale nel ciclo di esecuzione della piattaforma; in caso contrario, la funzione originale viene mantenuta senza alcuna interruzione del servizio.

I risultati sperimentali riportati dagli autori mostrano che le funzioni tradotte con successo possono ridurre il consumo energetico per invocazione fino al 70%, con un costo di traduzione che si ammortizza tipicamente dopo alcune migliaia di invocazioni. Tuttavia, l'approccio presenta limitazioni significative: il tasso di traduzione valida

si attesta intorno al 43% con i modelli linguistici testati, e alcune categorie di funzioni — in particolare quelle con dipendenze complesse o con logica di gestione degli errori specifica del linguaggio sorgente — risultano difficilmente traducibili in modo automatico.

Dal punto di vista concettuale, il contributo di *Code Once, Run Green* è rilevante perché dimostra empiricamente un principio fondamentale: la stessa funzione logica, implementata con tecnologie diverse, può presentare profili energetici significativamente differenti a parità di correttezza funzionale. Questo principio è alla base del concetto di *variante* utilizzato nei sistemi che, come quello descritto nella presente tesi, associano a una funzione logica un insieme di implementazioni alternative tra cui selezionare a runtime.

È opportuno osservare, tuttavia, che in *Code Once, Run Green* la decisione su quale variante eseguire è determinata staticamente, in fase di build: se la traduzione ha successo, la funzione tradotta sostituisce permanentemente quella originale. Non è previsto alcun meccanismo di selezione dinamica che consenta al sistema di scegliere tra la variante originale e quella tradotta in funzione delle condizioni operative correnti.

2.2.2.2 Controllo runtime del trade-off energia-accuratezza

Se *Code Once, Run Green* affronta il problema della generazione delle varianti, JouleGuard [7] affronta il problema complementare: dato un insieme di configurazioni applicative con diversi profili di accuratezza e consumo, come selezionare dinamicamente quella più appropriata in funzione di un obiettivo energetico. JouleGuard è un sistema di controllo runtime progettato per uno scenario in cui l'utente specifica un budget energetico — una quantità massima di energia che il sistema può consumare per unità di lavoro — e il sistema deve rispettare tale budget massimizzando al contempo

l'accuratezza del risultato prodotto.

L'intuizione fondamentale di JouleGuard consiste nel decomporre il problema di ottimizzazione in due sotto-problemi coordinati. Il primo, denominato *System Energy Optimizer* (SEO), ha il compito di individuare la configurazione di sistema — intesa come combinazione di risorse hardware, quali frequenza del processore, numero di core attivi e impostazioni del sottosistema di memoria — che massimizza l'efficienza energetica, definita come il rapporto tra lavoro svolto ed energia consumata. A tal fine, il SEO impiega medie mobili esponenziali ponderate (EWMA) per stimare l'efficienza delle diverse configurazioni sulla base delle misurazioni raccolte a runtime, bilanciando in modo adattivo l'esplorazione di configurazioni meno osservate con lo sfruttamento della configurazione attualmente più efficiente.

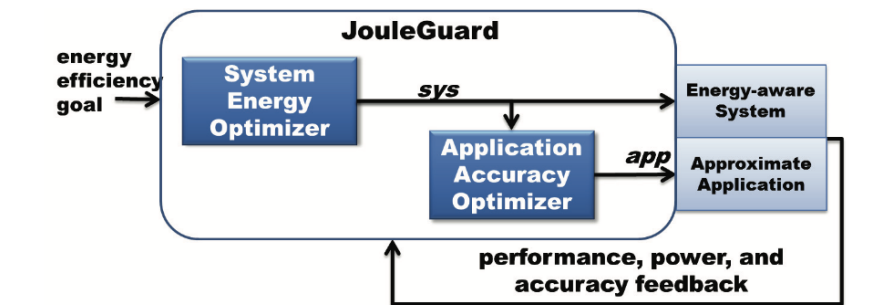


Figura 2.7: Schema concettuale di JouleGuard. Il *System Energy Optimizer* (SEO) identifica la configurazione di sistema più efficiente; l'*Application Accuracy Optimizer* (AAO) regola il livello di approssimazione dell'applicazione per rispettare il budget energetico, massimizzando l'accuratezza.

Il secondo sotto-problema, affidato all'*Application Accuracy Optimizer* (AAO), interviene quando la sola ottimizzazione delle risorse di sistema non è sufficiente a rispettare il budget energetico. In questo caso, è necessario ridurre ulteriormente il consumo agendo sull'applicazione stessa: l'AAO determina di quanto l'applicazione debba accelerare la propria esecuzione — riducendo il lavoro computazionale svolto,

e di conseguenza l'accuratezza del risultato — per colmare la distanza residua tra il consumo ottenuto con la configurazione di sistema ottimale e il budget richiesto dall'utente. Come illustrato in Figura 2.7, i due componenti operano in modo coordinato: il SEO comunica all'AAO le stime di prestazione e potenza della configurazione di sistema selezionata, e l'AAO utilizza queste informazioni per determinare il livello di approssimazione appropriato.

JouleGuard seleziona la configurazione con la massima accuratezza tra quelle che forniscono prestazioni sufficienti a rispettare il budget energetico. Questo meccanismo non richiede una quantificazione precisa dell'accuratezza delle configurazioni, ma soltanto un ordinamento totale: è sufficiente sapere che una configurazione è più accurata di un'altra, senza necessità di assegnare valori numerici assoluti alla qualità del risultato.

I risultati sperimentali riportati dagli autori, condotti su tre piattaforme hardware (mobile, tablet, server) con otto applicazioni approssimate costruite con due framework diversi (PowerDial e Loop Perforation), mostrano che JouleGuard rispetta i budget energetici con un errore relativo contenuto entro pochi punti percentuali e raggiunge un'accuratezza prossima all'ottimo teorico — calcolato come la migliore accuratezza ottenibile da un oracolo con conoscenza perfetta del futuro. Il sistema si adatta inoltre alle variazioni di fase del carico applicativo, aumentando automaticamente l'accuratezza quando le condizioni lo consentono.

Dal punto di vista concettuale, JouleGuard stabilisce diversi principi rilevanti nell'ambito dell'elaborazione serverless sostenibile: dimostra che il trade-off tra consumo energetico e accuratezza del risultato può essere gestito in modo formale e adattivo attraverso un ciclo di controllo runtime, e introduce l'uso di medie mobili esponenziali per la stima delle grandezze operative a partire da misurazioni raccolte durante

l'esecuzione.

È opportuno osservare, tuttavia, che JouleGuard opera su un'applicazione singola, con configurazioni di approssimazione predefinite e profilate offline. Il sistema non prevede la gestione di un insieme di varianti di funzione con profili energetici indipendenti, né contempla l'aggiornamento delle stime energetiche delle configurazioni applicative sulla base di misurazioni raccolte durante l'esecuzione: le stime delle risorse di sistema sono apprese a runtime, ma i profili energia-accuratezza delle configurazioni applicative sono determinati staticamente prima del deployment. In ambienti FaaS, dove il consumo per invocazione può variare in funzione del carico concorrente, dello stato del container e delle caratteristiche dei dati di input, questa assunzione di stazionarietà dei profili energetici costituisce un elemento da considerare.

In entrambi i casi esaminati, la decisione è guidata da grandezze interne al sistema: il budget energetico dell'utente in JouleGuard, il profilo di consumo del runtime in *Code Once, Run Green*. Nessuno dei due approcci incorpora segnali di contesto esterni che riflettano le condizioni della rete elettrica al momento dell'invocazione. Un filone di ricerca distinto, analizzato nella sottosezione successiva, esplora questa direzione introducendo la *carbon intensity* come criterio per modulare le decisioni di esecuzione.

2.2.3 Carbon-aware computing

Le sottosezioni precedenti hanno analizzato approcci che perseguono la sostenibilità energetica dell'elaborazione serverless operando su due leve distinte: la scelta del nodo di esecuzione (scheduling energy-aware) e la modulazione della logica applicativa (approximate computing). In entrambi i casi, il criterio di ottimizzazione è il consumo energetico in sé: l'obiettivo è ridurre i Joule consumati per invocazione o rispettare un budget energetico prefissato, indipendentemente dal momento in cui tale consumo

avviene.

Questa impostazione non cattura una dimensione rilevante del problema: come discusso nella Sezione 1.1.1, l'impatto ambientale di un'unità di energia consumata non è costante, ma dipende dalla carbon intensity della rete elettrica nel momento e nel luogo dell'esecuzione. Il paradigma del *carbon-aware computing* si fonda su questa osservazione: anziché trattare ogni kilowattora come equivalente, le decisioni di esecuzione possono essere modulate in funzione della carbon intensity corrente, spostando il criterio di ottimizzazione dal solo consumo energetico all'impatto ambientale complessivo dell'esecuzione.

In letteratura, le strategie carbon-aware sono state declinate attraverso due leve principali. La prima, denominata *temporal shifting*, consiste nel posticipare l'esecuzione di carichi di lavoro tolleranti al ritardo verso momenti in cui la carbon intensity della rete locale è prevista più bassa. La seconda, denominata *spatial shifting*, consiste nel distribuire i carichi di lavoro tra data center situati in regioni geografiche con diversi livelli di carbon intensity, privilegiando l'esecuzione nei siti con il mix energetico più favorevole al momento dell'invocazione. Entrambe le strategie presuppongono la disponibilità di dati sulla carbon intensity in tempo reale o tramite previsioni a breve termine, tipicamente forniti da servizi specializzati quali ElectricityMaps [6] o WattTime.

Nei paragrafi seguenti vengono analizzati due contributi che rappresentano queste direzioni in contesti distinti: il *Carbon-Intelligent Computing System* (CICS) di Google [23], che costituisce il primo sistema carbon-aware in produzione su scala industriale e opera attraverso temporal shifting di workload batch nei data center; e GreenCourier [24], che porta il paradigma carbon-aware specificamente nel contesto delle piattaforme FaaS/serverless, adottando una strategia di spatial shifting per la

schedulazione delle funzioni tra regioni geografiche distribuite.

2.2.3.1 Temporal shifting su scala industriale: il Carbon-Intelligent Computing System

Il *Carbon-Intelligent Computing System* (CICS) [23], sviluppato e operato da Google, rappresenta il primo sistema carbon-aware implementato in produzione su scala globale. Il sistema opera su oltre 20 data center distribuiti in 4 continenti e gestisce un consumo elettrico superiore a 15,5 terawattora annui, rendendolo il caso di studio di riferimento per l'applicazione del carbon-aware computing a infrastrutture di calcolo reali.

Il problema affrontato dal CICS può essere descritto in termini intuitivi. I data center di Google eseguono due tipologie di carichi di lavoro con caratteristiche temporali molto diverse. La prima comprende i servizi rivolti agli utenti — ricerca, mappe, streaming — e le macchine virtuali dei clienti cloud: questi carichi sono *inflexible*, nel senso che devono essere eseguiti nel momento in cui vengono richiesti, senza possibilità di posticipazione. La seconda tipologia comprende processi batch — compressione dei dati, addestramento di modelli di machine learning, pipeline di elaborazione video — che tollerano ritardi purché il lavoro venga completato entro 24 ore: questi carichi sono *flexible*. Il CICS sfrutta la flessibilità temporale di questa seconda categoria: se la carbon intensity della rete elettrica locale è alta in un determinato momento della giornata, il sistema riduce la quantità di lavoro batch ammesso in quel momento, accumulandolo per eseguirlo nelle ore successive in cui la rete è alimentata da fonti più pulite. Il volume complessivo di lavoro giornaliero non cambia — viene soltanto redistribuito nel tempo per allinearlo al profilo della carbon intensity.

Il meccanismo con cui il CICS realizza questa redistribuzione è la *Virtual Capacity Curve* (VCC). Per ciascun cluster di calcolo, il sistema genera giornalmente una curva che definisce, ora per ora, il limite massimo di risorse computazionali disponibili per i

workload flessibili nel giorno successivo. Questa curva non è uniforme: nelle ore in cui la carbon intensity è prevista alta, il limite viene abbassato, impedendo allo scheduler dei job (Borg) di ammettere nuovi job batch; nelle ore con carbon intensity bassa, il limite viene alzato, consentendo l'esecuzione del lavoro accumulato. Il risultato, illustrato in Figura 2.8, è uno spostamento del carico flessibile dalle ore centrali della giornata — tipicamente caratterizzate da carbon intensity più elevata — verso le prime ore del mattino e la sera, quando la rete tende ad essere più pulita.

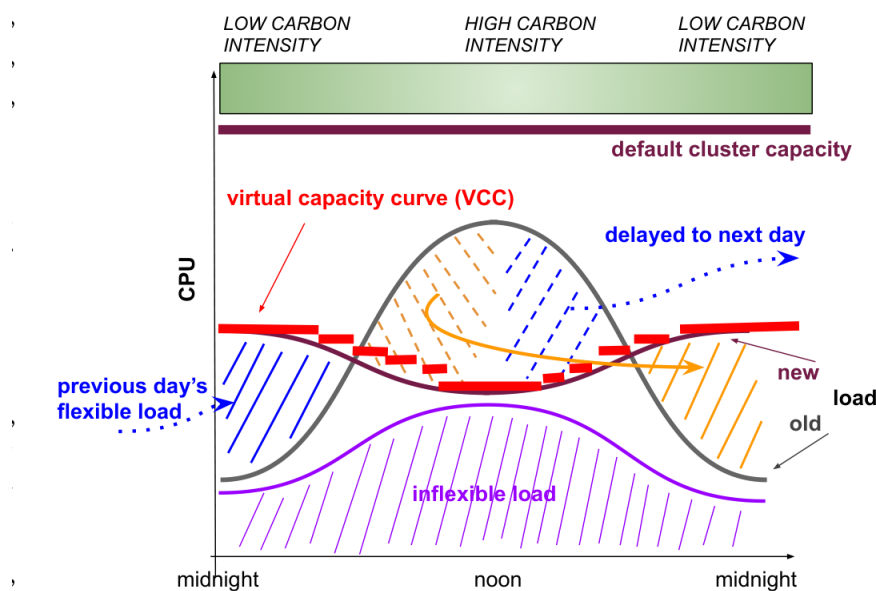


Fig. 2: Effect of using the VCC mechanism for load shaping in a cluster.

Figura 2.8: Effetto della Virtual Capacity Curve (VCC) sul profilo di carico di un cluster. La barra superiore indica i livelli di carbon intensity nel corso della giornata. La curva VCC (in rosso) riduce la capacità disponibile per i workload flessibili nelle ore ad alta carbon intensity, spostando il carico (aree arancione e blu) verso le ore più pulite della giornata. Il carico inflessibile (in viola) non viene alterato.

È importante osservare che la VCC non interviene sui singoli job: non decide quali

job eseguire o in quale ordine, né modifica la logica dei job stessi. La VCC agisce a un livello superiore, impostando un vincolo aggregato sulla quantità totale di risorse disponibili per i workload flessibili in ciascuna ora. Lo scheduler dei job (Borg) continua a operare normalmente entro questo vincolo, senza necessità di modifiche: se la capacità disponibile è ridotta, semplicemente meno job vengono ammessi e quelli in attesa rimangono in coda fino a quando la capacità non viene ripristinata.

I valori orari della VCC sono calcolati giornalmente attraverso un processo di ottimizzazione che utilizza previsioni di domanda per i workload flessibili e inflexible, modelli di traduzione dell'utilizzo CPU in consumo elettrico, e previsioni di carbon intensity oraria fornite da ElectricityMaps [6] per ciascuna zona della rete in cui risiedono i data center.

I risultati operativi riportati dagli autori mostrano che, nei cluster in cui il CICS è attivo, il consumo di potenza si riduce dell'1–2% nelle ore con la carbon intensity più alta rispetto ai cluster non ottimizzati.

Dal punto di vista concettuale, il CICS stabilisce il principio fondamentale del carbon-aware computing: la carbon intensity della rete elettrica può essere utilizzata come segnale operativo per modulare le decisioni di esecuzione, trasformando l'ottimizzazione del consumo energetico in un'ottimizzazione dell'impatto ambientale. È opportuno osservare, tuttavia, che il sistema opera esclusivamente attraverso temporal shifting di workload batch tolleranti al ritardo, su orizzonti temporali di 24 ore e a granularità di cluster. Non considera la possibilità di agire sulla logica applicativa dei workload né di selezionare tra varianti di esecuzione con profili diversi. Inoltre, il contesto operativo — data center hyperscale con job batch — è significativamente diverso da quello delle piattaforme FaaS, dove le funzioni sono invocate su richiesta con requisiti di latenza stringenti.

2.2.3.2 Spatial shifting carbon-aware per funzioni serverless

Se il CICS affronta il carbon-aware computing attraverso il temporal shifting di workload batch in data center hyperscale, GreenCourier [24] porta il paradigma nel contesto specifico delle piattaforme serverless, adottando una strategia di spatial shifting: anziché posticipare l'esecuzione nel tempo, il sistema distribuisce le invocazioni delle funzioni tra cluster Kubernetes situati in regioni geografiche diverse, privilegiando l'esecuzione nella regione con la carbon intensity marginale più bassa al momento dell'invocazione.

Come illustrato in Figura 2.9, il framework si integra con la piattaforma serverless Knative e si articola in tre componenti principali. Il *Metrics Server* raccoglie e normalizza in tempo reale i punteggi di carbon intensity per ciascuna regione geografica, utilizzando come fonti dati servizi quali WattTime o il Carbon-aware SDK della Green Software Foundation. Lo scheduler custom, implementato come plugin di scoring nel framework di scheduling di Kubernetes, assegna a ciascun nodo candidato un punteggio proporzionale alla carbon efficiency della regione in cui è localizzato. L'infrastruttura multi-cluster, realizzata tramite Liko, consente il *peering* tra cluster indipendenti distribuiti geograficamente e l'offloading trasparente delle funzioni verso regioni remote.

Il meccanismo decisionale opera per ciascuna invocazione: quando una funzione viene sottomessa, lo scheduler interroga il Metrics Server per ottenere i punteggi di carbon intensity correnti delle regioni disponibili, normalizza i valori e seleziona il nodo con il punteggio più favorevole. I punteggi vengono aggiornati con una granularità di cinque minuti, coerente con la frequenza di aggiornamento dei dati forniti dalle sorgenti di carbon intensity, e mantenuti in cache locale per ridurre l'overhead di comunicazione durante lo scheduling.

I risultati sperimentali, condotti su Google Kubernetes Engine con cluster distribuiti

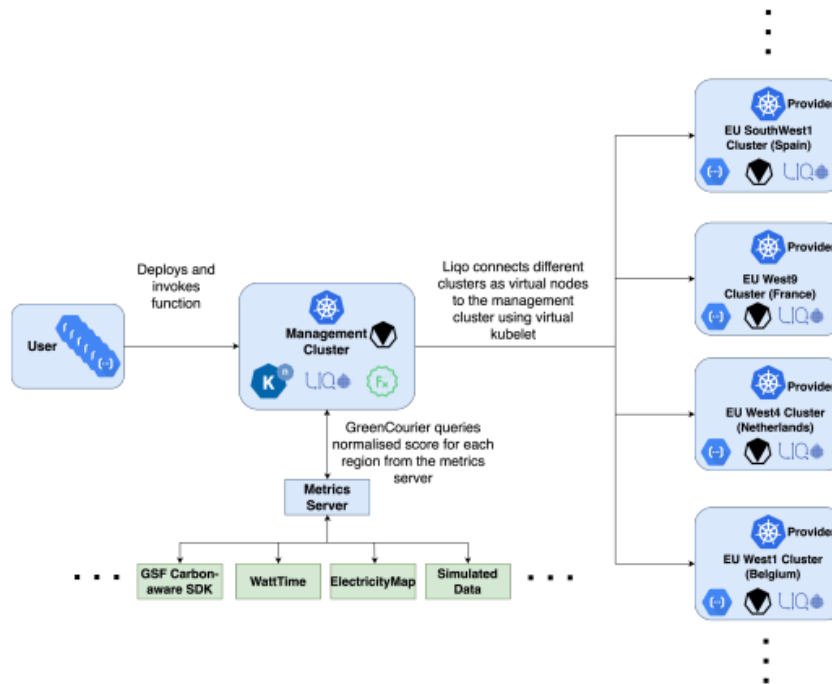


Figura 2.9: Architettura di GreenCourier. Il management cluster ospita lo scheduler carbon-aware e il Metrics Server, che interroga sorgenti di carbon intensity (WattTime, ElectricityMaps). I provider cluster, distribuiti in diverse regioni europee, sono collegati tramite Liqo come nodi virtuali. Lo scheduler assegna ciascuna invocazione alla regione con la carbon intensity più favorevole.

tra Spagna, Francia, Belgio e Paesi Bassi e utilizzando tracce di invocazione reali da Azure Functions, mostrano che GreenCourier riduce le emissioni di carbonio per invocazione in media del 13,25% rispetto alla strategia di scheduling predefinita di Kubernetes e del 17,8% rispetto a una strategia basata sulla prossimità geografica. Questo risultato è ottenuto al costo di un incremento medio del 10,26% nei tempi di risposta delle funzioni, dovuto all'esecuzione in regioni geograficamente più distanti ma con carbon intensity più favorevole.

Dal punto di vista della relazione con il contesto analizzato in questo capitolo, GreenCourier è il primo lavoro che applica il paradigma carbon-aware specificamente allo scheduling di funzioni serverless. A differenza del CICS, che opera su wor-

kload batch con flessibilità temporale, GreenCourier gestisce invocazioni on-demand con requisiti di risposta immediata, dimostrando che il spatial shifting carbon-aware è applicabile anche in contesti con vincoli di latenza.

I due contributi analizzati in questa sottosezione introducono la carbon intensity della rete elettrica come segnale di contesto per le decisioni di esecuzione, estendendo il criterio di ottimizzazione dal solo consumo energetico all'impatto ambientale complessivo. Entrambi gli approcci condividono tuttavia una caratteristica comune con le famiglie di soluzioni esaminate nelle sottosezioni precedenti: la carbon intensity viene utilizzata per decidere *dove* o *quando* eseguire un carico di lavoro, ma la funzione eseguita rimane invariante rispetto alle condizioni della rete elettrica, senza che il segnale ambientale orienti la selezione tra varianti con profili energia-accuratezza differenziati.

2.3 Il paradigma FaaS in ambienti cloud-edge

Negli ambienti cloud-edge l'elaborazione è distribuita su un insieme eterogeneo di nodi di calcolo, caratterizzati da differenti capacità computazionali, disponibilità di memoria, ampiezza di banda e latenza di rete. Nel paradigma Function-as-a-Service, le applicazioni sono scomposte in funzioni stateless invocate su richiesta, la cui esecuzione è demandata alla piattaforma. In un contesto cloud-edge, ciascuna funzione può essere eseguita localmente su un nodo edge, oppure instradata verso altri nodi o verso il cloud, in funzione dello stato delle risorse disponibili al momento dell'invocazione.

Le differenze strutturali tra nodi cloud e nodi edge rendono il consumo energetico un fattore particolarmente rilevante in questi scenari. I nodi edge dispongono tipicamente di risorse limitate e operano in condizioni di maggiore variabilità del carico, spesso in assenza di infrastrutture di raffreddamento dedicate. Di conseguenza, le decisioni di allocazione e gestione delle funzioni non possono basarsi esclusivamente su metriche

prestazionali, ma devono tenere conto dell'impatto energetico delle scelte effettuate, soprattutto in presenza di vincoli operativi stringenti.

Le tecniche discusse nelle sezioni precedenti — scheduling energeticamente consapevole, meccanismi applicativi basati su approximate computing e approcci carbon-aware — sono state sviluppate prevalentemente in contesti cloud o HPC, con assunzioni sull'infrastruttura che non sempre si trasferiscono direttamente all'ambiente cloud-edge. In particolare, la modularità della piattaforma serverless di riferimento è un requisito determinante: affinché meccanismi di controllo energetico possano essere integrati senza alterare il modello di interazione lato utente, l'architettura della piattaforma deve esporre punti di estensione adeguati a livello di scheduling e gestione delle funzioni.

In questa prospettiva, la scelta della piattaforma serverless su cui condurre la sperimentazione non è indifferente. Nella sezione seguente viene presentata Serverledge, la piattaforma adottata come riferimento nel presente lavoro, descrivendone le caratteristiche architettoniche rilevanti ai fini della ricerca.

2.3.1 La piattaforma Serverledge

Serverledge è una piattaforma Function-as-a-Service progettata per ambienti cloud-edge, caratterizzata da un'architettura decentralizzata che consente l'esecuzione di funzioni serverless all'interno di container. La piattaforma separa le componenti di controllo da quelle di esecuzione, permettendo una gestione distribuita delle istanze e delle informazioni di stato tra i nodi che compongono l'infrastruttura.

Ogni nodo Serverledge espone un'interfaccia per la creazione, l'eliminazione e l'invocazione delle funzioni, supportando sia la modalità sincrona sia quella asincrona. Le decisioni di esecuzione sono affidate a uno scheduler, responsabile del *placement* delle richieste sui nodi disponibili. L'esecuzione avviene tramite container Docker, man-

tenuti attivi per un intervallo di tempo limitato successivamente all'invocazione, con l'obiettivo di ridurre l'impatto dei *cold start* sulle invocazioni successive.

La condivisione delle informazioni tra nodi è realizzata tramite un meccanismo di storage distribuito, che consente di propagare metadati e informazioni di stato all'interno del cluster. Tali informazioni sono accessibili dallo scheduler al momento della decisione di *placement*, permettendo di scegliere tra l'esecuzione locale sul nodo ricevente e l'offloading verso altri nodi edge o verso il cloud, in funzione delle condizioni operative rilevate.

Dal punto di vista architetturale, Serverledge è progettata per essere estensibile: le logiche di scheduling e di gestione delle funzioni sono separate dal modello di interazione lato utente, rendendo possibile l'integrazione di componenti aggiuntivi di monitoraggio e controllo senza modificare l'interfaccia esposta agli utenti. Questa proprietà è particolarmente rilevante nel contesto del presente lavoro, in quanto consente di introdurre meccanismi di gestione energetica direttamente a livello di scheduler, sfruttando i punti di estensione nativi della piattaforma senza alterarne il comportamento di base.

2.4 Posizionamento del contributo

Le tre famiglie di soluzioni analizzate in questo capitolo — scheduling energy-aware, approximate computing e carbon-aware computing — operano su dimensioni distinte del problema: la scelta del contesto di esecuzione, la modulazione della logica applicativa e la sensibilità al segnale ambientale. Tuttavia, nessuno degli approcci esaminati le integra in un unico meccanismo decisionale.

In particolare, nessuno degli approcci esaminati affronta il problema di selezionare, al momento dell'invocazione, la variante di una funzione che offra il miglior compromesso tra consumo energetico e accuratezza del risultato in funzione dell'impatto am-

bientale corrente dell'esecuzione. Manca un meccanismo che utilizzi la carbon intensity non come criterio di *placement* geografico o temporale, ma come segnale per modulare il punto operativo sul trade-off energia-accuratezza della funzione stessa — spostando la selezione verso varianti a minor consumo quando la rete è alimentata da fonti ad alta intensità di carbonio, e verso varianti a maggiore accuratezza quando le condizioni della rete lo consentono.

A ciò si aggiunge una seconda lacuna, trasversale agli approcci esaminati: in ambienti FaaS, il consumo energetico di una funzione non è una grandezza stabile nel tempo, ma varia in funzione del carico concorrente sul nodo, dello stato del container e delle caratteristiche dei dati di input. Fondare le decisioni di selezione su profili energetici determinati offline significa operare su stime potenzialmente disallineate rispetto alle condizioni operative correnti, con conseguenze dirette sull'efficacia del meccanismo decisionale.

Il presente lavoro si propone di colmare questo spazio, integrando le tre dimensioni identificate in un unico sistema di scheduling che operi a livello di singola invocazione, selezionando tra varianti della stessa funzione sulla base di stime energetiche aggiornate continuamente e di un segnale di carbon intensity contestualizzato rispetto alla zona geografica di esecuzione.

Capitolo 3

Architettura del Sistema

In questo capitolo verranno descritte le scelte architetturali adottate per estendere la piattaforma Serverledge con le componenti necessarie a una schedulazione carbon-aware. Verranno presentati il modello di funzione logica con varianti, l'integrazione di Kepler per il monitoraggio dei consumi a livello di container, il meccanismo di aggiornamento adattivo dei profili energetici tramite media mobile esponenziale e il meccanismo di esplorazione LCB per il raffinamento delle stime nelle fasi iniziali. Lo scheduler che utilizza queste componenti per la selezione carbon-aware delle varianti è oggetto del Capitolo 4.

3.1 Integrazione di Kepler per il monitoraggio energetico

Il monitoraggio dei consumi energetici a livello di container costituisce il fondamento del sistema realizzato. A tal fine è stato adottato Kepler (Kubernetes-based Efficient Power Level Exporter), uno strumento open source sviluppato nell'ambito della CNCF (Cloud Native Computing Foundation) che stima il consumo energetico di processi, container e pod Kubernetes attraverso l'accesso diretto alle interfacce hardware del sistema operativo host [25].

La scelta di Kepler è motivata principalmente dalla sua granularità di misurazione, che opera a livello di singolo container Docker: questa corrisponde esattamente all'unità di esecuzione delle funzioni serverless in Serverledge. Kepler espone inoltre le proprie metriche nel formato standard Prometheus, consentendo un'integrazione diretta con l'ecosistema di monitoraggio senza richiedere componenti aggiuntivi per la raccolta dei dati.

3.1.1 Funzionamento di Kepler

Kepler opera combinando due meccanismi complementari per associare i consumi energetici ai singoli container. Il primo è un programma eBPF (Extended Berkeley Packet Filter) integrato nel kernel, che traccia l'attività dei processi e li associa ai container tramite il file `/proc/PID/cgroup`, consentendo di correlare il Process ID di ciascun processo all'identificatore del container che lo ospita. Il secondo è l'interfaccia RAPL (Running Average Power Limit), disponibile sui processori Intel e AMD moderni, che permette di leggere i contatori energetici cumulativi direttamente dai registri hardware della CPU.

Una volta raccolti i dati di utilization dei processi e di consumo energetico del sistema, Kepler applica il *Ratio Power Model* per attribuire a ciascun container la propria quota di consumo: la potenza dinamica viene distribuita tra i container in proporzione alla loro utilization rispetto all'utilization totale del sistema. Il risultato viene aggregato a livello di container e reso disponibile tramite Prometheus nella metrica `kepler_container_cpu_joules_total`, un contatore cumulativo espresso in Joule che cresce monotonicamente nel tempo per ogni container attivo.

Come illustrato in Figura 3.1, il comportamento di Kepler varia in base all'ambiente di esecuzione. In ambienti bare-metal, Kepler accede direttamente ai contatori

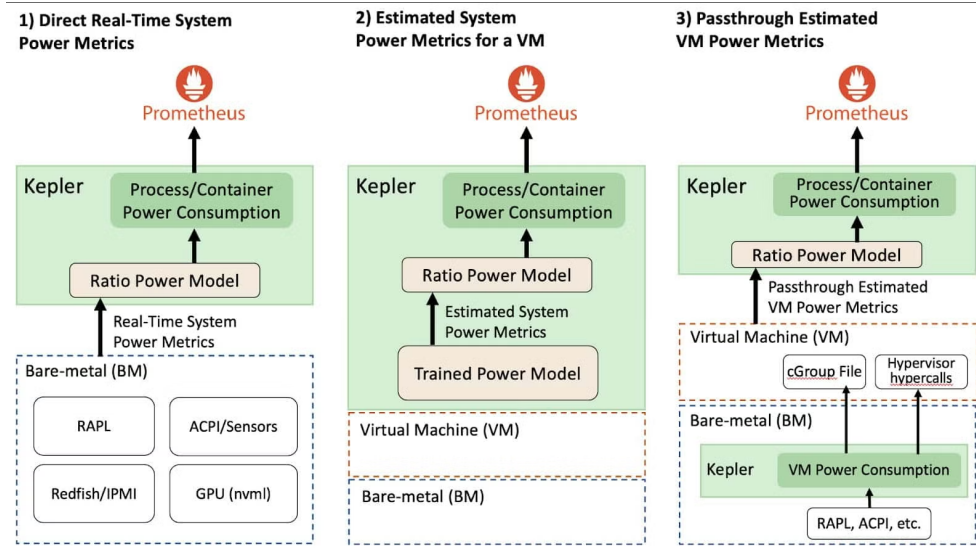


Figura 3.1: Architettura di Kepler per la raccolta dei consumi energetici in ambienti bare-metal e virtuali. In ambienti bare-metal, Kepler accede direttamente ai contatori hardware tramite RAPL, ottenendo misurazioni real-time senza necessità di modelli predittivi.

RAPL, ottenendo misurazioni energetiche real-time senza ricorrere a modelli predittivi pre-addestrati. In ambienti virtualizzati, invece, l'assenza di accesso diretto all'hardware richiede l'uso di modelli di regressione, introducendo le relative limitazioni di accuratezza.

Per accedere alle interfacce hardware necessarie, Kepler deve essere eseguito con privilegi elevati sull'host: richiede accesso a `/sys` e `/proc` per leggere i contatori RAPL e i cgroup, e al socket Docker per identificare i container attivi e associarvi le metriche energetiche.

3.1.2 Infrastruttura di supporto

L'avvio dell'infrastruttura di monitoraggio è gestito dallo script `StartMetrics.sh`, che inizializza come container Docker tutti i servizi necessari al funzionamento del sistema.

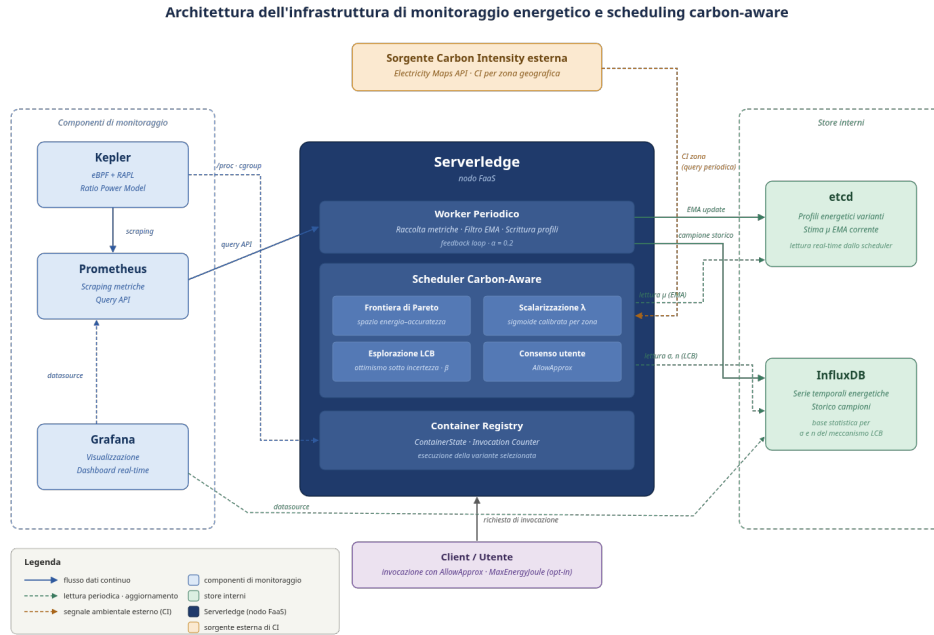


Figura 3.2: Architettura dell'infrastruttura di monitoraggio energetico e scheduling carbon-aware. I componenti di monitoraggio esterni (Kepler, Prometheus, Grafana) sono separati dagli store interni (etcd, InfluxDB), con Serverledge al centro che interagisce con entrambi i livelli. Lo scheduler carbon-aware integra la logica di selezione della variante — basata su frontiera di Pareto, scalarizzazione tramite λ , esplorazione LCB e consenso utente — e riceve il segnale di carbon intensity tramite query periodica a una sorgente esterna.

I componenti avviati, come mostrato anche in Figura 3.2, sono i seguenti:

- **Kepler**: eseguito in modalità privilegiata con accesso a `/sys`, `/proc` e al socket Docker dell'host, condizioni necessarie per la lettura dei contatori RAPL e l'associazione dei consumi ai container tramite cgroup.
- **Prometheus**: configurato per effettuare lo scraping periodico delle metriche esposte da Kepler, rendendole disponibili tramite API di interrogazione.
- **InfluxDB**: database a serie temporali utilizzato per l'archiviazione storica dei campioni energetici prodotti dal sistema ad ogni ciclo di raccolta.

- **etcd**: store distribuito chiave-valore che funge da registro delle funzioni e delle relative varianti, inclusi i profili energetici aggiornati dinamicamente a runtime.
- **Grafana**: strumento di visualizzazione utilizzato per il monitoraggio in tempo reale dei consumi durante gli esperimenti.

Una volta avviata, l'infrastruttura opera in modo autonomo: Kepler rileva i container Docker presenti sull'host e ne aggiorna le metriche energetiche ad ogni ciclo di campionamento; Prometheus le raccoglie e le rende interrogabili; il sistema interno di Serverledge, descritto nelle sezioni successive, le legge periodicamente — limitatamente ai container attivi da esso gestiti — per aggiornare i profili energetici delle varianti di funzione. Il segnale di carbon intensity, necessario allo scheduler per la selezione carbon-aware della variante, non è parte di questa infrastruttura: viene acquisito dallo scheduler a runtime tramite query periodica a una sorgente esterna, come descritto nella Sezione 4.

3.2 Il modello di funzione logica con varianti

Il supporto all'esecuzione energeticamente consapevole richiede che il sistema possa rappresentare, per una stessa funzione, implementazioni alternative con diversi profili di consumo energetico e livello di accuratezza. A tal fine è stato introdotto il concetto di *funzione logica con varianti*: un'astrazione che raggruppa sotto un unico identificatore logico più implementazioni della stessa funzione, ciascuna caratterizzata da un proprio profilo energetico e da un modello di output che ne quantifica l'accuratezza.

3.2.1 Struttura della funzione e profilo energetico

In Serverledge, ogni funzione è rappresentata dalla struttura **Function**, estesa in questo lavoro con i campi necessari al supporto delle varianti energetiche. I campi rilevanti

per il sistema proposto sono i seguenti:

- **LogicalName**: identificatore condiviso da tutte le varianti appartenenti alla stessa funzione logica. Costituisce il riferimento con cui il client invoca la funzione, indipendentemente dalla variante che verrà selezionata a runtime. La registrazione delle varianti avviene su consenso esplicito dell'utente in fase di creazione della funzione;
- **VariantID**: identificatore univoco della specifica variante all'interno del gruppo logico;
- **EnergyProfile**: struttura che mantiene la stima energetica della variante, composta da due campi:
 - **ColdStartJoule**: energia stimata per l'avvio a freddo del container, comprensiva dell'inizializzazione del runtime;
 - **InvocationJoule**: energia media stimata per una singola invocazione della funzione in warm start.
- **OutputModel**: struttura che descrive la qualità dell'output della variante, utilizzata dallo scheduler per il confronto tra varianti in termini di accuratezza. Il suo contenuto è descritto in dettaglio nella Sezione 3.2.2.

3.2.1.1 Determinazione dei profili energetici iniziali

I valori iniziali di **EnergyProfile** sono stati determinati offline tramite una campagna di misurazioni preliminari condotta con Kepler. La metodologia adottata tiene conto della natura delle metriche esposte da Kepler: trattandosi di contatori cumulativi,

l'energia osservata al termine di n invocazioni consecutive sullo stesso container rappresenta la somma dei contributi di tutte le esecuzioni, inclusa quella iniziale soggetta al cold start.

Per separare i due contributi energetici si è proceduto nel modo seguente. Dato un container su cui vengono eseguite n invocazioni consecutive, si indica con E_{tot} l'energia cumulativa totale rilevata da Kepler al termine dell'ultima invocazione e con E_1 l'energia della prima invocazione, che comprende sia il costo di inizializzazione del container che quello di esecuzione della funzione. Le invocazioni successive, eseguite a container già attivo, sono invece soggette al solo costo di esecuzione. Il costo medio di invocazione in warm start è quindi stimato come:

$$E_{inv} = \frac{E_{tot} - E_1}{n - 1} \quad (3.2.1)$$

e il costo di cold start come:

$$E_{cold} = E_1 - E_{inv} \quad (3.2.2)$$

La ripetizione delle misurazioni su un numero statisticamente significativo di invocazioni consente di attenuare la varianza introdotta dal sistema operativo e dal runtime, ottenendo stime robuste per entrambi i contributi. I valori così ottenuti vengono codificati nel file JSON di definizione delle varianti e costituiscono la stima iniziale del profilo energetico. Il Codice 3.1 mostra un esempio concreto per la funzione `PiLeibniz`, che espone tre varianti esatte a runtime diverso e sei varianti approssimate con numero di iterazioni ridotto.

Codice 3.1: File di definizione delle varianti per la funzione `PiLeibniz`. Le prime due varianti sono esatte (`is_approximate: false`) e differiscono per runtime e consumo. (omesse per brevità le quattro varianti intermedie).

```
1 {  
2   "variants": [  
3     {  
4       "id": "base",
```

```
5     "runtime": "python310",
6     "entry_point": "piLeibniz.handler",
7     "src": "piLeibniz.py",
8     "energy": {
9         "cold_start_joule": 0.070134,
10        "invocation_joule": 0.007823
11    },
12    "output": { "type": "error",
13                "error_estimate": 0.0 },
14    "is_approximate": false
15 },
16 {
17     "id": "c",
18     "runtime": "native",
19     "entry_point": "piLeibniz_base",
20     "src": "piLeibniz_base",
21     "energy": {
22         "cold_start_joule": 0.008847,
23         "invocation_joule": 0.001052
24     },
25     "output": { "type": "error",
26                 "error_estimate": 0.0 },
27     "is_approximate": false
28 },
29 {
30     "id": "n1000",
31     "runtime": "python310",
32     "entry_point": "piLeibniz_n1000.handler",
33     "src": "piLeibniz_n1000.py",
34     "energy": {
35         "cold_start_joule": 0.070681,
36         "invocation_joule": 0.000196
37     },
38     "output": { "type": "error",
39                 "error_estimate": 0.001999 },
40     "is_approximate": true
41 },
42 {
43     "id": "n200000",
44     "runtime": "python310",
45     "entry_point": "piLeibniz_n200000.handler",
46     "src": "piLeibniz_n200000.py",
47     "energy": {
48         "cold_start_joule": 0.069587,
49         "invocation_joule": 0.001534
```

```
50     },  
51     "output": { "type": "error",  
52                 "error_estimate": 0.000010 },  
53     "is_approximate": true  
54 }  
55 ]  
56 }
```

I valori iniziali del profilo energetico sono progressivamente raffinati a runtime attraverso due meccanismi complementari, descritti nelle Sezioni 3.3 e 3.4: il worker periodico aggiorna le stime delle varianti effettivamente eseguite tramite filtro EMA, mentre il meccanismo di esplorazione LCB permette di correggere le stime delle varianti sotto-campionate, prevenendo che una stima inizialmente errata le escluda permanentemente dalla selezione. I risultati numerici delle misurazioni preliminari per tutte le varianti sono riportati nel Capitolo 5.

3.2.2 Il modello di output e la classificazione dell'accuratezza

Per consentire allo scheduler di confrontare le varianti in termini di accuratezza, ciascuna variante è dotata di un `OutputModel` che descrive la qualità del suo output. Il modello supporta due tipologie di classificazione, scelte in base alla natura computazionale della funzione:

- **Tipo error:** adatto a funzioni il cui output è un valore numerico misurabile. L'accuratezza è espressa tramite il campo `error_estimate`, che rappresenta l'errore relativo atteso rispetto alla variante di riferimento. Una variante esatta presenta `error_estimate = 0.0`, mentre varianti approssimate presentano valori positivi proporzionali alla deviazione introdotta dalla semplificazione algoritmica adottata.

- **Tipo quality:** adatto a funzioni il cui output non è facilmente quantificabile con un errore numerico, come classificatori o modelli di machine learning. L'accuratezza è espressa tramite il campo numerico `quality_score` $\in [0, 1]$, dove 1 corrisponde all'accuratezza massima;

Il Codice 3.2 mostra un esempio applicativo del tipo `quality` per la funzione `ImageClassification`, in cui la variante base utilizza un modello ViT più accurato ma più oneroso energeticamente, mentre le varianti successive adottano architetture più compatte (EfficientNet-B0, MobileNet-V2, MobileNet-V2-Tiny) a qualità progressivamente ridotta.

Codice 3.2: File di definizione delle varianti per la funzione `ImageClassification`. Il campo `quality_score` riporta la Top-1 accuracy su ImageNet pubblicata dagli autori di ciascun modello, espressa in $[0, 1]$.

```
1 {
2   "variants": [
3     {
4       "id": "vit-base",
5       "runtime": "python-ml",
6       "entry_point": "ImageClassification.handler",
7       "src": "ImageClassification.py",
8       "energy": { "cold_start_joule": 1.895,
9                  "invocation_joule": 1.455 },
10      "output": { "type": "quality",
11                 "quality": "high",
12                 "quality_score": 0.818 },
13      "is_approximate": false
14    },
15    {
16      "id": "efficientnet-b0",
17      "runtime": "python-ml",
18      "entry_point":
19        "ImageClassificationEfficientNetB0.handler",
20      "src":
21        "ImageClassificationEfficientNetB0.py",
22      "energy": { "cold_start_joule": 1.183,
23                 "invocation_joule": 0.418 },
24      "output": { "type": "quality",
25                 "quality": "medium-high",
```



```

26         "quality_score": 0.771 },
27     "is_approximate": true
28 },
29 {
30     "id": "mobilenet-v2",
31     "runtime": "python-ml",
32     "entry_point":
33         "ImageClassificationLight.handler",
34     "src": "ImageClassificationLight.py",
35     "energy": { "cold_start_joule": 1.071,
36                 "invocation_joule": 0.229 },
37     "output": { "type": "quality",
38                 "quality": "medium-low",
39                 "quality_score": 0.718 },
40     "is_approximate": true
41 },
42 {
43     "id": "mobilenet-v2-tiny",
44     "runtime": "python-ml",
45     "entry_point":
46         "ImageClassificationMobileNetTiny.handler",
47     "src":
48         "ImageClassificationMobileNetTiny.py",
49     "energy": { "cold_start_joule": 0.779,
50                 "invocation_joule": 0.077 },
51     "output": { "type": "quality",
52                 "quality": "low",
53                 "quality_score": 0.603 },
54     "is_approximate": true
55 }
56 ]
57 }

```

3.2.3 Registrazione e indicizzazione delle varianti in etcd

Le varianti di una funzione logica vengono definite tramite un file JSON collocato nella directory `variants/<LogicalName>/` del progetto. Al momento della registrazione, il sistema legge questo file attraverso il componente `FileSource`, prepara il codice sorgente di ciascuna variante in formato tar e materializza le varianti in etcd attraverso la funzione `CreateInternalVariants`.

3.2.3.1 Convenzione di denominazione

La prima variante definita nel file JSON —corrispondente alla funzione di base— mantiene come nome interno esattamente il `LogicalName`, preservando la compatibilità con le invocazioni che non specificano alcuna preferenza energetica. Le varianti successive ricevono un nome nella forma `<LogicalName>-<VariantID>`. Per la funzione `PiLeibniz` (Codice 3.1), ad esempio, i nomi interni assegnati sono `PiLeibniz`, `PiLeibniz-optimization-py`, `PiLeibniz-c`, `PiLeibniz-n1000`, e così via.

3.2.3.2 Struttura delle chiavi in etcd

Il sistema mantiene in etcd due livelli distinti di chiavi, come illustrato in Figura 3.3. Il prefisso `/function/` contiene una chiave per ogni variante registrata, il cui valore è la serializzazione JSON completa della struttura `Function` — runtime, codice sorgente, profilo energetico (`ColdStartJoule` e `InvocationJoule`) e modello di output. Il prefisso `/index/logical/<LogicalName>/` costituisce invece un indice che, tramite una singola query a prefisso, consente allo scheduler di recuperare tutte le varianti di una funzione logica senza scorrere l'intero `/function/` contenendo in tal modo la latenza introdotta dalla selezione della variante nel path della richiesta di invocazione.

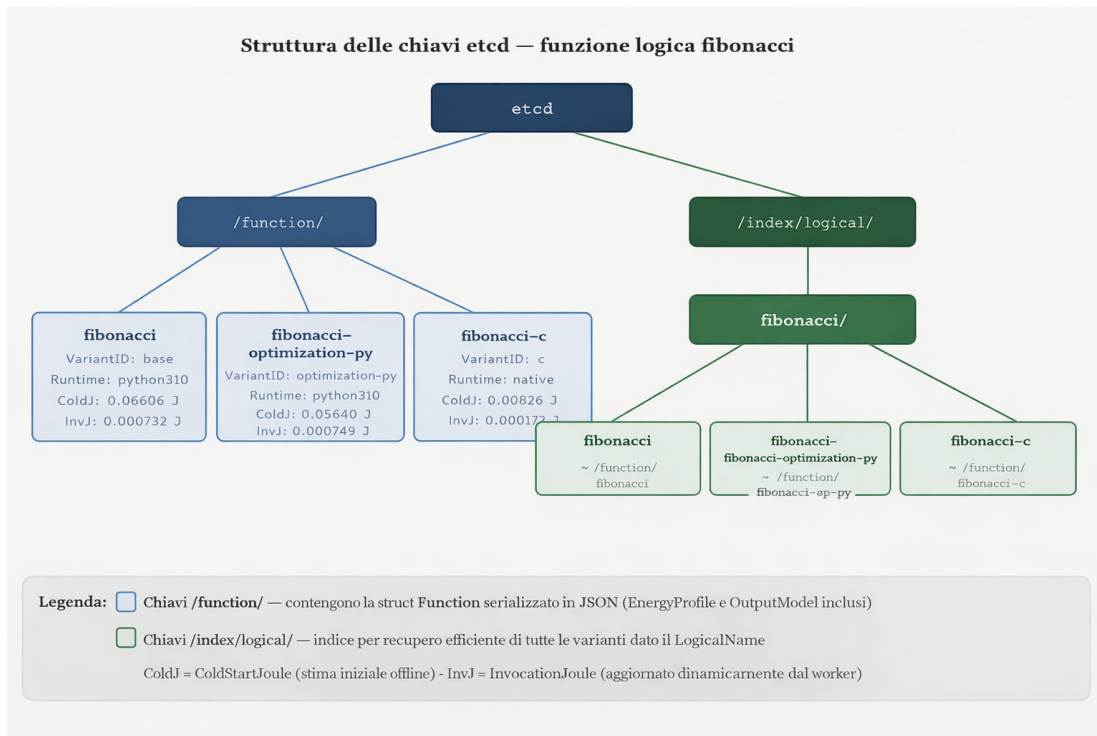


Figura 3.3: Struttura delle chiavi etcd per la funzione logica PiLeibniz. Le chiavi sotto `/function/` contengono i profili completi; l'indice sotto `/index/logical/` consente la lookup efficiente delle sole varianti richieste.

3.3 Misurazione e aggiornamento adattivo dei profili energetici

Una volta introdotto il modello di funzione logica con varianti multiple, è necessario dotare il sistema di un meccanismo capace di aggiornare dinamicamente i profili energetici sulla base dei consumi effettivamente osservati a runtime. Un approccio statico, in cui il costo energetico di una variante viene stimato offline e rimane invariato nel tempo, risulta inadeguato in contesti di esecuzione reali: il consumo energetico di un container è influenzato da fattori mutevoli quali il carico del nodo, le interferenze tra istanze concorrenti e le variazioni dinamiche della frequenza di CPU.

La soluzione adottata è un *feedback loop* periodico che trasforma il profilo energe-

tico in una grandezza adattiva, aggiornata continuamente in funzione del contesto di esecuzione. Il componente responsabile è un worker di raccolta eseguito in background, illustrato in Figura 3.4.

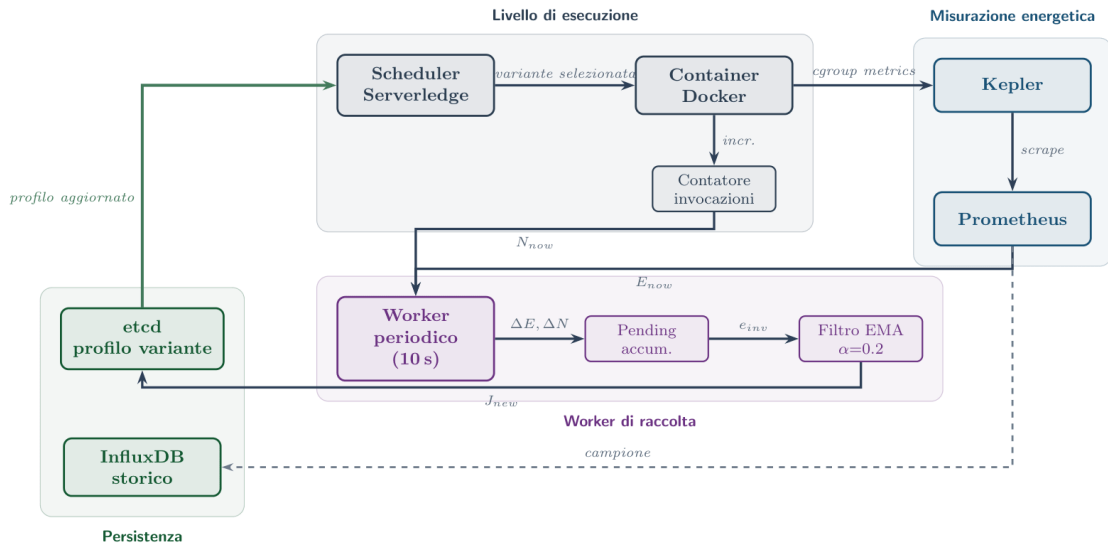


Figura 3.4: Schema del feedback loop energetico. Il worker periodico legge le metriche cumulative da Kepler tramite Prometheus e il numero di invocazioni dal contatore locale, calcola i delta, applica un filtro EMA e aggiorna il profilo persistito in etcd, archiviando contestualmente i campioni in InfluxDB.

3.3.1 Tracciamento dei container e conteggio delle invocazioni

Per attribuire correttamente il consumo energetico alle singole varianti, il sistema deve mantenere una corrispondenza stabile tra i container Docker attivi e le funzioni in esecuzione. A tal fine il package `energy` mantiene un registro in memoria che associa a ciascun container una struttura `ContainerState` contenente:

- l'identificatore del container (`ContainerID`);
- il nome interno della variante (`FunctionName`), il `LogicalName` e il `VariantID`;

- i valori di baseline dell'energia cumulativa e del numero di invocazioni (`LastJoule`, `LastInvocations`), impiegati per il calcolo dei delta incrementali;
- gli accumulatori `PendingEnergyJ` e `PendingInvocations`, necessari per gestire il disallineamento temporale tra metriche energetiche e contatori locali.

La registrazione avviene sia alla creazione del container sia, in modo difensivo, al momento dell'invocazione, così da gestire correttamente il caso di *warm start*. Se un container è già presente nel registro, la sua identità (mapping container-variante) non viene modificata, garantendo coerenza per l'intero ciclo di vita del container.

Parallelamente, viene mantenuto un contatore atomico di invocazioni per container, incrementato al completamento di ogni richiesta.

3.3.2 Il worker periodico di raccolta

Il worker è implementato come una goroutine avviata all'inizializzazione della piattaforma. Con cadenza di 10 secondi, esegue un ciclo di raccolta (`Collect`) operando su uno snapshot thread-safe del `ContainerRegistry`, in modo da isolare la lettura dallo stato condiviso senza bloccare le operazioni concorrenti.

Per ciascun container attivo vengono acquisiti:

- il valore cumulativo di energia E_{now} , esposto da Kepler e reso disponibile tramite l'endpoint Prometheus;
- il numero cumulativo di invocazioni N_{now} , mantenuto aggiornato dal contatore locale.

Vengono quindi calcolati i delta rispetto all'osservazione precedente:

$$\Delta E = E_{now} - E_{last} \quad \Delta N = N_{now} - N_{last} \quad (3.3.1)$$

La presenza di valori negativi segnala un reset del container o del contatore, condizione possibile ad esempio in seguito a un riavvio dell'istanza, e comporta la reinizializzazione delle variabili senza produzione di campioni, evitando attribuzioni energetiche errate.

3.3.3 Attribuzione dell'energia per invocazione e gestione del lag

Le metriche esposte da Kepler hanno natura cumulativa e sono soggette a un ritardo di campionamento intrinseco al ciclo di scraping di Prometheus. Il contatore locale di invocazioni, al contrario, viene aggiornato immediatamente al termine di ogni esecuzione. Può quindi verificarsi la condizione:

$$\Delta N > 0 \quad \text{e} \quad \Delta E = 0$$

in cui nuove invocazioni sono già state completate ma il relativo consumo energetico non è ancora riflesso nella metrica cumulativa. Calcolare in questo caso il consumo medio per invocazione produrrebbe una sottostima sistematica.

Per gestire questo disallineamento, i delta vengono accumulati nei campi `PendingEnergyJ` e `PendingInvocations`. L'attribuzione effettiva avviene solo quando entrambi gli accumulatori risultano positivi, garantendo che l'energia osservata e il numero di esecuzioni considerate siano coerenti tra loro:

$$e_{inv} = \frac{E_{pending}}{N_{pending}} \tag{3.3.2}$$

Il valore e_{inv} rappresenta la stima istantanea del consumo medio per invocazione nel periodo considerato e costituisce l'ingresso per la fase di aggiornamento adattivo del profilo.

3.3.4 Aggiornamento del profilo tramite media mobile esponenziale

La stima istantanea e_{inv} può essere affetta da rumore dovuto a interferenze tra container, variazioni dinamiche della frequenza di CPU o fluttuazioni del carico di sistema. Applicare direttamente tale valore al profilo energetico potrebbe causare oscillazioni indesiderate nelle decisioni dello scheduler.

Per stabilizzare la stima viene applicata una *Exponential Moving Average* (EMA):

$$J_{new} = \alpha \cdot e_{inv} + (1 - \alpha) \cdot J_{old} \quad (3.3.3)$$

con $\alpha = 0.2$. Il parametro α governa il compromesso tra due esigenze contrastanti:

- **Reattività:** valori elevati rendono il profilo rapidamente adattabile a variazioni strutturali del consumo energetico;
- **Stabilità:** valori ridotti attenuano il contributo di singoli campioni anomali, producendo una stima più robusta.

La scelta di $\alpha = 0.2$ rappresenta un equilibrio empirico: il sistema filtra efficacemente i picchi transitori, mantenendo al contempo una capacità di adattamento progressivo al nuovo regime energetico nell'arco di circa $1/\alpha$ cicli di aggiornamento, corrispondenti a circa 50 secondi con la cadenza attuale del worker.

Il valore aggiornato J_{new} viene scritto nel campo `InvocationJoule` della variante corrispondente e persistito in etcd, rendendolo immediatamente disponibile allo scheduler per le decisioni di selezione successive.

3.3.5 Archiviazione su InfluxDB

Contestualmente all'aggiornamento in etcd, ogni campione viene archiviato in InfluxDB come punto della *measurement* `energy_sample`, con tag `container_id`, `logical_name`, `function_name`, `variant_id` e campo `invocation_joule`.

La separazione tra i due store riflette le loro funzioni distinte: etcd fornisce allo scheduler la stima corrente con latenza minima; InfluxDB preserva la serie storica dei campioni, che costituisce la base statistica per il meccanismo di esplorazione descritto nella Sezione 3.4.

3.4 Esplorazione LCB per il raffinamento delle stime energetiche

Il meccanismo EMA descritto nella sezione precedente consente di raffinare i profili energetici a runtime, ma presenta un prerequisito implicito: aggiorna solo le varianti che vengono effettivamente eseguite. Se la stima iniziale di una variante è sovrastimata rispetto al suo consumo reale, lo scheduler tenderà a non selezionarla, il worker non raccoglierà mai campioni su di essa, impedendo la correzione dell'errore. Il problema si manifesta in particolare nelle fasi iniziali del sistema, quando i profili offline (determinati su un hardware di riferimento) potrebbero non riflettere il comportamento reale sul nodo edge in produzione, sia per la *dipendenza dall'hardware* (CPU, workload e condizioni termiche differenti), sia per la *non stazionarietà* del consumo energetico (che varia con il carico concorrente, la temperatura del processore e lo stato della cache del runtime). In queste condizioni, una variante con consumo energetico sovrastimato tende ad essere sistematicamente penalizzata nella scalarizzazione Pareto e a non venire mai selezionata, impedendo al meccanismo EMA di raccogliere campioni reali su di

Simbolo	Significato
μ_i^{EMA}	Stima EMA dell'energia della variante i , letta da etcd ($\alpha = 0.2$)
σ_i	Deviazione standard calcolata sui campioni InfluxDB.
n_i^{eff}	Numero effettivo di campioni: $n_i + 0.5$
β	Parametro di esplorazione (configurabile, default 1.0)

Tabella 3.1: Simboli della formula LCB e relativi valori.

lei e correggere l'errore. È necessario quindi introdurre un meccanismo che promuova l'esplorazione di varianti poco osservate, indipendentemente dalla loro stima corrente.

3.4.1 Il principio di ottimismo sotto incertezza

Il meccanismo adottato si basa sul principio di *ottimismo sotto incertezza* (Optimism in the Face of Uncertainty) [26]. Tale meccanismo incentiva l'esplorazione di varianti sotto-campionate, raccogliendo dati reali e convergendo progressivamente a stime accurate.

Per applicare il principio alla minimizzazione del consumo energetico si utilizza il Lower Confidence Bound (LCB), equivalente all'UCB classico:

$$\text{LCB}(i) = \mu_i^{\text{EMA}} - \beta \cdot \frac{\sigma_i}{\sqrt{n_i^{\text{eff}}}} \quad (3.4.1)$$

dove $n_i^{\text{eff}} = n_i + 0.5$, con n_i il numero di campioni InfluxDB disponibili per la variante i . L'addendo costante 0.5 evita la divisione per zero nel caso $n_i = 0$ e assicura la massima correzione ottimistica per varianti mai osservate. La Tabella 3.1 descrive i simboli della formula.

Una LCB bassa segnala che la variante è sotto-esplorata: il sistema la favorisce nella selezione per raccogliere dati aggiuntivi. All'aumentare di n_i , la correzione si azzerava e la stima converge alla media empirica.

3.4.2 Proprietà di convergenza

La formula (3.4.1) gode delle seguenti proprietà asintotiche, che ne garantiscono il corretto bilanciamento tra esplorazione e sfruttamento:

- quando n_i è grande (variante ben esplorata), il termine $\sigma_i/\sqrt{n_i^{\text{eff}}}$ tende a zero e $\text{LCB}(i) \rightarrow \mu_i^{\text{EMA}}$: il sistema sfrutta la stima corrente senza alterarla;
- quando $n_i = 0$ (variante mai osservata), la correzione è massima ($n_i^{\text{eff}} = 0.5$)
- $\beta = 0$ disabilita completamente l'esplorazione, riducendo la formula alla sola stima EMA;
- in caso di indisponibilità di InfluxDB, il sistema degrada a μ^{EMA} senza correzione, garantendo il funzionamento anche in assenza del componente esterno.

Le componenti descritte in questo capitolo costituiscono l'infrastruttura su cui si fonda la schedulazione carbon-aware: il modello di funzione logica con varianti fornisce lo spazio decisionale, Kepler ne misura i consumi a runtime, il worker periodico aggiorna i profili energetici tramite filtro EMA e il meccanismo LCB garantisce che anche le varianti sotto-campionate vengano esplorate nelle fasi iniziali. Il capitolo seguente descrive come questi elementi vengano integrati nello scheduler carbon-aware, che li utilizza per selezionare, a ogni invocazione, la variante più appropriata in funzione della carbon intensity della zona geografica del nodo.

Capitolo 4

Lo scheduler carbon-aware

In questo capitolo verrà descritto lo scheduler carbon-aware, il componente decisionale che costituisce il nucleo dell'estensione proposta. Verranno presentati il meccanismo di normalizzazione della carbon intensity in un peso decisionale contestualizzato per zona geografica, la costruzione della frontiera di Pareto nello spazio energia-accuratezza e la logica di scalarizzazione che seleziona, a ogni invocazione, la variante più appropriata in funzione del segnale ambientale corrente.

4.1 La carbon intensity come segnale decisionale

Come già detto precedentemente, la carbon intensity (CI) esprime la quantità di CO₂ equivalente emessa per ogni kilowattora prodotto dalla rete elettrica locale, misurata in g CO₂eq/kWh. Questo valore riflette la composizione del mix energetico di una zona geografica in un dato istante: una rete alimentata prevalentemente da fonti rinnovabili presenta valori di CI ridotti, mentre una rete dipendente da combustibili fossili mostra valori elevati.

Nel contesto di questo lavoro, la CI viene utilizzata come segnale per derivare il parametro di trade-off $\lambda \in (0, 1)$ che governa la selezione della variante (Sezione 4.2): quando l'energia disponibile è a bassa intensità carbonica, il sistema può privilegiare

l'accuratezza del risultato senza un costo ambientale significativo; quando la rete è carbon-intensive, è preferibile ridurre i consumi a scapito dell'accuratezza.

Tuttavia, il valore assoluto della CI non è sufficiente per prendere questa decisione: una CI di 100 g CO₂/kWh rappresenta un picco anomalo per la Norvegia, ma è del tutto ordinaria per la Spagna. Stabilire se l'energia è più o meno “green” in un dato momento richiede quindi un riferimento contestuale, ovvero la distribuzione storica della CI per la zona geografica considerata.

È necessario pertanto un modello di normalizzazione che trasformi la CI corrente in un peso $\lambda \in (0, 1)$ relativo al contesto storico della zona. La semantica adottata è la seguente:

- $\lambda \rightarrow 1$: la CI corrente è bassa rispetto alla distribuzione storica della zona (rete elettrica relativamente *pulita*); il sistema privilegia l'accuratezza del risultato;
- $\lambda \rightarrow 0$: la CI corrente è elevata (rete *sporca*); il sistema privilegia l'efficienza energetica.

4.1.1 La funzione sigmoide calibrata per zona

Per tradurre la CI corrente in un valore di λ relativo al contesto storico della zona, si adotta una funzione sigmoide logistica nella forma:

$$\lambda(CI) = \frac{1}{1 + \exp\left(k \cdot \frac{CI - \mu_z}{s_z}\right)} \quad (4.1.1)$$

dove μ_z è la media storica della CI per la zona geografica z (punto di simmetria in cui $\lambda = 0,5$), s_z è un fattore di scala derivato dalla dispersione storica locale, e $k > 0$ è un parametro di pendenza che controlla la velocità di transizione. La scelta di questa famiglia di funzioni è motivata da tre proprietà desiderabili nel contesto dello scheduling:

- **Monotonicità decrescente:** λ decresce monotonamente al crescere di CI, associando in modo coerente condizioni di rete più carbon-intensive a un peso maggiore sull'efficienza energetica.
- **Saturazione:** λ tende asintoticamente a 1 per CI basse e a 0 per CI alte.
- **Sensibilità contestuale:** i parametri μ_z e s_z sono calibrati a partire dalla distribuzione storica della CI nella zona z , permettendo alla sigmoide di adattarsi al contesto locale. In questo modo, il valore di λ esprime una misura relativa rispetto alla norma locale, rendendo confrontabili le decisioni di scheduling tra aree geografiche diverse, anche quando i livelli assoluti di carbon intensity differiscono significativamente.

Calibrazione dei parametri. I parametri μ_z , s_z e k sono stimati a partire dai dati storici di carbon intensity per ciascuna zona, recuperati da [6]. La procedura è la seguente:

1. μ_z è posto pari alla *media* storica della CI per la zona, garantendo $\lambda(\mu_z) = 0,5$, ovvero che il punto di bilanciamento neutro corrisponda al valore tipico della zona;
2. s_z è ricavato dal quinto e novantacinquesimo percentile della distribuzione storica:

$$s_z = \frac{CI_{95} - CI_5}{2 \ln\left(\frac{1-\alpha}{\alpha}\right)} \quad (4.1.2)$$

con $\alpha = 0,01$. L'utilizzo dei percentili estremi consente di catturare l'intervallo di variabilità tipica della CI nella zona, trascurando i valori anomali. In particolare, il 90 % centrale della distribuzione storica viene mappato nell'intervallo $[\alpha, 1 - \alpha]$ di λ , garantendo che la funzione sia sufficientemente sensibile alle variazioni reali della CI, senza risultare distorta dalla presenza di outlier.

3. k è un parametro di pendenza configurabile per zona: valori più alti producono una transizione più netta attorno a μ_z , valori più bassi una risposta più graduale.

Calcolo a runtime.

Algoritmo 1 Calcolo del peso λ dalla carbon intensity corrente

Require: zona geografica z , soglia di fallback $\lambda_0 = 0,5$

Ensure: $\lambda \in (0, 1)$

```

1:  $CI \leftarrow \text{GETCARBONINTENSITY}(z)$  ▷ API ElectricityMaps
2: if  $CI$  non disponibile then
3:   return  $\lambda_0$ 
4:  $\mu_z \leftarrow \text{GETMEAN}(z)$ 
5:  $s_z \leftarrow \frac{CI_{95}(z) - CI_5(z)}{2 \ln\left(\frac{1-\alpha}{\alpha}\right)}$ 
6:  $k \leftarrow \text{GETSLOPEPARAM}(z)$ 
7: return  $\frac{1}{1 + \exp\left(k \cdot \frac{CI - \mu_z}{s_z}\right)}$ 

```

Al momento dell’invocazione, lo scheduler recupera la CI corrente tramite l’API di ElectricityMaps [6], legge i parametri della zona, e calcola λ secondo l’Equazione (4.1.1), come descritto nell’algoritmo 1. In caso di indisponibilità dell’API, il sistema degrada a $\lambda = 0,5$ (bilanciamento neutro tra accuratezza ed efficienza energetica).

4.2 La frontiera di Pareto nello spazio energia-errore

Il valore di λ derivato nella sezione precedente costituisce il segnale che guida la selezione della variante da eseguire: un λ prossimo a 1 orienta lo scheduler verso le varianti più accurate, mentre un λ prossimo a 0 privilegia quelle a minore consumo energetico. Tuttavia, prima di applicare tale peso, è necessario determinare l’insieme di varianti su cui operare. Non tutte le varianti disponibili rappresentano scelte razionali: alcune sono dominate da altre, ovvero esistono alternative che consumano meno energia

e producono un errore inferiore. Includere tali varianti nella selezione introdurrebbe ridondanza senza apportare benefici decisionali. Il sistema restringe quindi il dominio di scelta alle sole varianti *Pareto-ottimali*, ovvero quelle per cui non esiste alcuna alternativa migliore in entrambe le dimensioni simultaneamente.

Definizione. Sia $V = \{v_1, \dots, v_n\}$ l'insieme delle varianti disponibili. Una variante v_i è *dominata* da v_j se:

$$E(v_j) \leq E(v_i) \wedge Err(v_j) \leq Err(v_i) \wedge (E(v_j) < E(v_i) \vee Err(v_j) < Err(v_i)) \quad (4.2.1)$$

La frontiera di Pareto $\mathcal{P} \subseteq V$ è l'insieme delle varianti non dominate:

$$\mathcal{P} = \{v \in V \mid \nexists v' \in V : v' \text{ domina } v\} \quad (4.2.2)$$

Ogni variante sulla frontiera rappresenta un compromesso efficiente: ridurre una delle due grandezze implica necessariamente aumentare l'altra. È su questo insieme ridotto che λ esercita la propria funzione di selezione.

Costruzione. La frontiera è costruita a runtime immediatamente prima di ogni selezione, a partire dai profili energetici correnti. La scelta di ricostruire la frontiera a ogni invocazione, anziché mantenerla precalcolata, è motivata dal fatto che i profili energetici sono aggiornati dinamicamente: nella fase iniziale, quando le misurazioni sono ancora scarse, la frontiera evolve a ogni invocazione in funzione delle nuove osservazioni raccolte. Man mano che le stime energetiche convergono, la frontiera si stabilizza, e le ricostruzioni successive producono risultati sostanzialmente invariati. Il costo computazionale della ricostruzione a runtime è quindi giustificato nella fase esplorativa e diventa trascurabile a regime, quando la frontiera è ormai stabile.

Come coordinata energetica non viene utilizzata la stima EMA grezza, bensì la stima LCB descritta nella Sezione 3.4. Questa scelta è intenzionale: il termine di

esplorazione abbassa la stima energetica delle varianti sotto-campionate, rendendole apparentemente più convenienti. Di conseguenza, tali varianti tendono a posizionarsi più favorevolmente sulla frontiera e a essere selezionate più frequentemente, accumulando osservazioni reali che affinano progressivamente la stima EMA. L'esplorazione non è quindi un meccanismo separato dalla selezione, ma è integrata nella costruzione stessa della frontiera: ogni invocazione rappresenta simultaneamente una decisione di scheduling e un'opportunità di apprendimento.

4.3 Scalarizzazione e selezione della variante

Una volta costruita la frontiera $\mathcal{P}$, il problema si riduce alla scelta del compromesso più appropriato dato il contesto ambientale corrente. La selezione è realizzata tramite una funzione di scalarizzazione lineare pesata da λ , che collassa le due dimensioni in un unico punteggio confrontabile.

4.3.1 Normalizzazione e funzione obiettivo

Energia ed errore sono grandezze eterogenee, espresse in unità di misura diverse e potenzialmente caratterizzate da ordini di grandezza differenti. Per renderle confrontabili, entrambe vengono normalizzate nell'intervallo $[0, 1]$ rispetto al minimo e al massimo osservati sulla frontiera $\mathcal{P}$:

$$\hat{E}(v) = \frac{E(v) - E_{\min}}{E_{\max} - E_{\min}} \quad (4.3.1)$$

$$\widehat{Err}(v) = \frac{Err(v) - Err_{\min}}{Err_{\max} - Err_{\min}} \quad (4.3.2)$$

La normalizzazione è calcolata esclusivamente sulle varianti Pareto-ottimali, in modo che i valori estremi della scala corrispondano sempre ai casi limite effettivamente

raggiungibili lungo la frontiera. La funzione di scalarizzazione lineare è quindi:

$$S(v, \lambda) = (1 - \lambda) \hat{E}(v) + \lambda \widehat{Err}(v) \quad (4.3.3)$$

La variante selezionata v^* è quella che minimizza S sulla frontiera:

$$v^* = \arg \min_{v \in \mathcal{P}} S(v, \lambda) \quad (4.3.4)$$

Per costruzione, v^* appartiene sempre a $\mathcal{P}$ ed è quindi non dominata: nessuna alternativa è contemporaneamente più efficiente e più accurata. I casi limite sono immediati: per $\lambda = 1$ la funzione si riduce a $S = \widehat{Err}(v)$ e viene selezionata la variante più accurata; per $\lambda = 0$ si ha $S = \hat{E}(v)$ e viene selezionata quella a consumo minimo. Per valori intermedi la selezione bilancia le due dimensioni in modo continuo e proporzionale al segnale ambientale.

4.3.2 Analisi delle soglie di transizione

La linearità della funzione di scalarizzazione consente di identificare analiticamente i valori di λ in corrispondenza dei quali la variante selezionata cambia. Sviluppando l'Equazione (4.3.3) si ottiene:

$$S_i(\lambda) = \hat{E}_i + \lambda (\widehat{Err}_i - \hat{E}_i) \quad (4.3.5)$$

Per ogni variante $v_i \in \mathcal{P}$, lo score è una funzione lineare in λ . Nel piano (λ, S) , ciascuna variante è rappresentata da una retta e lo scheduler seleziona, per ogni valore di λ , la variante associata alla retta con valore minimo.

I punti di transizione corrispondono alle intersezioni tra queste rette. Uguagliando gli score di due varianti v_i e v_j si ottiene:

$$\lambda_{ij} = \frac{\hat{E}_j - \hat{E}_i}{(\hat{E}_j - \hat{E}_i) + (\widehat{Err}_i - \widehat{Err}_j)} \quad (4.3.6)$$

Per $\lambda < \lambda_{ij}$ è preferita la variante a minore energia, mentre per $\lambda > \lambda_{ij}$ quella a minore errore.

Ne segue che la variante selezionata varia a tratti costanti al variare di λ , dando luogo a un andamento a gradini: tra due soglie consecutive la scelta rimane invariata, mentre al superamento di una soglia si verifica una transizione verso una variante più accurata. All'aumentare di λ , la selezione si sposta progressivamente lungo la frontiera di Pareto, dalle varianti a minimo consumo verso quelle a minima imprecisione. Il comportamento a gradini che ne deriva verrà illustrato sperimentalmente nel Capitolo 5, dove la selezione lungo la frontiera di Pareto è analizzata su funzioni rappresentative al variare di λ .

4.4 Formalizzazione della logica decisionale

La logica complessiva dello scheduler carbon-aware è formalizzata nell'Algoritmo 2, che integra le fasi descritte nelle sezioni precedenti: valutazione delle varianti con stima LCB, costruzione della frontiera di Pareto, acquisizione del segnale ambientale tramite l'Algoritmo 1 e scalarizzazione finale.

Algoritmo 2 SELECTPARETOVARIANT: selezione carbon-aware

```

1: function SELECTPARETOVARIANT( $r$  : Request)
2:    $base \leftarrow r.Fun$ 
3:    $V \leftarrow \text{GETFUNCTIONSBYLOGICALNAME}(base.LogicalName)$ 
4:   if  $V = \emptyset$  then
5:     return ( $base$ , “fallback-base”)
6:
7:                                      $\triangleright$  Valutazione delle varianti (energia LCB + errore)
8:    $Eval \leftarrow []$ 
9:   for  $v \in V$  do
10:     $warm \leftarrow \text{HASWARMCONTAINER}(v)$ 
11:     $E(v) \leftarrow \text{ENERGYLCB}(v, warm, \beta, k)$ 
12:     $Err(v) \leftarrow \text{ERRORSCORE}(v)$ 
13:     $Eval.APPEND(v, E(v), Err(v))$ 
14:
15:                                      $\triangleright$  Frontiera di Pareto —  $O(n \log n)$ 
16:    $\mathcal{P} \leftarrow \text{PARETOFILTER}(Eval)$ 
17:   if  $|\mathcal{P}| = 1$  then
18:     return ( $\mathcal{P}[0]$ , “pareto-singleton”)
19:
20:                                      $\triangleright$  Peso  $\lambda$  dalla CI corrente — Algoritmo 1
21:    $\lambda \leftarrow \text{GETLAMBDA}(r.Zone)$ 
22:
23:                                      $\triangleright$  Normalizzazione e scalarizzazione
24:    $E_{\min}, E_{\max} \leftarrow \text{MINMAX}([E(v) : v \in \mathcal{P}])$ 
25:    $Err_{\min}, Err_{\max} \leftarrow \text{MINMAX}([Err(v) : v \in \mathcal{P}])$ 
26:    $v^* \leftarrow \arg \min_{v \in \mathcal{P}} [(1 - \lambda) \hat{E}(v) + \lambda \widehat{Err}(v)]$ 
27:   return ( $v^*$ , “pareto-scalarisation”)

```

L'algoritmo evidenzia la separazione netta tra le quattro fasi: valutazione dei profili con stima ottimistica, costruzione della frontiera, acquisizione del segnale ambientale e scalarizzazione. La struttura modulare consente di sostituire ciascun componente indipendentemente dagli altri: il modello di normalizzazione della CI, la funzione di scalarizzazione e il criterio di dominanza di Pareto sono tutti intercambiabili senza modificare l'architettura complessiva. La chiamata GETLAMBDA opera su dati mantenuti in memoria locale (la CI è aggiornata in background ogni 15 minuti) evitando latenze

di rete nel percorso critico della richiesta.

Lo scheduler carbon-aware descritto in questo capitolo integra in un unico meccanismo decisionale le tre famiglie di soluzioni identificate nel Capitolo 2: la selezione tra varianti con diversi profili energia-accuratezza, la sensibilità al contesto carbonico della rete elettrica e l'adattività delle stime energetiche alle condizioni operative correnti. La sigmoide calibrata per zona traduce la carbon intensity in un peso λ contestualizzato rispetto alla distribuzione storica locale; la frontiera di Pareto restringe lo spazio decisionale alle sole varianti non dominate; la scalarizzazione lineare seleziona il compromesso più appropriato in funzione del segnale ambientale. Due casi limite della scalarizzazione — la politica *always-accurate* ($\lambda = 1$) e la politica *always-efficient* ($\lambda = 0$) — costituiranno i termini di confronto nella valutazione sperimentale del Capitolo 5, dove verrà quantificato il beneficio del controllo adattivo rispetto a strategie statiche che ignorano il segnale ambientale.

Capitolo 5

Risultati Sperimentali

Questo capitolo valuta sperimentalmente l'estensione carbon-aware di Serverledge proposta in questa tesi. La valutazione non mira a dimostrare una riduzione generalizzata del consumo energetico: il consumo è considerato una dimensione del trade-off con la qualità del risultato, da modulare in funzione del contesto carbonico dell'esecuzione. L'obiettivo è verificare che lo scheduler carbon-aware traduca la carbon intensity della zona in scelte coerenti lungo la frontiera di Pareto: privilegiare varianti più efficienti quando il costo carbonico dell'energia è elevato, mantenere varianti più accurate quando tale costo è ridotto.

5.1 Setup sperimentale

La valutazione è stata condotta su un'installazione di Serverledge estesa con i componenti descritti nei Capitoli 3 e 4. L'ambiente è composto da tre macchine virtuali ospitate su una macchina fisica equipaggiata con processore Intel Core i5-12400, tutte eseguite su Ubuntu Server 22.04 LTS. La prima macchina virtuale (4 vCPU, 12 GB RAM) ospita il nodo Serverledge insieme all'intero stack di monitoraggio e al registro delle varianti in `etcd`. La seconda (2 vCPU, 4 GB RAM) è dedicata alla generazione del carico, tramite la CLI di Serverledge e gli script di benchmark. La terza (2 vCPU,

4 GB RAM) esegue InfluxDB, separata dal nodo principale per evitare contesa sulle risorse durante la raccolta delle metriche.

5.1.1 Funzioni utilizzate

Le funzioni usate negli esperimenti coprono due famiglie applicative: *funzioni numeriche* e *funzioni di classificazione* (ML). Questa scelta consente di valutare lo scheduler su due modalità di approssimazione strutturalmente diverse, con implicazioni distinte sul comportamento della frontiera di Pareto.

Funzioni numeriche. Le tre funzioni numeriche calcolano grandezze matematiche note tramite serie o metodi iterativi, e la qualità è misurata come errore relativo rispetto al valore esatto. *PiLeibniz* approssima π mediante la serie di Leibniz troncata a un numero variabile di termini; le varianti si distinguono per il numero di iterazioni e per il linguaggio di implementazione (Python e C). *LnHarmonic* stima $\ln 2$ tramite la serie armonica alternata, con un insieme analogo di varianti a granularità crescente. *MonteCarlo* stima π tramite campionamento casuale; in questo caso l'intervallo energetico tra varianti è particolarmente ampio ($1575\times$), poiché il numero di campioni varia su più ordini di grandezza.

Funzioni di classificazione. Le tre funzioni ML eseguono compiti di classificazione su testo o immagini, e la qualità è espressa come `quality_score` normalizzato in $[0, 1]$, derivato dall'accuratezza del modello sul dataset di riferimento. *SentimentAnalysis* classifica il tono di un testo e dispone di quattro modelli: RoBERTa-large, DistilBERT, BERT-tiny e VADER, classificatore a dizionario privo di inferenza neurale. *SpamDetection* individua messaggi indesiderati con un'architettura analoga, con RoBERTa-spam come variante di riferimento e un classificatore a parole chiave come variante minima.

ImageClassification classifica immagini su 1000 categorie ImageNet tramite quattro architetture: ViT-Base, EfficientNet-B0, MobileNet-V2 e MobileNet-V2-Tiny.

Nelle funzioni numeriche la relazione tra numero di iterazioni ed errore è continua: la frontiera di Pareto è densa e lo scheduler può modulare il compromesso energia–qualità in modo graduale. Nelle funzioni ML, invece, ogni variante corrisponde a un modello architetaturalmente distinto: la frontiera è composta da pochi punti discreti e il passaggio a una variante più leggera può comportare un salto qualitativo più marcato. Questa differenza strutturale si riflette direttamente nelle scelte dello scheduler e sarà visibile nei risultati sperimentali.

La Tabella 5.1 riassume il numero di varianti disponibili e l’intervallo di energia di invocazione per ciascuna funzione. Le Figure 5.1 e 5.2 mostrano due casi rappresentativi, uno per famiglia, mettendo in relazione il costo energetico con la misura di qualità usata dallo scheduler.

Tabella 5.1: Funzioni utilizzate nella valutazione sperimentale e profili energetici iniziali delle varianti.

Funzione	Famiglia	Varianti	$E_{\min}$ (J)	$E_{\max}$ (J)
PiLeibniz	Numerica	9	0.000196	0.007823
LnHarmonic	Numerica	7	0.000196	0.007700
MonteCarlo	Numerica	6	0.000147	0.231480
SentimentAnalysis	ML	4	0.002400	0.091579
SpamDetection	ML	4	0.002000	0.085000
ImageClassification	ML	4	0.077000	1.455263

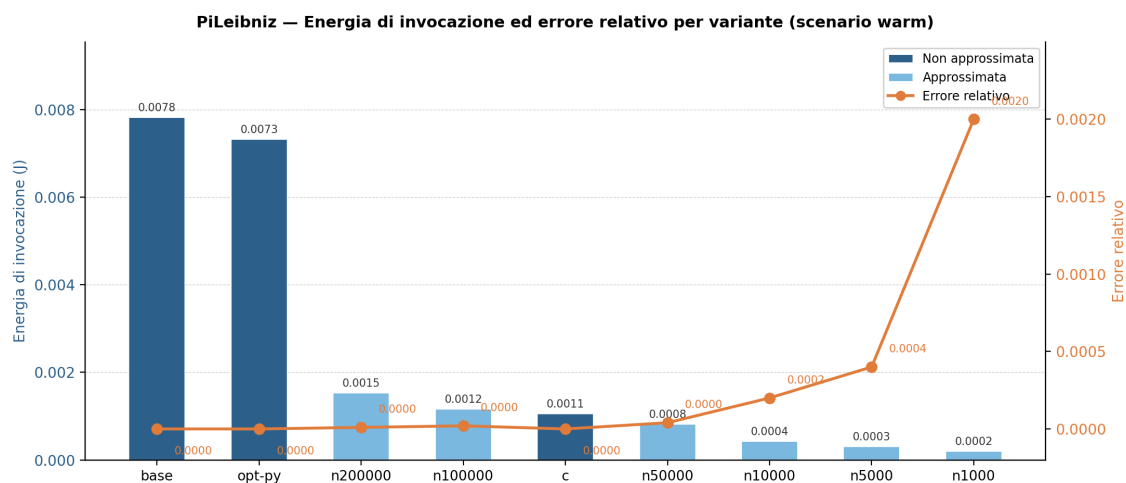


Figura 5.1: Energia di invocazione ed errore relativo delle varianti di PiLeibniz in scenario warm. Le varianti esatte presentano errore nullo, mentre le varianti approssimate riducono progressivamente l’energia diminuendo il numero di termini della serie. L’errore resta contenuto per le varianti intermedie e cresce solo per le approssimazioni più spinte, evidenziando una frontiera energia–accuratezza densa e modulabile.

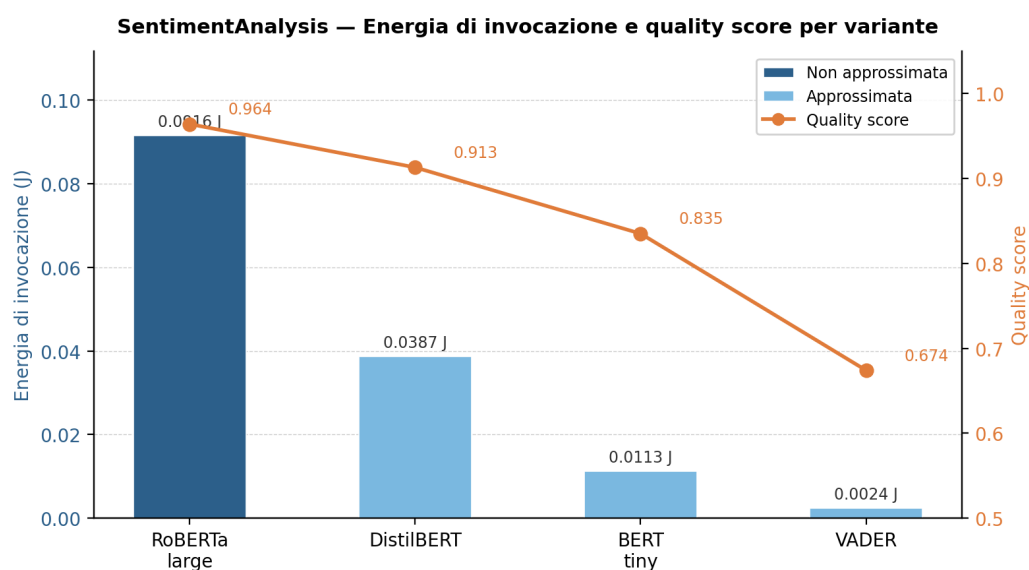


Figura 5.2: Energia di invocazione e `quality_score` delle varianti di SentimentAnalysis. Il passaggio a modelli più leggeri riduce sensibilmente il costo energetico, ma la qualità diminuisce per salti discreti, riflettendo la natura architetturalmente distinta delle varianti ML.

5.1.2 Strategie di confronto

In tutti gli esperimenti vengono confrontate tre strategie. La prima, *always-accurate*, rappresenta il comportamento legacy della piattaforma: viene selezionata sempre la variante con errore minimo o qualità massima, indipendentemente dalle condizioni ambientali. La seconda, *always-energy*, seleziona sempre la variante con minore energia di invocazione, senza considerare la qualità del risultato; questa politica non costituisce un caso operativo realistico, ma fornisce il limite inferiore del consumo ottenibile con il modello di varianti adottato. La terza è lo scheduler carbon-aware oggetto di questa tesi: quando l'utente autorizza l'uso di varianti approssimate tramite il parametro `AllowApprox=true`, il sistema costruisce la frontiera di Pareto nello spazio energia-qualità e seleziona la variante in base alla scalarizzazione descritta nel Capitolo 4.

Il parametro $\lambda \in [0, 1]$ regola il peso relativo tra qualità del risultato e risparmio energetico: valori elevati di λ privilegiano la qualità, valori ridotti favoriscono l'efficienza energetica. Il valore di λ è derivato dalla carbon intensity (CI) della zona di esecuzione tramite una funzione sigmoideale, garantendo che la selezione della variante sia contestuale all'impatto carbonico dell'energia disponibile.

5.2 Protocollo di valutazione

Metriche raccolte. Per ogni invocazione vengono raccolte quattro famiglie di metriche. La prima è l'energia consumata, espressa come valore totale e come media per invocazione. La seconda è l'emissione stimata di CO₂, calcolata moltiplicando l'energia dell'invocazione per la carbon intensity corrente della zona:

$$\text{CO}_2(r) = \frac{E(r)}{3,6 \cdot 10^6} \cdot \text{CI}(r) \quad (5.2.1)$$

dove $E(r)$ è l'energia dell'invocazione r espressa in joule e $CI(r)$ è la carbon intensity corrente in $\text{gCO}_2\text{eq/kWh}$. Il fattore $3,6 \cdot 10^6$ converte l'energia da joule a kWh, poiché $1 \text{ kWh} = 3,6 \cdot 10^6 \text{ J}$. La terza metrica è il tempo medio di esecuzione per invocazione, espresso in millisecondi. La quarta, impiegata in particolare nell'Esperimento 1, è la distribuzione percentuale delle varianti selezionate.

Zone e dati di carbon intensity. Le zone geografiche considerate sono Francia (FR), Spagna (ES), Italia (IT), Germania (DE) e Polonia (PL), ordinate per CI media crescente. La scelta copre un intervallo ampio e rappresentativo del panorama europeo: la Francia rappresenta un regime a bassa CI (prevalente energia nucleare), la Polonia un regime ad alta CI (prevalente energia da carbone), mentre Spagna, Italia e Germania occupano fasce intermedie con mix energetici diversificati. I valori di carbon intensity provengono da serie storiche orarie di ElectricityMaps [6]. La Tabella 5.2 riporta le statistiche sintetiche della traccia impiegata, mentre la Figura 5.3 ne mostra la distribuzione per zona.

Tabella 5.2: Statistiche della carbon intensity nelle zone considerate negli esperimenti.

Zona	CI media (g/kWh)	CI minima (g/kWh)	CI massima (g/kWh)	Campioni
Francia	14.6	3.1	64.3	1152
Spagna	89.3	31.2	222.4	1152
Italia	195.1	66.0	307.2	1152
Germania	271.1	81.9	550.7	1152
Polonia	518.1	257.3	720.6	1152

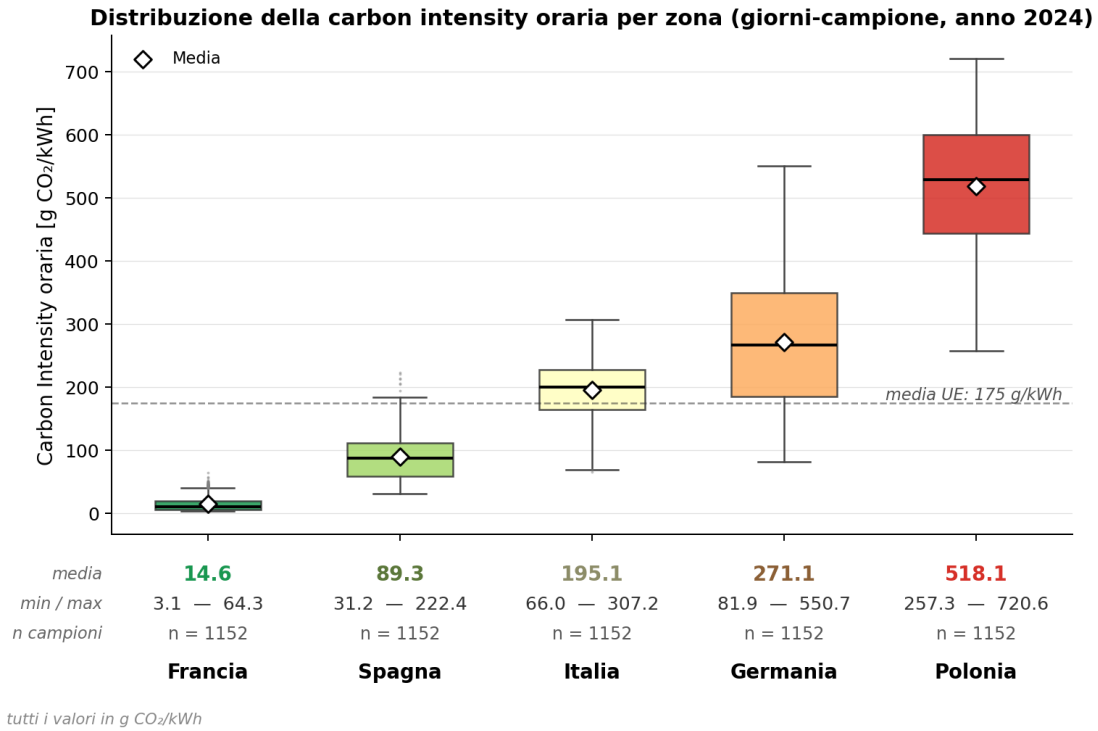


Figura 5.3: Distribuzione della carbon intensity oraria per zona nella traccia usata negli esperimenti. Le zone sono ordinate per CI media crescente da sinistra a destra.

Struttura degli esperimenti. Gli Esperimenti 1 e 2 sono condotti a tempo discreto: una traccia di invocazioni viene riletta da file e ogni richiesta viene valutata rispetto alla CI associata al passo temporale corrente. Questo approccio isola il comportamento decisionale dello scheduler da eventuali variazioni del carico e consente di confrontare due diverse calibrazioni del segnale carbon-aware. L'Esperimento 1 adotta un riferimento europeo comune, utile per leggere tutte le zone sulla stessa scala e rendere direttamente confrontabili le scelte prodotte in reti elettriche diverse. L'Esperimento 2 utilizza invece una calibrazione locale, più aderente a un deployment reale, in cui la CI corrente viene interpretata rispetto alla distribuzione storica della zona. L'Esperimento 3 adotta infine un modello a tempo continuo, con richieste che arrivano secondo un profilo di carico variabile, al fine di valutare la stabilità del sistema e l'overhead della

selezione a runtime.

5.3 Esperimento 1: calibrazione europea della carbon intensity

5.3.1 Obiettivo

Il primo esperimento valuta lo scheduler quando il segnale di carbon intensity viene normalizzato rispetto a un riferimento europeo comune. In questa configurazione tutte le zone condividono la stessa funzione $\lambda(CI)$: una medesima CI assoluta produce quindi lo stesso peso energia–qualità, indipendentemente dalla zona in cui viene osservata. L’obiettivo è analizzare il comportamento di uno scheduler carbon-aware europeo, in cui le reti elettriche vengono confrontate su una scala uniforme.

5.3.2 Configurazione

La normalizzazione del segnale di carbon intensity utilizza una sigmoide calibrata su un riferimento europeo comune, con punto medio fissato al valore medio europeo. In questo modo la medesima funzione $\lambda(CI)$ viene applicata a tutte le zone e le differenze osservate dipendono dalla posizione delle rispettive tracce di CI rispetto alla scala europea. La Figura 5.4 mostra la sigmoide europea e la collocazione delle zone lungo l’asse decisionale.

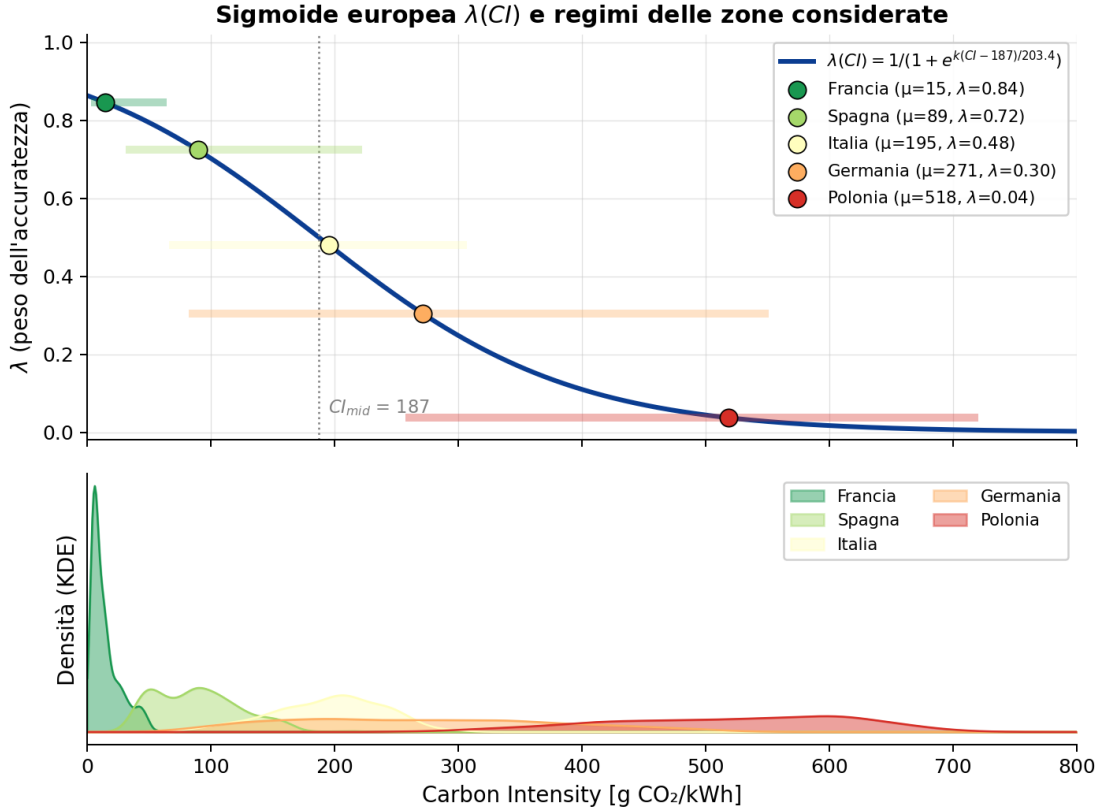


Figura 5.4: Sigmoide europea utilizzata per trasformare la CI nel parametro λ e collocazione delle zone considerate. La funzione mappa le CI più basse verso $\lambda \approx 1$ (preferenza per la qualità) e le CI più alte verso $\lambda \approx 0$ (preferenza per il risparmio energetico).

Per ogni coppia funzione-zona viene eseguita la stessa traccia nei tre regimi *always-accurate*, *always-energy* e *carbon-aware*: 864 invocazioni per coppia nelle funzioni numeriche e 576 nelle funzioni ML.

5.3.3 Risultati

La metrica principale dell'esperimento è la distribuzione delle varianti selezionate. La Figura 5.5 riporta, per ciascuna funzione, la composizione percentuale delle varianti scelte dallo scheduler con calibrazione europea nelle cinque zone, ordinate per CI media crescente.

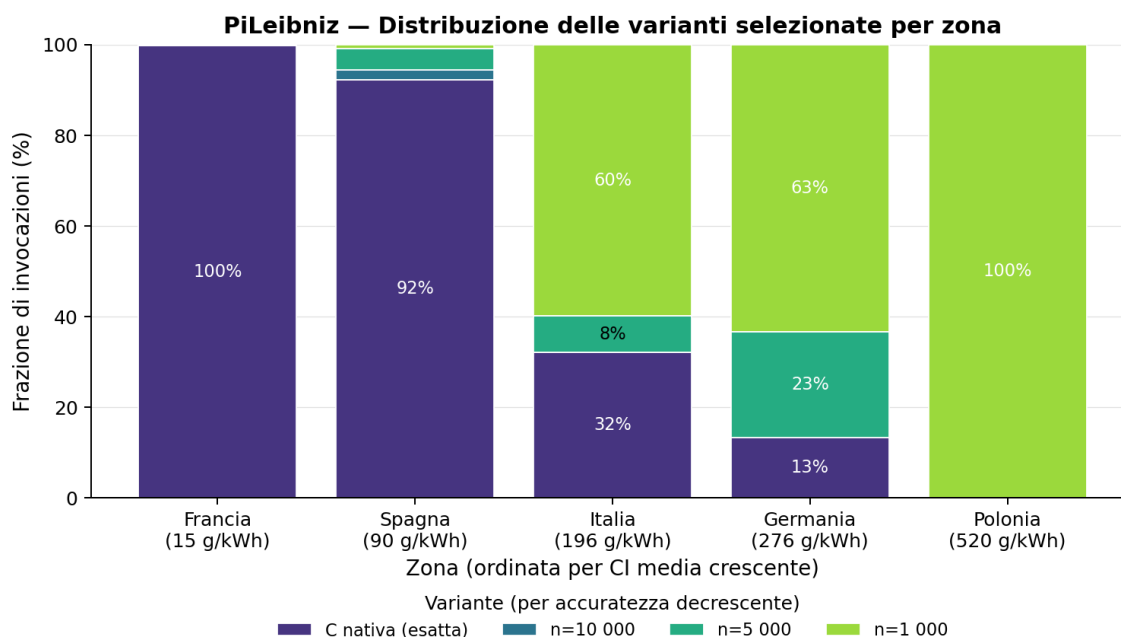


Figura 5.5: Distribuzione delle varianti selezionate per PiLeibniz al variare della zona. Al crescere della CI media, la quota delle varianti più accurate si riduce progressivamente a favore di varianti a minore consumo energetico.

Funzioni numeriche. Per le funzioni numeriche il comportamento è progressivo. In LnHarmonic, Francia e Spagna selezionano la variante più accurata $n100000$ nel 100% delle invocazioni; in Italia la stessa variante resta dominante al 97%, mentre in Germania la quota scende al 54% e in Polonia al 25%. In parallelo cresce la quota di varianti meno costose: $n10000$ raggiunge il 46% in Germania e il 75% in Polonia, mentre $n1000$ compare solo in Polonia con il 3%.

MonteCarlo mostra una transizione più netta: la Francia utilizza $n100000$ nel 100% delle invocazioni, mentre Spagna e Italia passano prevalentemente a $n50000$. In Polonia compaiono anche $n10000$ e $n1000$, con quote rispettivamente del 29% e dell'8%. PiLeibniz presenta la frontiera più articolata: la variante nativa in C domina in Francia, ma già in Spagna vengono selezionate anche $n10000$, $n5000$ e $n1000$. Questo comportamento è coerente con la struttura della frontiera di Pareto della funzione, illustrata in

Figura 5.6: diverse varianti approssimate offrono scambi energia–errore non dominati e possono quindi risultare ottimali per intervalli distinti di λ .

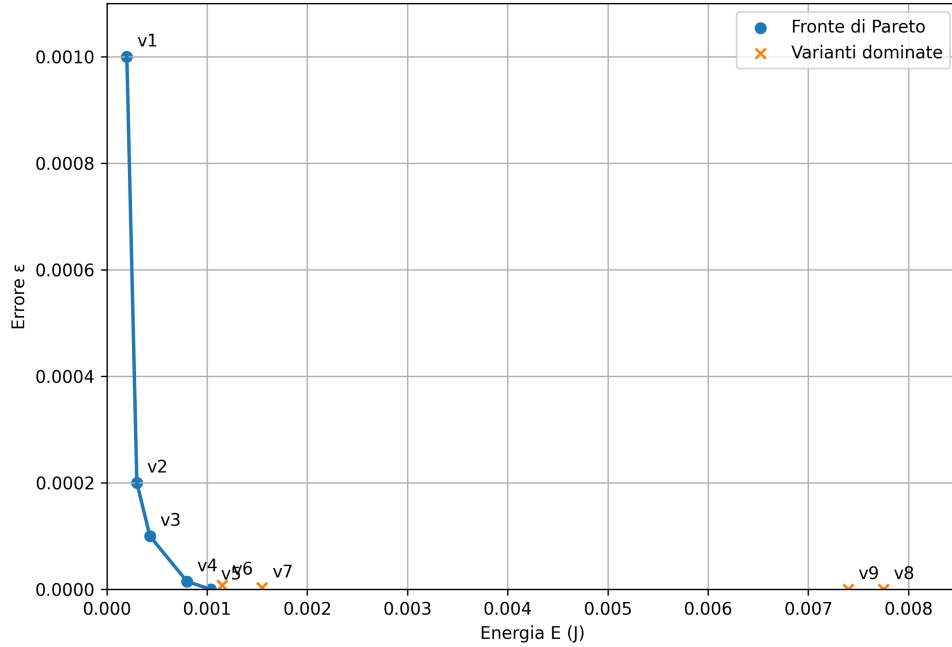


Figura 5.6: Frontiera di Pareto di PiLeibniz nello scenario warm. Le varianti dominate non vengono considerate dallo scheduler. La densità dei punti non dominati determina la granularità con cui è possibile modulare il trade-off energia–qualità.

Funzioni ML. Per le funzioni ML le transizioni sono meno continue, poiché il numero di modelli disponibili è ridotto e ogni modello corrisponde a un salto architetturale discreto. In SentimentAnalysis la Francia seleziona RoBERTa-large, mentre Spagna, Italia e Germania sono dominate da DistilBERT; in Polonia compare VADER, variante più leggera e di conseguenza meno accurata. ImageClassification è il caso in cui la variabilità decisionale è più marcata: nella stessa zona (Germania) vengono selezionate ViT-Base, EfficientNet-B0, MobileNet-V2 e MobileNet-V2-Tiny, a indicare che il valore di λ oscilla intorno a più soglie di transizione. SpamDetection presenta invece una transizione più netta: RoBERTa-spam è prevalente nelle zone a bassa CI, BERT-tiny

nelle zone intermedie, e il classificatore keyword diventa esclusivo in Polonia.

Trade-off aggregato. La Figura 5.7 riassume l'effetto aggregato della selezione mostrando energia media e qualità media per zona. Per MonteCarlo l'energia media passa da 19.4 mJ in Francia a 1.4 mJ in Polonia, con qualità media da 0.995 a 0.986. Per ImageClassification l'energia scende da 294 mJ a 16 mJ, mentre la qualità media passa da 0.82 a 0.61. Il caso di SpamDetection è il più rilevante dal punto di vista qualitativo: in Polonia il classificatore keyword riduce il consumo in modo significativo, ma porta la qualità media al valore più basso tra le funzioni ML, a indicare che il costo del risparmio energetico non è uniforme e dipende dalla struttura della frontiera di Pareto della specifica funzione.

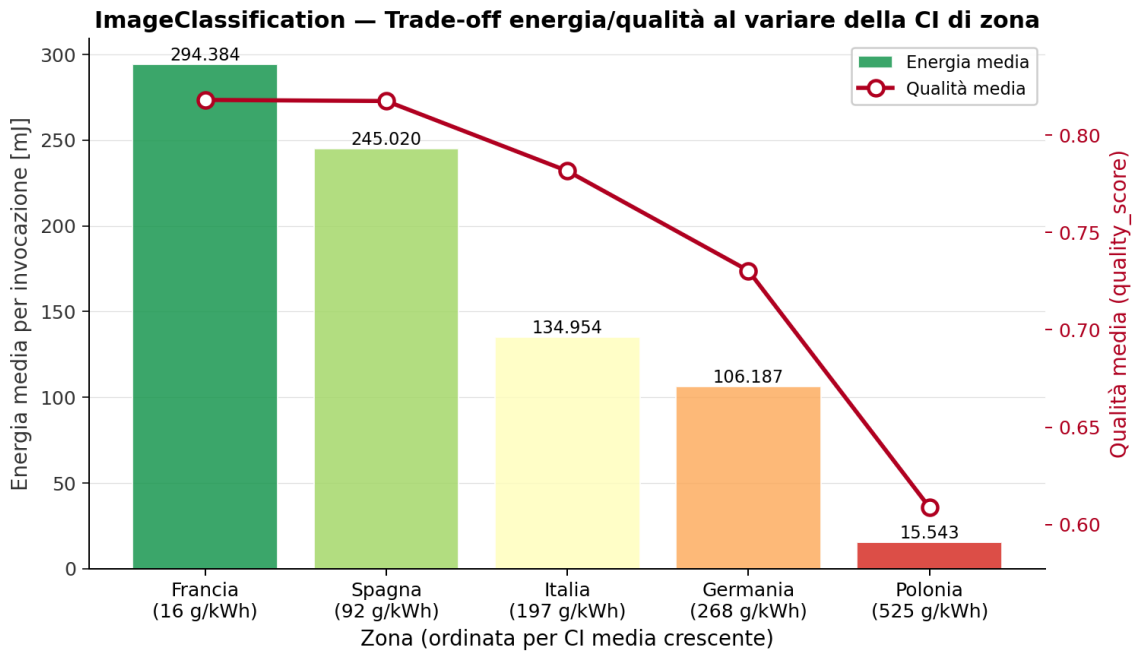


Figura 5.7: Trade-off energia–qualità per ImageClassification nelle zone considerate. La strategia carbon-aware occupa una posizione intermedia tra le due baseline, la cui distanza dal punto *always-accurate* cresce proporzionalmente alla CI media della zona.

Il confronto con la baseline *always-accurate* quantifica il beneficio energetico e il

costo qualitativo dello scheduler. La Tabella 5.3 riporta il risparmio energetico relativo ΔE e la riduzione di qualità relativa ΔQ per funzione e zona. Nelle funzioni numeriche il risparmio cresce con la CI e la perdita di qualità resta inferiore all'1%. Nelle funzioni ML il risparmio può essere più ampio, ma il costo qualitativo dipende dal salto tra modelli adiacenti.

Tabella 5.3: Risparmio energetico e riduzione di qualità rispetto ad *always-accurate* nell'Esperimento 1.

Funzione	Metrica	Francia	Spagna	Italia	Germania	Polonia
LnHarmonic	ΔE	0%	0%	28%	47%	80%
LnHarmonic	ΔQ	0%	0%	0.01%	0.01%	0.02%
MonteCarlo	ΔE	0%	58%	91%	91%	93%
MonteCarlo	ΔQ	0%	0.1%	0.2%	0.2%	0.9%
PiLeibniz	ΔE	0%	-1%	21%	15%	50%
PiLeibniz	ΔQ	0%	0%	0.1%	0.1%	0.2%
SentimentAnalysis	ΔE	0%	100%	99%	99%	99%
SentimentAnalysis	ΔQ	0%	5.3%	5.2%	5.2%	10.8%
ImageClassification	ΔE	0%	1%	46%	60%	94%
ImageClassification	ΔQ	0%	0.1%	4.5%	10.7%	25.6%
SpamDetection	ΔE	0%	5%	65%	73%	96%
SpamDetection	ΔQ	0%	1.8%	25.2%	33.4%	47.4%

5.3.4 Interpretazione

I risultati mostrano l'effetto della calibrazione europea sulle scelte dello scheduler. Poiché tutte le zone sono valutate rispetto alla stessa funzione $\lambda(CI)$, le reti con CI mediamente più bassa, come la Francia, ricadono più spesso in regioni che privilegiano la qualità, mentre le reti con CI più elevata, come la Polonia, ricadono più spesso in regioni che favoriscono il risparmio energetico. L'esperimento non definisce una scelta corretta in senso assoluto, ma descrive il comportamento della politica quando il segnale carbon-aware è interpretato su scala europea.

La Figura 5.8 rende visibile questa interpretazione nello spazio di λ : il range operativo della Polonia si sovrappone prevalentemente alle regioni in cui risultano ottimali le varianti a minore energia, mentre il range della Francia ricade vicino alle regioni di alta qualità. Le differenze tra le funzioni derivano dalla forma della frontiera di Pareto: quando la frontiera è densa, come per alcune funzioni numeriche, il passaggio tra varianti è graduale; quando è composta da pochi modelli distanti, come nelle funzioni ML, i salti di qualità sono più marcati, coerentemente con quanto osservato nella Sezione 5.1.1.

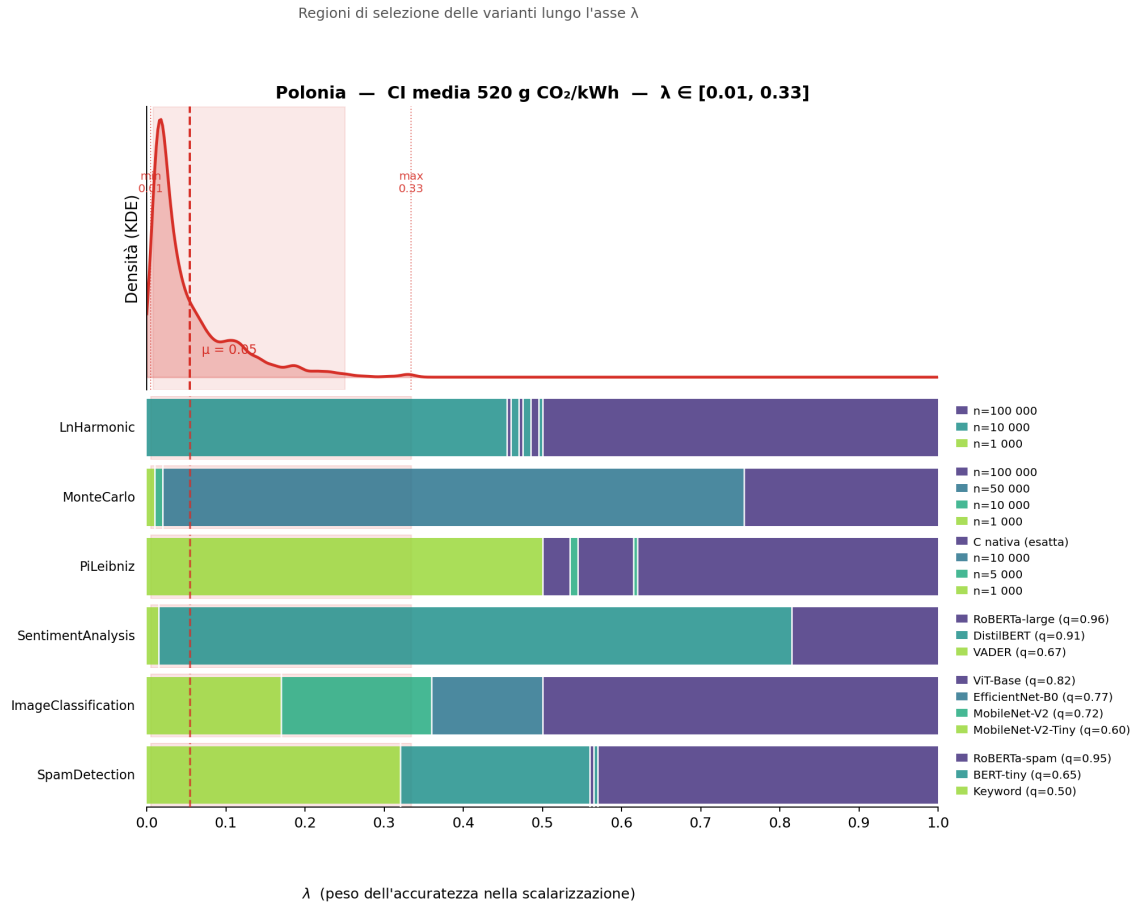


Figura 5.8: Regioni di selezione lungo l'asse λ per la Polonia. La zona opera prevalentemente in intervalli di λ associati a varianti efficienti, coerentemente con la sua carbon intensity media elevata.

5.4 Esperimento 2: calibrazione locale della carbon intensity

5.4.1 Obiettivo

Il secondo esperimento valuta lo stesso scheduler in una configurazione locale. A differenza dell'Esperimento 1, in cui tutte le zone condividono un riferimento europeo, qui la sigmoide è calibrata sulla distribuzione storica di ciascuna zona. Il valore di λ non dipende quindi soltanto dalla CI assoluta, ma anche dalla sua posizione rispetto al regime abituale della rete considerata. Questa configurazione rappresenta il caso più vicino a un deployment reale, in cui un nodo interpreta il segnale carbon-aware rispetto al contesto elettrico in cui opera.

5.4.2 Configurazione

La struttura sperimentale è la stessa dell'Esperimento 1: per ogni zona viene rieseguita la stessa traccia nei tre regimi. Cambia la normalizzazione del segnale ambientale: per ogni zona z , la sigmoide utilizza la media storica locale μ_z e i percentili locali P_5 e P_{95} stimati dalle serie ElectricityMaps [6]. La Figura 5.9 mostra esempi di calibrazione locale per Francia, Italia e Polonia. La stessa CI assoluta può generare valori di λ differenti in zone diverse, poiché rappresenta una condizione ordinaria in una rete e un'anomalia in un'altra: questo aspetto motiva l'adozione di una calibrazione locale rispetto a una soglia globale uniforme.

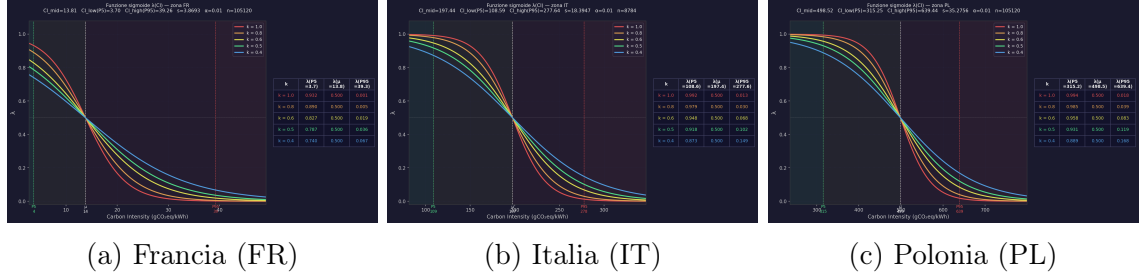


Figura 5.9: Esempi di sigmoide calibrata sulla distribuzione locale della carbon intensity. La posizione del punto di inflessione e la pendenza della curva differiscono tra zone, riflettendo il regime energetico caratteristico di ciascuna rete elettrica.

In questo esperimento l'attenzione si sposta dalle sole quote di selezione alle metriche aggregate: energia totale, emissioni totali e tempo medio di esecuzione. Per ciascuna zona viene calcolato il delta percentuale dello scheduler carbon-aware rispetto ad *always-accurate*:

$$\Delta_{CO_2} = \frac{CO_{2accurate} - CO_{2carbon}}{CO_{2accurate}} \cdot 100 \quad (5.4.1)$$

e il costo qualitativo medio:

$$\Delta_Q = \frac{Q_{accurate} - Q_{carbon}}{Q_{accurate}} \cdot 100. \quad (5.4.2)$$

Questi due indici sono sempre riportati congiuntamente, poiché una riduzione delle emissioni non può essere valutata correttamente senza quantificare il costo qualitativo che la accompagna.

5.4.3 Risultati

La Tabella 5.4 riporta i risultati aggregati per zona. Il confronto deve essere letto per colonna: *always-accurate* fornisce il massimo livello qualitativo, *always-energy* il limite inferiore del consumo, mentre lo scheduler carbon-aware occupa una posizione intermedia determinata dalla calibrazione locale della CI.

Tabella 5.4: Risultati aggregati dell'Esperimento 2 per zona.

Zona	Strategia	Energia (J)	CO ₂ (g)	Tempo medio (ms)	Qualità media	Δ_{CO_2}
Francia	always-accurate	[VALORE]	[VALORE]	[VALORE]	[VALORE]	–
Francia	always-energy	[VALORE]	[VALORE]	[VALORE]	[VALORE]	[VALORE]%
Francia	carbon-aware	[VALORE]	[VALORE]	[VALORE]	[VALORE]	[VALORE]%
Italia	always-accurate	[VALORE]	[VALORE]	[VALORE]	[VALORE]	–
Italia	always-energy	[VALORE]	[VALORE]	[VALORE]	[VALORE]	[VALORE]%
Italia	carbon-aware	[VALORE]	[VALORE]	[VALORE]	[VALORE]	[VALORE]%
Polonia	always-accurate	[VALORE]	[VALORE]	[VALORE]	[VALORE]	–
Polonia	always-energy	[VALORE]	[VALORE]	[VALORE]	[VALORE]	[VALORE]%
Polonia	carbon-aware	[VALORE]	[VALORE]	[VALORE]	[VALORE]	[VALORE]%

La Tabella 5.5 separa il risultato per funzione, al fine di evitare che una media aggregata nasconda comportamenti eterogenei. Le funzioni numeriche tendono a produrre riduzioni energetiche con perdite qualitative molto ridotte, mentre le funzioni ML possono introdurre decrementi di qualità più visibili quando la selezione ricade su modelli di dimensione significativamente inferiore.

Tabella 5.5: Riduzione di emissioni e costo qualitativo dello scheduler carbon-aware rispetto ad *always-accurate* per funzione e zona.

Zona	Funzione	ΔE	Δ_{CO_2}	Δ_Q
Francia	PiLeibniz	[VALORE]%	[VALORE]%	[VALORE]%
Francia	ImageClassification	[VALORE]%	[VALORE]%	[VALORE]%
Italia	PiLeibniz	[VALORE]%	[VALORE]%	[VALORE]%
Italia	ImageClassification	[VALORE]%	[VALORE]%	[VALORE]%
Polonia	PiLeibniz	[VALORE]%	[VALORE]%	[VALORE]%
Polonia	ImageClassification	[VALORE]%	[VALORE]%	[VALORE]%

5.4.4 Interpretazione

L'Esperimento 2 mostra come le decisioni cambiano quando la CI viene interpretata rispetto al contesto locale anziché rispetto a una scala europea uniforme. Il guadagno in emissioni non è gratuito: deriva da una riduzione dell'energia consumata e, quando la struttura della frontiera di Pareto lo richiede, da una riduzione della qualità media. Per

questo motivo il risultato viene sempre espresso come coppia beneficio-costo: riduzione di gCO₂ e perdita di accuratezza o qualità.

La calibrazione locale rende il valore di λ più aderente al contesto della zona. In una rete strutturalmente pulita, una CI assoluta moderata può rappresentare un valore elevato rispetto alla storia locale e giustificare una selezione più efficiente. In una rete strutturalmente carbon-intensive, lo stesso valore assoluto potrebbe essere irraggiungibile o non rappresentativo. L'adozione di una sigmoide calibrata localmente evita che una soglia globale unica produca decisioni eccessivamente aggressive in alcune zone e troppo conservative in altre.

Il risultato è considerato positivo se lo scheduler riduce le emissioni rispetto ad *always-accurate*, si mantiene distante dal comportamento estremo di *always-energy* nei casi in cui il costo qualitativo sarebbe elevato, e mostra una perdita di qualità coerente con il consenso esplicito dell'utente all'uso di varianti approssimate. Questo consenso rimane un vincolo progettuale centrale: in assenza di `AllowApprox=true`, la piattaforma preserva il comportamento legacy.

5.5 Esperimento 3: comportamento con flusso di arrivo variabile

5.5.1 Obiettivo

Il terzo esperimento valuta il comportamento del sistema in condizioni di carico non uniforme nel tempo. Gli Esperimenti 1 e 2 adottano un replay a tempo discreto, utile per confrontare in modo controllato le due calibrazioni del segnale carbon-aware. In un deployment reale, tuttavia, le richieste possono arrivare in burst e lo scheduler deve eseguire, senza introdurre latenza significativa, le seguenti operazioni nel percorso critico dell'invocazione: lettura dei profili energetici da `etcd`, accesso al valore corrente di

λ , costruzione della frontiera di Pareto sul sottoinsieme di varianti disponibili, e valutazione della funzione di scalarizzazione. L'obiettivo è verificare la stabilità del sistema sotto carico variabile e quantificare l'overhead introdotto dallo scheduler rispetto alle baseline.

5.5.2 Configurazione

Il carico viene generato con un profilo composto da tre fasi: una prima fase a bassa intensità (circa 1–2 richieste al secondo), una seconda con un burst sostenuto (circa 8–10 richieste al secondo) e una terza che riduce progressivamente il tasso di arrivo fino al livello iniziale. La CI viene mantenuta fissa per tutta la durata dell'esperimento, utilizzando il valore locale di una zona rappresentativa, al fine di evitare di sovrapporre due sorgenti di variabilità indipendenti.

Vengono confrontate le tre strategie *always-accurate*, *always-energy* e *carbon-aware*. Le metriche principali sono la latenza (percentili p50, p95 e p99), il throughput effettivo, la frazione di risposte non valide e l'energia per invocazione nel tempo. La distribuzione delle varianti viene osservata su finestre temporali per verificare che lo scheduler non modifichi il proprio comportamento in funzione del tasso di arrivo, ma esclusivamente del valore di λ e dei profili energetici correnti.

5.5.3 Risultati

La Tabella 5.6 riporta le metriche di latenza e throughput. L'aspetto centrale è il confronto tra *carbon-aware* e *always-accurate*: se la latenza mediana rimane comparabile e l'incremento di p99 è contenuto, il costo della selezione a runtime può essere considerato trascurabile rispetto al tempo di esecuzione della variante.

Tabella 5.6: Metriche di latenza e throughput nell’Esperimento 3.

Strategia	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Errori 422
always-accurate	[VALORE]	[VALORE]	[VALORE]	[VALORE]	[VALORE]
always-energy	[VALORE]	[VALORE]	[VALORE]	[VALORE]	[VALORE]
carbon-aware	[VALORE]	[VALORE]	[VALORE]	[VALORE]	[VALORE]

La Tabella 5.7 riporta il comportamento energetico durante il test. La metrica più informativa non è soltanto l’energia media finale, ma la sua evoluzione durante le fasi di burst: se le stime EMA rimangono stabili e non oscillano in modo sproporzionato, il sistema dimostra di assorbire il carico senza alterare il profilo decisionale.

Tabella 5.7: Energia per invocazione e distribuzione delle varianti nel test a carico variabile.

Strategia	Energia media (J)	Energia p95 (J)	Variante dominante	Quota variante dom.
always-accurate	[VALORE]	[VALORE]	[VALORE]	[VALORE]%
always-energy	[VALORE]	[VALORE]	[VALORE]	[VALORE]%
carbon-aware	[VALORE]	[VALORE]	[VALORE]	[VALORE]%

5.5.4 Interpretazione

L’overhead introdotto dallo scheduler carbon-aware è composto da operazioni computazionalmente leggere: lettura dei profili energetici da `etcd`, calcolo della frontiera di Pareto su un numero ridotto di varianti, lettura del valore corrente di λ e valutazione della funzione di scalarizzazione. Il valore di CI è aggiornato in background e non richiede una chiamata remota nel percorso critico dell’invocazione. Per questo motivo ci si attende che l’impatto sulla latenza mediana sia limitato.

La coda della distribuzione, in particolare p99, è la metrica più sensibile durante le fasi di burst. Un aumento contenuto di p99 è accettabile se il throughput effettivo segue il tasso di arrivo e non compaiono risposte HTTP 422. Al contrario, una crescita marcata della coda indicherebbe che il collo di bottiglia risiede non nel tempo di esecu-

zione della variante, ma nella capacità del piano di controllo di sostenere la frequenza delle decisioni.

Un secondo elemento da osservare è la coerenza della distribuzione delle varianti. Con CI fissa, la selezione non dovrebbe cambiare sistematicamente tra la fase a basso carico e quella di burst, salvo gli effetti transitori dovuti all'esplorazione LCB nelle prime invocazioni. Quando il numero di campioni cresce, il contributo del termine LCB diminuisce e le stime EMA convergono verso un profilo stabile, garantendo che lo scheduler non confonda la variabilità del carico con una variazione del segnale ambientale.

5.6 Discussione generale

I tre esperimenti rispondono a domande complementari. I primi due confrontano due modi diversi di interpretare il segnale di carbon intensity. La calibrazione europea dell'Esperimento 1 fornisce una scala comune: è utile per confrontare zone diverse senza introdurre soglie specifiche per ciascuna rete e per osservare come la stessa politica si comporta al variare del mix elettrico. La calibrazione locale dell'Esperimento 2 è invece più vicina a un deployment reale: il valore di λ riflette quanto la CI corrente sia alta o bassa rispetto alla storia della zona, evitando che una soglia europea unica produca decisioni non proporzionate in reti strutturalmente molto diverse.

In entrambi i casi lo scheduler occupa una posizione intermedia tra *always-accurate*, che preserva la qualità ignorando il costo ambientale, e *always-energy*, che minimizza il consumo ignorando la qualità. Il terzo esperimento verifica infine che questa selezione possa essere eseguita anche in presenza di carico variabile, senza introdurre un overhead tale da compromettere la latenza del servizio.

Limitazioni. I risultati evidenziano alcune limitazioni del sistema proposto. La prima riguarda le zone con CI prossima al punto medio della sigmoide: in tali condizioni, piccole variazioni della CI possono attraversare soglie di selezione e produrre cambi più frequenti di variante. La seconda riguarda le funzioni con frontiera di Pareto poco estesa: quando sono disponibili solo pochi punti non dominati, lo scheduler ha meno possibilità di modulare gradualmente il compromesso e i salti di qualità possono essere più marcati. La terza riguarda i burst molto intensi: il sistema resta vincolato alla capacità del piano di controllo, alla latenza di `etcd` e alla frequenza con cui le metriche energetiche vengono aggiornate.

Sintesi. Nel complesso, la valutazione mostra che l'estensione proposta consente di collocare Serverledge tra due estremi operativi ben definiti: *always-accurate*, che massimizza la qualità ignorando il costo ambientale, e *always-energy*, che minimizza il consumo ignorando la qualità. Lo scheduler carbon-aware occupa una regione intermedia e variabile dello spazio di trade-off, determinata dalla carbon intensity della zona e vincolata al consenso esplicito dell'utente. La riduzione delle emissioni non è ottenuta tramite una degradazione indiscriminata, ma attraverso una scelta contestuale e misurabile lungo la frontiera di Pareto, coerente con il segnale ambientale osservato a runtime.

Capitolo 6

Conclusioni

Elenco delle figure

2.1	Esempio di domini di potenza definiti dal modello RAPL. Il dominio <i>Package</i> rappresenta il consumo energetico complessivo della CPU, mentre domini più specifici, come <i>PP0</i> e <i>DRAM</i> , forniscono stime riferite rispettivamente ai core di calcolo e alla memoria principale.	22
2.2	Architettura concettuale di PowerAPI. I contatori di prestazione hardware e le misure energetiche aggregate (RAPL) vengono combinati dal modello SmartWatts per stimare il consumo energetico dei singoli processi.	25
2.3	Architettura di Kepler.	26
2.4	Architettura di E ² FIS. I nodi edge sono caratterizzati da differenti capacità computazionali e profili di consumo. Il modulo di <i>Scheduling computation</i> determina periodicamente l’allocazione delle funzioni e il sottoinsieme di nodi da mantenere attivi.	31
2.5	Architettura di alto livello di GreenFaaS: monitoraggio degli endpoint, stima del costo energetico dei trasferimenti e scheduling energy-aware per l’allocazione dei task in ambienti federati eterogenei.	33

2.6	Pipeline di traduzione automatica proposta in <i>Code Once, Run Green</i> . La trasformazione avviene in fase di build tramite un processo iterativo di conversione e validazione del codice, con passaggi di recupero in caso di errori di compilazione o di test.	38
2.7	Schema concettuale di JouleGuard. Il <i>System Energy Optimizer</i> (SEO) identifica la configurazione di sistema più efficiente; l' <i>Application Accuracy Optimizer</i> (AAO) regola il livello di approssimazione dell'applicazione per rispettare il budget energetico, massimizzando l'accuratezza. .	40
2.8	Effetto della Virtual Capacity Curve (VCC) sul profilo di carico di un cluster. La barra superiore indica i livelli di carbon intensity nel corso della giornata. La curva VCC (in rosso) riduce la capacità disponibile per i workload flessibili nelle ore ad alta carbon intensity, spostando il carico (aree arancione e blu) verso le ore più pulite della giornata. Il carico inflexible (in viola) non viene alterato.	45
2.9	Architettura di GreenCourier. Il management cluster ospita lo scheduler carbon-aware e il Metrics Server, che interroga sorgenti di carbon intensity (WattTime, ElectricityMaps). I provider cluster, distribuiti in diverse regioni europee, sono collegati tramite Ligo come nodi virtuali. Lo scheduler assegna ciascuna invocazione alla regione con la carbon intensity più favorevole.	48
3.1	Architettura di Kepler per la raccolta dei consumi energetici in ambienti bare-metal e virtuali. In ambienti bare-metal, Kepler accede direttamente ai contatori hardware tramite RAPL, ottenendo misurazioni real-time senza necessità di modelli predittivi.	55

3.2	Architettura dell'infrastruttura di monitoraggio energetico e scheduling carbon-aware. I componenti di monitoraggio esterni (Kepler, Prometheus, Grafana) sono separati dagli store interni (etcd, InfluxDB), con Serverledge al centro che interagisce con entrambi i livelli. Lo scheduler carbon-aware integra la logica di selezione della variante — basata su frontiera di Pareto, scalarizzazione tramite λ , esplorazione LCB e consenso utente — e riceve il segnale di carbon intensity tramite query periodica a una sorgente esterna.	56
3.3	Struttura delle chiavi etcd per la funzione logica PiLeibniz. Le chiavi sotto <code>/function/</code> contengono i profili completi; l'indice sotto <code>/index/logical/</code> consente la lookup efficiente delle sole varianti richieste.	65
3.4	Schema del feedback loop energetico. Il worker periodico legge le metriche cumulative da Kepler tramite Prometheus e il numero di invocazioni dal contatore locale, calcola i delta, applica un filtro EMA e aggiorna il profilo persistito in etcd, archiviando contestualmente i campioni in InfluxDB.	66
5.1	Energia di invocazione ed errore relativo delle varianti di PiLeibniz in scenario warm. Le varianti esatte presentano errore nullo, mentre le varianti approssimate riducono progressivamente l'energia diminuendo il numero di termini della serie. L'errore resta contenuto per le varianti intermedie e cresce solo per le approssimazioni più spinte, evidenziando una frontiera energia-accuratezza densa e modulabile.	86

5.2	Energia di invocazione e <code>quality_score</code> delle varianti di SentimentAnalysis. Il passaggio a modelli più leggeri riduce sensibilmente il costo energetico, ma la qualità diminuisce per salti discreti, riflettendo la natura architetaturalmente distinta delle varianti ML.	86
5.3	Distribuzione della carbon intensity oraria per zona nella traccia usata negli esperimenti. Le zone sono ordinate per CI media crescente da sinistra a destra.	89
5.4	Sigmoide europea utilizzata per trasformare la CI nel parametro λ e collocazione delle zone considerate. La funzione mappa le CI più basse verso $\lambda \approx 1$ (preferenza per la qualità) e le CI più alte verso $\lambda \approx 0$ (preferenza per il risparmio energetico).	91
5.5	Distribuzione delle varianti selezionate per PiLeibniz al variare della zona. Al crescere della CI media, la quota delle varianti più accurate si riduce progressivamente a favore di varianti a minore consumo energetico.	92
5.6	Frontiera di Pareto di PiLeibniz nello scenario warm. Le varianti dominate non vengono considerate dallo scheduler. La densità dei punti non dominati determina la granularità con cui è possibile modulare il trade-off energia-qualità.	93
5.7	Trade-off energia-qualità per ImageClassification nelle zone considerate. La strategia carbon-aware occupa una posizione intermedia tra le due baseline, la cui distanza dal punto <i>always-accurate</i> cresce proporzionalmente alla CI media della zona.	94
5.8	Regioni di selezione lungo l'asse λ per la Polonia. La zona opera prevalentemente in intervalli di λ associati a varianti efficienti, coerentemente con la sua carbon intensity media elevata.	96

5.9	Esempi di sigmoide calibrata sulla distribuzione locale della carbon intensity. La posizione del punto di inflessione e la pendenza della curva differiscono tra zone, riflettendo il regime energetico caratteristico di ciascuna rete elettrica.	98
-----	--	----

Elenco dei codici

- 3.1 File di definizione delle varianti per la funzione `PiLeibniz`. Le prime due varianti sono esatte (`is_approximate: false`) e differiscono per runtime e consumo. (omesse per brevità le quattro varianti intermedie). 59
- 3.2 File di definizione delle varianti per la funzione `ImageClassification`. Il campo `quality_score` riporta la Top-1 accuracy su ImageNet pubblicata dagli autori di ciascun modello, espressa in $[0, 1]$ 62

List of Algorithms

1	Calcolo del peso λ dalla carbon intensity corrente	76
2	SELECTPARETOVARIANT: selezione carbon-aware	81

Bibliografia

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [2] E. van Eyk, L. Toader, S. Talluri, L. Versluis, A. Uta, and A. Iosup, “Serverless is more: From PaaS to present cloud computing,” *IEEE Internet Computing*, vol. 22, no. 5, pp. 8–17, 2018.
- [3] G. Russo, T. Mannucci, V. Cardellini, and F. Lo Presti, “Serverledge: Decentralized function-as-a-service for the edge-cloud continuum,” in *2023 IEEE International Conference on Pervasive Computing and Communications (PerCom)*, 2023, pp. 131–140.
- [4] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, “Resource scheduling in edge computing: A survey,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2131–2165, 2021.
- [5] International Energy Agency, “Electricity 2025 – emissions,” 2025, consultato: aprile 2026. [Online]. Available: <https://www.iea.org/reports/electricity-2025/emissions>
- [6] Electricity Maps, “Electricity maps — historical carbon intensity data,” <https://www.electricitymaps.com>, 2024, accessed: 2025.

- [7] H. Hoffmann, “Jouleguard: energy guarantees for approximate applications,” in *Proceedings of the 25th Symposium on Operating Systems Principles*, ser. SO-SP '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 198–214.
- [8] M. Amaral *et al.*, “Kepler: A framework to calculate the energy consumption of containerized applications,” in *Proceedings of IEEE CLOUD 2023*, 2023. [Online]. Available: <https://research.ibm.com/publications/kepler-a-framework-to-calculate-the-energy-consumption-of-containerized-applications>
- [9] M. Lipp, A. Kogler, D. Oswald, M. Schwarz, C. Easdon, C. Canella, and D. Gruss, “PLATYPUS: Software-based Power Side-Channel Attacks on x86,” in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021.
- [10] G. Fieni, R. Rouvoy, and L. Seinturier, “SmartWatts: Self-Calibrating Software-Defined Power Meter for Containers,” in *CCGRID 2020 - 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing*, Melbourne, Australia, May 2020. [Online]. Available: <https://inria.hal.science/hal-02470128>
- [11] D. Freina, M. Jansen, and A. Trivedi, “A survey of energy measurement methodologies for computer systems,” 2024.
- [12] Intel Corporation, “Intel running average power limit (rapl),” <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/running-average-power-limit-energy-reporting.html>, 2023.
- [13] V. Ostapenco, L. Lefèvre, A.-C. Orgerie, and B. Fichel, “Exploring rapl as a power capping leverage for power-constrained infrastructures,” in *Algorithms and Archi-*

- lectures for Parallel Processing*, T. Zhu, J. Li, and A. Castiglione, Eds. Singapore: Springer Nature Singapore, 2025, pp. 323–333.
- [14] M. Hähnel, B. Döbel, M. Völpl, and H. Härtig, “Measuring energy consumption for short code paths using rapl,” in *ACM SIGMETRICS Performance Evaluation Review*. New York, NY, USA: Association for Computing Machinery, 2012, p. 84–90. [Online]. Available: <https://doi.org/10.1145/2425248.2425252>
- [15] PowerAPI Project. (2023) Hwpc sensor – powerapi documentation. [Online]. Available: https://powerapi.org/reference/overview/#usage_1
- [16] ——. (2023) Smartwatts energy model. [Online]. Available: <https://powerapi.org/reference/formulas/smartwatts/>
- [17] V. Shponarskyi, “Monitoring environmental impact of kubernetes clusters,” <https://medium.com/@vasyl.shponarskyi/monitoring-environmental-impact-of-kubernetes-clusters-8c81f72ac55f>, 2022.
- [18] Red Hat, “Introducing kepler: Efficient power monitoring for kubernetes,” <https://next.redhat.com/2023/08/22/introducing-kepler-efficient-power-monitoring-for-kubernetes/>, 2023.
- [19] F. Righetti, B. Cornacchia, G. R. Russo, N. Tonellotto, V. Cardellini, and C. Valati, “Energy-efficient function invocation scheduling for edge faas platforms,” in *2025 IEEE International Conference on Smart Computing (SMARTCOMP)*, 2025, pp. 18–25.
- [20] A. Kamatar, V. Hayot-Sasson, Y. Babuji, A. Bauer, G. Rattihalli, N. Hogade, D. Milojevic, K. Chard, and I. Foster, “Greenfaas: Maximizing

- energy efficiency of hpc workloads with faas,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.17710>
- [21] S. Mittal, “A survey of techniques for approximate computing,” *ACM Computing Surveys*, vol. 48, no. 4, pp. 1–33, 2016.
- [22] S. Werner, M. Kähler, and A. Hakamian, “Code once, run green: Automated green code translation in serverless computing,” in *2025 IEEE International Conference on Cloud Engineering (IC2E)*, 2025, pp. 105–113.
- [23] A. Radovanović, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D. Xiao, M. Haridasan, P. Hung, N. Care, S. Talukdar, E. Mullen, K. Smith, M. Cottman, and W. Cirne, “Carbon-aware computing for datacenters,” *IEEE Transactions on Power Systems*, 2022.
- [24] M. Chadha, T. Subramanian, E. Arima, M. Gerndt, M. Schulz, and O. Abboud, “GreenCourier: Carbon-aware scheduling for serverless functions,” in *9th International Workshop on Serverless Computing (WoSC ’23)*. ACM, 2023, pp. 18–23.
- [25] M. Amaral, S. Choochotkaew, E. K. Lee, H. Chen, and T. Eilam, “Exploring Kepler’s potentials: unveiling cloud application power consumption,” 2023, cNCF Blog. [Online]. Available: <https://www.cncf.io/blog/2023/10/11/exploring-keplers-potentials-unveiling-cloud-application-power-consumption/>
- [26] P. Auer, N. Cesa-Bianchi, and P. Fischer, “Finite-time analysis of the multiarmed bandit problem,” *Machine Learning*, vol. 47, no. 2–3, pp. 235–256, 2002.